



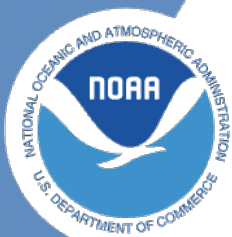
NOAA Technical Memorandum NMFS-XXX-##

GAP Production Data Documentation

Bering Sea Survey Team, Gulf of
Alaska Survey Team, Aleutian Islands
Survey Team

U.S. DEPARTMENT OF COMMERCE

National Oceanic and Atmospheric
Administration
National Marine Fisheries Service
Northwest Fisheries Science Center



**NOAA
FISHERIES**

GAP Production Data Documentation

Bering Sea Survey Team^{1,*}, Gulf of
Alaska Survey Team^{1,*} and Aleutian
Islands Survey Team^{1,*}

1. NOAA Fisheries Alaska Fisheries Science Center, Groundfish Assessment Program

** Correspondence:* Bering Sea Survey Team nmfs.afsc.gap.metadata@noaa.gov ** Correspondence:* Gulf of Alaska Survey Team nmfs.afsc.gap.metadata@noaa.gov ** Correspondence:* Aleutian Islands Survey Team nmfs.afsc.gap.metadata@noaa.gov

Table of contents

I. Welcome	1
AFSC Bottom Trawl Surveys	2
Documentation Objective	3
User Resources	3
Cite this data	3
Access Constraints	5
Suggestions and comments	5
NOAA README	5
NOAA License	5
1. Survey Background	7
1.1. What we do	7
1.2. Who is conducting the research?	7
1.3. What is the research objective?	7
1.4. Who is conducting the research?	7
1.5. Bottom trawl surveys and regions	8
2. Workflow	15
2.1. Operational Product Development Timeline	15
2.2. Data workflow from boat to production	16
2.3. Data levels	19
3. News	23
3.1. News/change logs	23
4. Code of Conduct	26
4.1. What are Codes of Conduct?	26
5. NOAA Fisheries Open Science Code of Conduct	27
5.1. Our Pledge	27

Table of contents

5.2. Our Standards	27
5.3. Our Responsibilities	28
5.4. Scope	28
5.5. Enforcement	28
5.6. Attribution	29
 II. GAP Production Data	 30
Data Description	31
gapindex	31
Cite this data	31
The Alaska Fisheries Information Network	32
Fisheries One Stop Shop data platform	34
 6. Data Description	 37
6.1. Data tables	37
 7. Universal Column Metadata	 77
 8. Access data via Oracle and R	 170
Access data via Oracle (AFSC only)	170
Data SQL Query Examples:	170
 9. Access API data via R	 201
9.1. Ex. Direct database query in R using the akfingapdata R package README:	201
 10. Using the FOSS platform	 202
10.1. Select and filter	202
10.2. Search options	204
10.3. Run report	205
10.4. API	206
 11. Use data	 207
 12. Access via API and R	 208
12.1. Ex. Load all rows of the catch, haul, and species data tables	208
12.2. Ex. Create zero-filled data using data loaded in last example	213
12.3. Ex. Visualize zero-filled data for 2023 eastern Bering Sea walleye pollock in CPUE data in distribution map	214
12.4. Ex. Show catch data for 2023 eastern Bering Sea Walleye Pollock (one species in one survey region in one year)	219
12.5. Plot locations	223

Table of contents

13. Access via API and Python	225
III. Data Products & Tools	230
14. Open source code	234
14.1. R Packages	234
IV. Contact us	235
This code is primarily maintained by:	236
15. Production run notes	237
16. R Version Metadata	238
17. Acknowledgments	240
18. Community Acknowledgments	241
19. Land Acknowledgements	242
20. Technical Acknowledgments	243
20.1. Partners	243
20.2. Collaborators	243
21. Citations and References	244
22. Access Constraints	245
23. References	246

List of Figures

1. Sorting and weighing fish on deck on the 2022 Bering Sea groundfish survey aboard the F/V Alaska Knight. Credit: Emily Markowitz/NOAA Fisheries.	2
1.1. Strata used in the all surveys.	8
1.2. Strata used in the Aleutian Islands bottom trawl survey.	10
1.3. Strata used in the Gulf of Alaska bottom trawl survey.	11
1.4. Strata used in the Eastern Bering Sea bottom trawl survey.	12
1.5. Strata used in the Northern Bering Sea bottom trawl survey.	13
1.6. Strata used in the Bering Sea Slope bottom trawl survey.	14
2.1. Simplified boat deck processing workflow.	17
2.2. Simplified data workflow from boat to production.	20
2.3. Major end-users of the GAP data product tables.	21
5.1. AKFIN platfrom.	33
8.1. EBS Pacific Ocean perch CPUE and <code>akgfm</code> maps map.	179
8.2. GOA Pacific Ocean perch biomass and abundance.	182
8.3. AI Rock sole size compositions and ridge plot.	185
8.4. 2023 EBS Walleye Pollock Age Compositions and Age Pyramid.	188
8.5. NBS Pacific cod biomass and abundance.	192
8.6. GOA Pacific Ocean perch biomass and line plot.	195
10.1. AFSC Groundfish and Crab Assessment Program Bottom Trawl Survey data interface on the Fisheries One Stop Shop platform.	202
10.2. Catch data on the AFSC Groundfish and Crab Assessment Program Bottom Trawl Survey data interface on the Fisheries One Stop Shop platform.	203
10.3. Haul data on the AFSC Groundfish and Crab Assessment Program Bottom Trawl Survey data interface on the Fisheries One Stop Shop platform.	203
10.4. All species observed by survey on the AFSC Groundfish and Crab Assessment Program Bottom Trawl Survey data interface on the Fisheries One Stop Shop platform.	204
10.5. Diagram of selection and search tools available on the FOSS platfrom.	205

List of Figures

10.6.Example data returned from running the report.	206
---	-----

List of Tables

1.1. Survey summary stats	9
2.1. Operational product development timeline.	15
7.1. Universal stock metadata that users can use to document their table columns.	78
8.1. CPUE for all EBS and NBS stations with associated haul, cruise, and species information.	174
8.2. CPUE for all stations contained in the Shumagin region (AREA_ID = 919).	176
8.3. EBS Pacific Ocean perch CPUE and <code>akgfm</code> map.	177
8.4. GOA Pacific Ocean perch biomass and abundance.	181
8.5. AI Rock sole size compositions and ridge plot.	183
8.6. EBS Walleye Pollock Age Compositions and Age Pyramid.	186
8.7. NBS Pacific cod biomass and abundance.	189
8.8. GOA Pacific Ocean perch biomass and line plot.	193
8.9. 2022 AI Atka mackerel age specimen summary: all ages determined.	196
8.10.Ex.: 2022 AI Atka mackerel age specimen summary: how many of each age were determined.	197
8.11.2022 AI Atka mackerel age specimen summary: how many otoliths were aged. This query was created using SQL.	200
13.2.Survey of products developed by GAP	231

Part I.

Welcome

AFSC Bottom Trawl Surveys

Report run date: Tuesday, July 07, 2026

AFSC Bottom Trawl Surveys

AFSC bottom trawl surveys are conducted by the AFSC's Groundfish Assessment Program and Shellfish Assessment Program and are conducted in the Gulf of Alaska, Aleutian Islands, Eastern Bering Sea Slope, Eastern Bering Sea Shelf, and Northern Bering Sea. Each survey is a multispecies survey that collects data on the distribution, abundance, and biological characteristics of fish, crab, and other resources to inform groundfish stock assessment and management. These fishery-independent surveys are conducted in the summer aboard contracted commercial fishing vessels. Specifics regarding each of the surveys can be found below.



Figure 1.: Sorting and weighing fish on deck on the 2022 Bering Sea groundfish survey aboard the F/V Alaska Knight. Credit: Emily Markowitz/NOAA Fisheries.

Documentation Objective

As part of our commitment to open science, reproducibility, and transparency, we provide this metadata guide to compliment our public-domain data.

Please consider this resource to be a **Living Document**. The code in this repository is regularly being updated and improved. Please refer to releases for finalized products and project milestones.

At this time, these master production and AKFIN tables are **provisional and we are welcoming feedback before the 2024 survey season**. We look forward to hearing from you. Do not hesitate to reach out (to us at either nmfs.afsc.gap.metadata@noaa.gov or GitHub issues, especially if you find discrepancies in the data or want to suggest improvements to infrastructure. Thank you in advance for your collaboration and partnership with us as we develop our future data universe.

User Resources

- Groundfish Assessment Program Bottom Trawl Surveys
- AFSC's Resource Assessment and Conservation Engineering Division
- All AFSC Research Surveys
- Survey code books
- Publications and Data Reports
- Research Surveys conducted at AFSC

Cite this data

Use the below bibtext citations, as cited in our group's citation repository for citing the data created and maintained in this repo. Add "note = {Accessed: mm/dd/yyyy}" to append the day this data was accessed. Included here are AFSC RACE Groundfish and Shellfish Assessment Program's:

- Design-Based Production Data (internal) (NOAA Fisheries Alaska Fisheries Science Center, Goundfish Assessment Program, 2024).

Cite this data

- AFSC RACE Groundfish Data for AKFIN (Alaska Fisheries Information Network (AKFIN), 2024).
- Public Data hosted on the Fisheries One Stop Shop (FOSS) Data Platform (NOAA Fisheries Alaska Fisheries Science Center, 2024).

```
@misc{GAPProducts,  
  author = {{NOAA Fisheries Alaska Fisheries Science Center, Goundfish Assessment Program}},  
  year = {2023},  
  title = {AFSC Goundfish Assessment Program Design-Based Production Data},  
  howpublished = {https://www.fisheries.noaa.gov/alaska/science-data/groundfish-  
assessment-program-bottom-trawl-surveys},  
  publisher = {{U.S. Dep. Commer.}},  
  copyright = {Public Domain}  
}
```

```
@misc{FOSSAFSCData,  
  author = {{NOAA Fisheries Alaska Fisheries Science Center}},  
  year = {2023},  
  title = {Fisheries One Stop Shop Public Data: RACE Division Bottom Trawl Survey Data Quer},  
  howpublished = {https://www.fisheries.noaa.gov/foss},  
  publisher = {{U.S. Dep. Commer.}},  
  copyright = {Public Domain}  
}
```

```
@misc{GAPakfin,  
  author = {{Alaska Fisheries Information Network (AKFIN)}},  
  institution = {{NOAA Fisheries Alaska Fisheries Science Center, Goundfish Assessment Prog}},  
  year = {2023},  
  title = {AFSC Goundfish Assessment Program Design-Based Production Data},  
  howpublished = {https://www.psmfc.org/program/alaska-fisheries-information-  
network-akfin},  
  publisher = {{U.S. Dep. Commer.}},  
  copyright = {Public Domain}  
}
```


Access Constraints

There are no legal restrictions on access to the data. They reside in public domain and can be freely distributed.

User Constraints: Users must read and fully comprehend the metadata and code of conduct prior to use. Data should not be used beyond the limits of the source scale. Acknowledgement of AFSC Groundfish Assessment Program, as the source from which these data were obtained, in any publications and/or other representations of these data, is suggested.

Suggestions and comments

If the data or metadata can be improved, please create a pull request, submit an issue to the GitHub organization or submit an issue to the code's repository.

NOAA README

This repository is a scientific product and is not official communication of the National Oceanic and Atmospheric Administration, or the United States Department of Commerce. All NOAA GitHub project code is provided on an 'as is' basis and the user assumes responsibility for its use. Any claims against the Department of Commerce or Department of Commerce bureaus stemming from the use of this GitHub project will be governed by all applicable Federal law. Any reference to specific commercial products, processes, or services by service mark, trademark, manufacturer, or otherwise, does not constitute or imply their endorsement, recommendation or favoring by the Department of Commerce. The Department of Commerce seal and logo, or the seal and logo of a DOC bureau, shall not be used in any manner to imply endorsement of any commercial product or activity by DOC or the United States Government.

NOAA License

Software code created by U.S. Government employees is not subject to copyright in the United States (17 U.S.C. §105). The United States/Department of Commerce reserve all rights to seek and obtain copyright protection in countries other than the United States

NOAA License

for Software authored in its entirety by the Department of Commerce. To this end, the Department of Commerce hereby grants to Recipient a royalty-free, nonexclusive license to use, copy, and create derivative works of the Software outside of the United States.

1. Survey Background

1.1. What we do

1.2. Who is conducting the research?

Scientists from the Alaska Fisheries Science Center's Groundfish Assessment Program (GAP) conduct these bottom trawl surveys with participation from the Alaska Department of Fish & Game (ADF&G), the International Pacific Halibut Commission (IPHC), universities, and other organizations. This research is conducted primarily on chartered fishing vessels.

1.3. What is the research objective?

Learn more about the program. The objectives of these surveys are to:

- monitor the population and environmental trends in the marine ecosystem of the Bering Sea, Aleutian Islands, and Gulf of Alaska,
- produce fishery-independent biomass (weight) and abundance (number) estimates for commercially important fish and crab species, and
- collect other biological and environmental data for use in ecosystem-based fishery management.

1.4. Who is conducting the research?

Scientists from the Alaska Fisheries Science Center conduct these bottom trawl surveys with participation from the Alaska Department of Fish & Game (ADF&G), the International Pacific Halibut Commission (IPHC), and universities. This research is conducted on chartered fishing vessels.

1. Survey Background

1.5. Bottom trawl surveys and regions

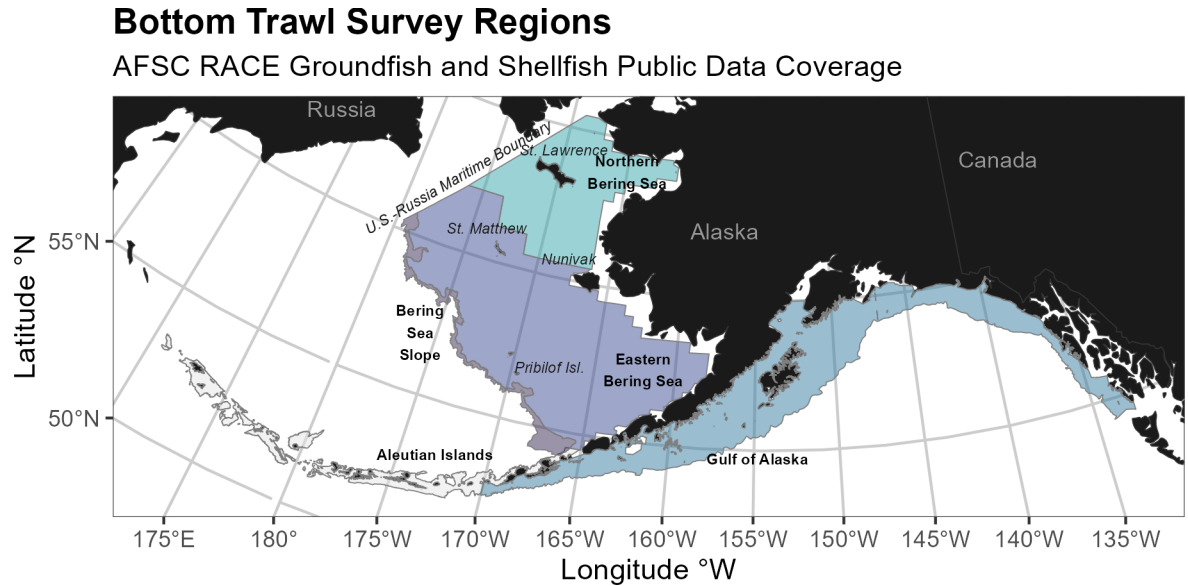


Figure 1.1.: Strata used in the all surveys.

Each survey conducted by the Groundfish Assessment Program are multispecies bottom trawl surveys. We collect environmental and biological data to assess how climate variability and loss of sea ice are affecting bottom-dwelling marine life on the Bering Sea shelf. We monitor trends in the distribution (location and movement patterns) and abundance of groundfish and crab species as well as oceanographic data (e.g., water temperature, depth). We collect biological information such as organism weight, length, stomachs to learn about diets, and otoliths to determine fish ages. We use this information in annual stock assessments and to assess the state of the ecosystem. This research is conducted on fishing industry contract vessels.

1. Survey Background

Table 1.1.: Survey summary stats

Survey	Survey Definition ID	Years	Depth (m)	Area (km2)	# Statistical Areas	# Possible Stations
Aleutian Islands Bottom Trawl Survey	52	2024 - 1991 (14)	1 - 500	64,415.0	80	1,312
Eastern Bering Sea Slope Bottom Trawl Survey	78	2016 - 2002 (6)	201 - 1,200	32,861.3	37	
Eastern Bering Sea Crab/Groundfish Bottom Trawl Survey	98	2025 - 1982 (43)	1 - 200	492,989.9	28	515
Gulf of Alaska Bottom Trawl Survey	47	2025 - 1990 (17)	1 - 1,000	315,501.9	37	6,939
Northern Bering Sea Crab/Groundfish Survey - Eastern Bering Sea Shelf Survey Extension	143	2025 - 2010 (7)	1 - 100	189,191.4	4	144

1.5.1. Aleutian Islands

Most recent data report: (Von Szalay et al., 2023)

- Upper Continental Slope of the Aleutian Islands from Unimak Pass to Stalemate Bank
- Triennial (1990s)/Biennial since 2000 in even years, since 1992
- Modified Index-Stratified Random of Successful Stations Survey Design
- Important commercial fish species include Atka mackerel, Pacific ocean perch, walleye pollock, Pacific cod, sablefish, and other rockfish species.

1. Survey Background

AI Bottom Trawl Survey Region

AFSC RACE Groundfish and Shellfish Public Data Coverage

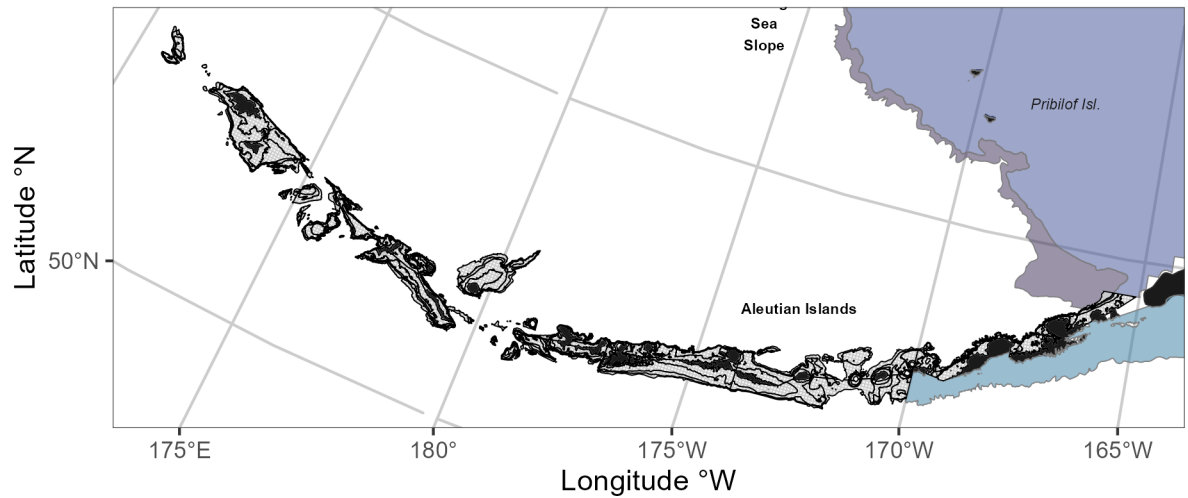


Figure 1.2.: Strata used in the Aleutian Islands bottom trawl survey.

1.5.2. Gulf of Alaska

Most recent data report: (Siple et al., 2024)

- Continental Shelf and Upper Slope of the Gulf of Alaska extending from the Islands of Four Mountains 2,300 km east to Dixon Entrance
- Triennial (1990s)/Biennial since 2001 in odd years, since 1991
- Stratified Random Survey Design
- Important commercial species in the Gulf of Alaska include Pacific ocean perch, walleye pollock, Pacific cod, flatfish, and other rockfish species.

1. Survey Background

GOA Bottom Trawl Survey Region

AFSC RACE Groundfish and Shellfish Public Data Coverage

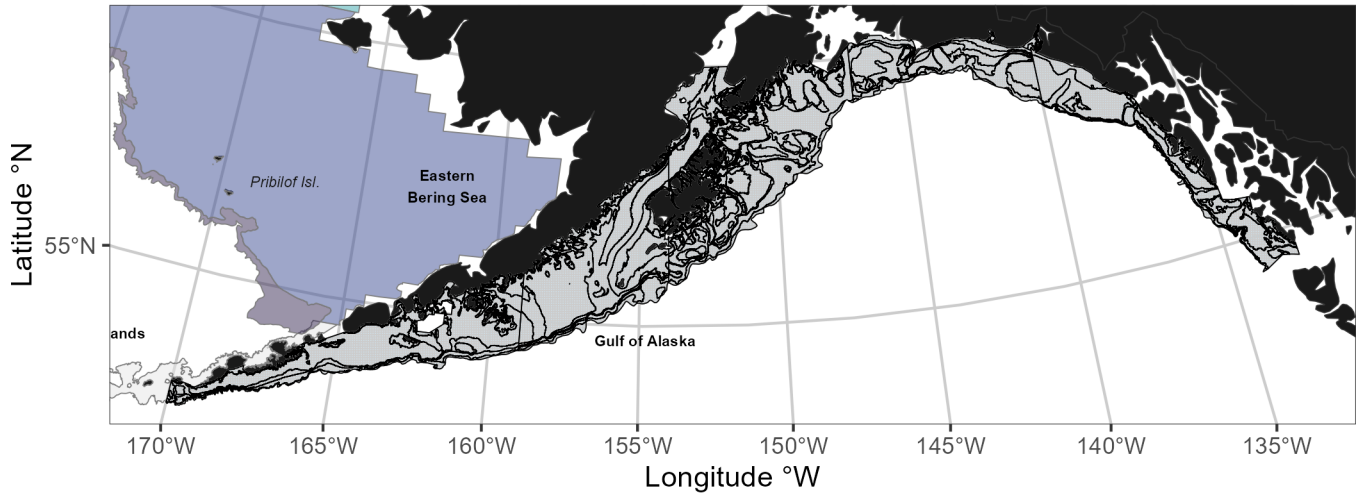


Figure 1.3.: Strata used in the Gulf of Alaska bottom trawl survey.

1.5.3. Eastern Bering Sea Shelf

Most recent data report: (Markowitz et al., 2025)

- The continental shelf of the eastern Bering Sea from the Aleutian Islands to the Bering Strait
- Conducted annually since 1982.
- Uses a stratified systematic sampling survey design with fixed stations at center of 20 x 20 nm grid.
- Similar in design to the northern Bering Sea shelf bottom trawl survey.
- Focus species for the Bering Sea include walleye pollock, Pacific cod, Greenland turbot, yellowfin sole, northern rock sole, red king crab, and snow and Tanner crabs.

1. Survey Background

EBS Bottom Trawl Survey Region

AFSC RACE Groundfish and Shellfish Public Data Coverage

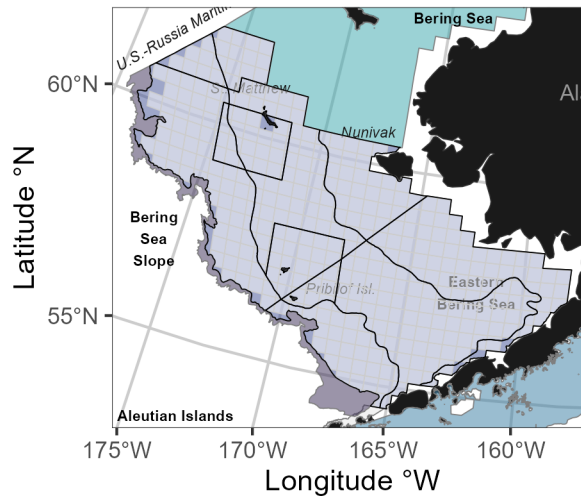


Figure 1.4.: Strata used in the Eastern Bering Sea bottom trawl survey.

1.5.4. Northern Bering Sea

Most recent data report: (Markowitz et al., 2024)

- The continental shelf of the northern Bering Sea, including the area north of St. Lawrence Island and Norton Sound
- Biennial/Annual; conducted intermittently since 2010
- Uses a stratified systematic sampling survey design with fixed stations at center of 20 x 20 nm grid.
- Similar in design to the eastern Bering Sea shelf bottom trawl survey.

1. Survey Background

NBS Bottom Trawl Survey Region

AFSC RACE Groundfish and Shellfish Public Data Coverage

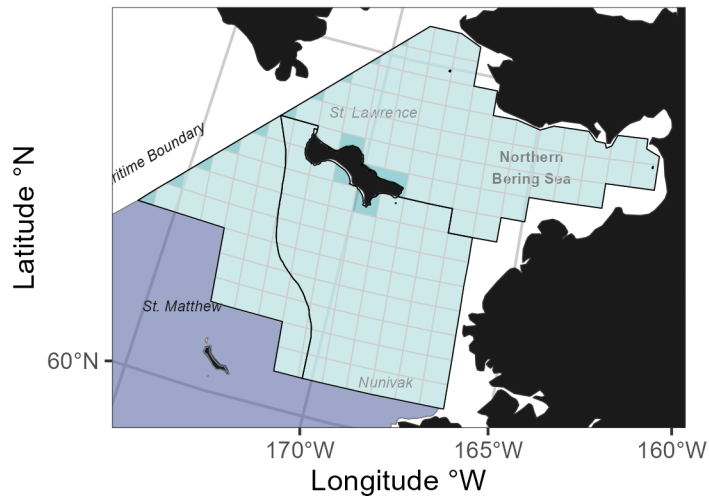


Figure 1.5.: Strata used in the Northern Bering Sea bottom trawl survey.

1.5.5. Eastern Bering Sea Upper Continental Slope

Most recent data report: (Hoff, 2016)

- The eastern Bering Sea upper continental slope survey area extends from Unalaska and Akutan Islands to the U.S.-Russian Maritime Boundary at 61° N near the International Date Line (166° E to 180° W) at depths from 200 to 1,200 m
- Conducted intermittently since 2002 (funding dependent)
- Modified Index-Stratified Random of Successful Stations Survey Design
- Focus species for the Bering Sea slope include giant grenadier, Pacific ocean perch, popeye grenadier, walleye pollock, and arrowtooth flounder.

1. Survey Background

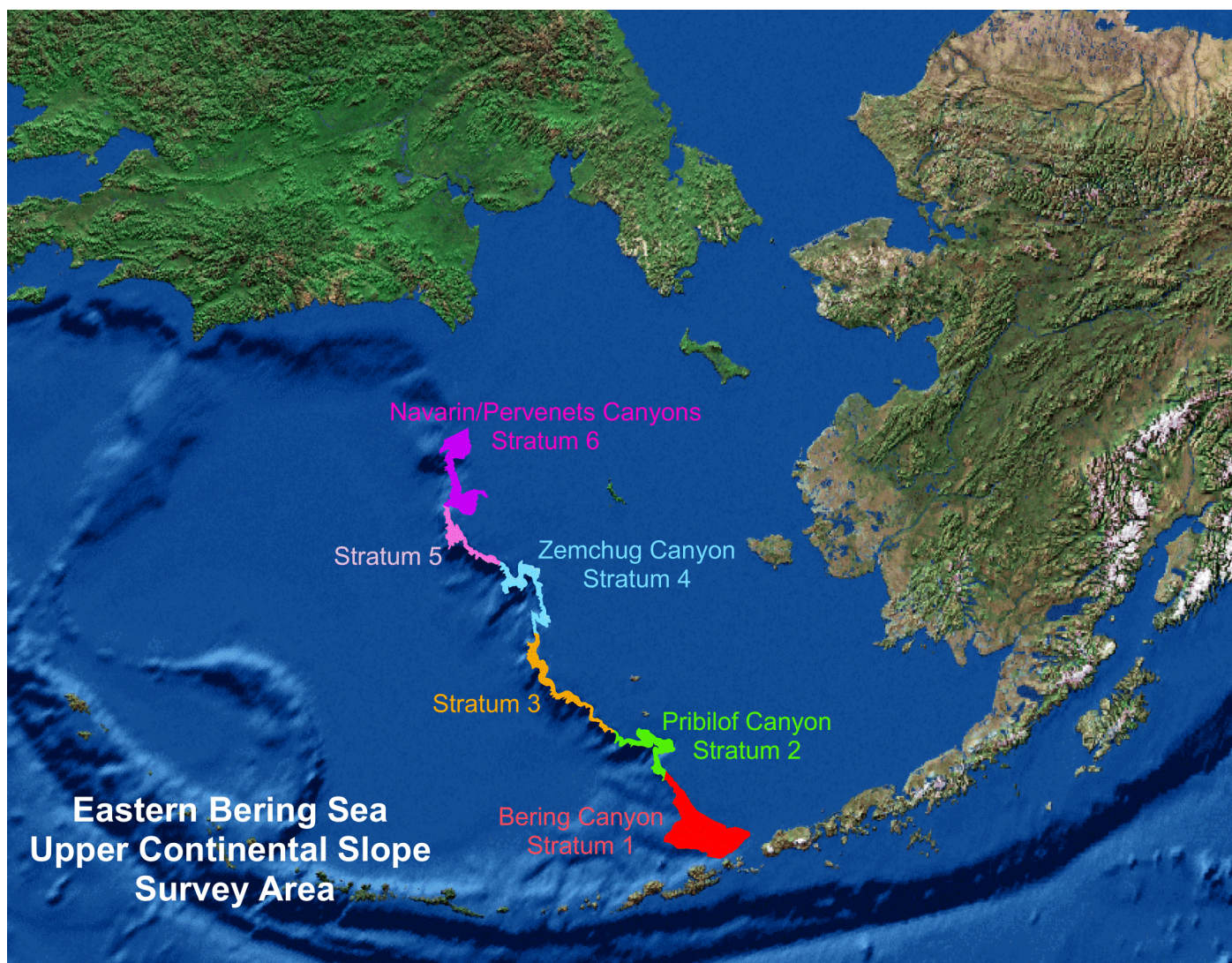


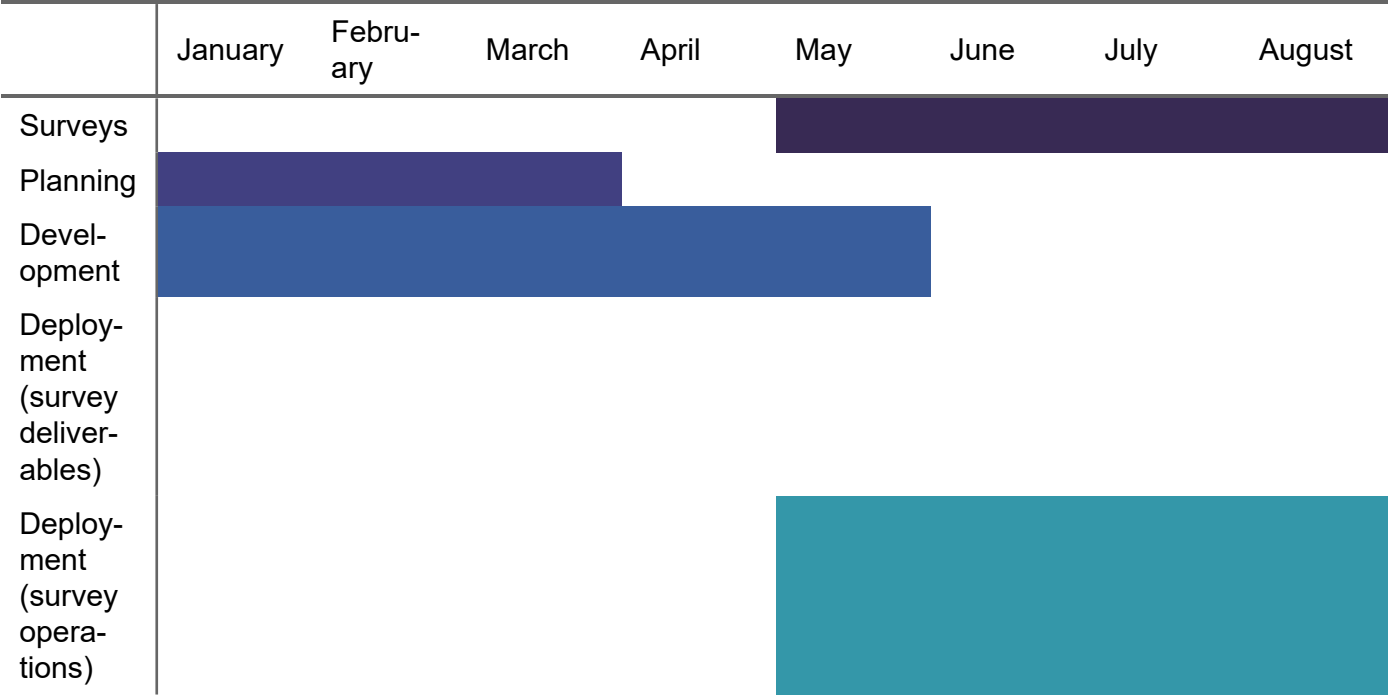
Figure 1.6.: Strata used in the Bering Sea Slope bottom trawl survey.

2. Workflow

2.1. Operational Product Development Timeline

Over the course of the year, the survey team is developing a variety of different data products. Planning and preparation for surveys happens in the late winter and spring, surveys occur in the summer, data validation takes place over the course of the survey and after the survey, and data products are produced through fall and late winter.

Table 2.1.: Operational product development timeline.



1. Survey Background

	January	February	March	April	May	June	July	August
Triage (fixing bugs and errors)								
User feedback and brainstorming								

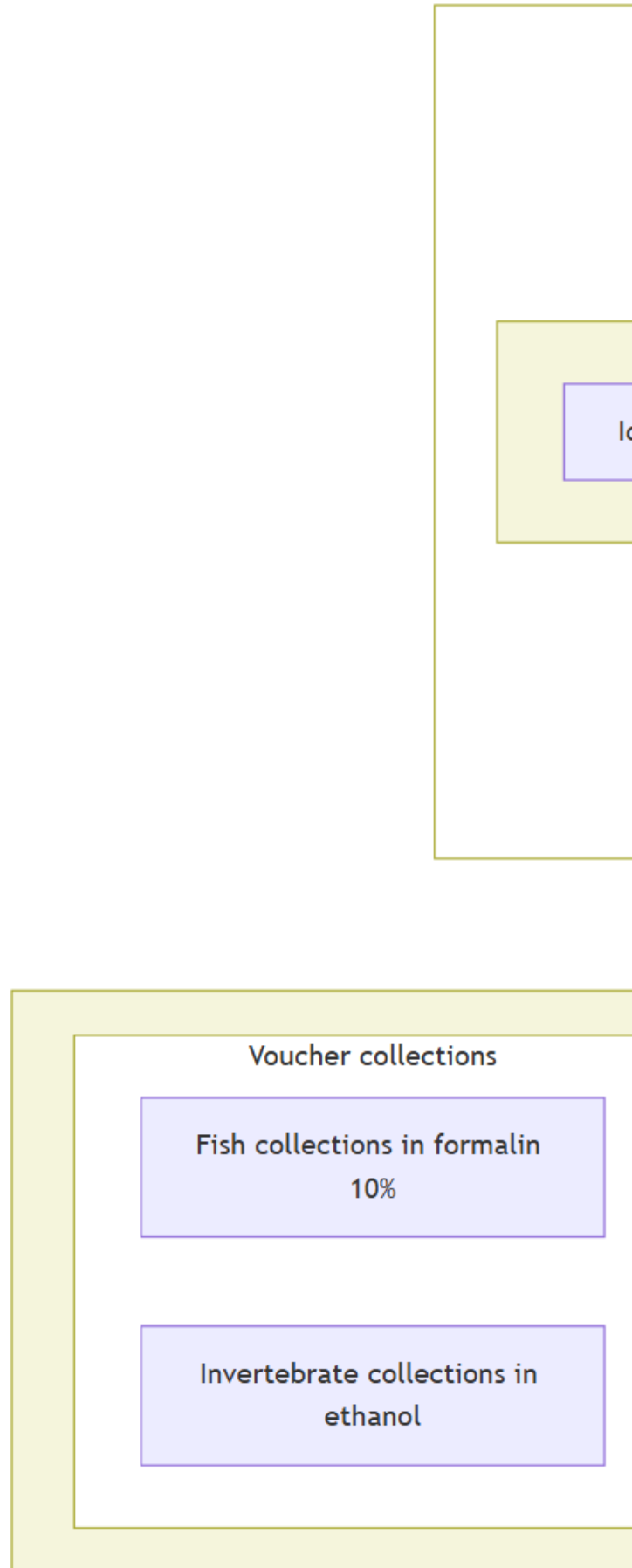
2.2. Data workflow from boat to production

Organisms first need to be collected aboard the vessel before data can be entered into tablets.

The objective of this process is to take raw data, QA/QC and clean these data, curate standard data products for these survey. Please note, through this process we are not providing “data” (what we consider lower level data material; see the data levels section below) but “data products”, which is intended to facilitate the most fool-proof standard interpretation of the data. These data products only use data from standard and validated hauls, and has undergone careful review.

Once survey data collected on the vessel has been checked and validated, the `gap_products/code/run.R` script is used to orchestrate a sequence of programs that calculate the standard data products resulting from the NOAA AFSC GAP bottom trawl surveys. Standard data products are the CPUE, BIOMASS, SIZECOMP, and AGECOMP tables in the GAP_PRODUCTS Oracle schema. The tables are slated to be updated twice a year: once after the survey season following finalization of that summer’s bottom trawl survey data to incorporate the new catch, size, and effort data and once prior to an upcoming survey to incorporate new age data that were processed after the prior summer’s survey season ended. This second pre-survey production run will also incorporate changes in the data due to the specimen voucher process as well as other post-hoc changes in the survey data.

1. Survey Background



1. Survey Background

The data from these surveys constitute a **living data set** so we can continue to **provide the best available data to all partners, stakeholders, and fellow scientists**.

2.2.1. During each data product run cycle:

1. Versions of the tables in GAP_PRODUCTS are locally imported within the gap_products repository to compare with the updated production tables. Any changes to a production table will be compared and checked to make sure those changes are intentional and documented.
2. Use the `gapindex` R package to calculate the four major standard data products: CPUE, BIOMASS, SIZECOMP, AGECOMP. These tables are compared and checked to their respective locally saved copies and any changes to the tables are vetted and documented. These tables are then uploaded to the GAP_PRODUCTS Oracle schema.
3. Calculate the various materialized views for AKFIN and FOSS purposes. Since these are derivative of the tables in GAP_PRODUCTS as well as other base tables in RACEBASE and RACE_DATA, it is not necessary to check these views in addition to the data checks done in the previous steps.

TABLE NAMES	TABLE TYPE	REFRESH FREQUENCY
AGECOMP BIOMASS CPUE SIZECOMP	TABLE	Quarterly: manually updated according to survey data Late Summer: after survey data are finalized Early Fall: after Bering Sea pollock otter trawl survey Winter: If there are changes to the taxonomic classification Late Spring: after Spring ages are updated
CRUISE	HAUL	LENGTH
SPECIMEN	MATERIALIZED VIEW	As needed: Refreshes any time the unprocessed data is updated
SURVEY_DESIGN TAXONOMIC_CLASSIFICATION TAXONOMIC_GROUPS TAXONOMIC_CHANGES TAXON_GROUPS	TABLE	Annually (beginning of year): manually updated

1. Survey Background

TABLE NAMES	TABLE TYPE	REFRESH FREQUENCY
AREA STRATUM_GROUPS METADATA_COLUMN METADATA_TABLE SPECIES_YEAR	TABLE	As needed: manually updated.
SAMPLE_COUNT	VIEW	When queried.

2.3. Data levels

GAP produces numerous data products that are subjected to different levels of processing, ranging from raw to highly-derived. The suitability of these data products for analysis varies and there is ambiguity about which data products can be used for which purpose. This ambiguity can create challenges in communicating about data products and potentially lead to misunderstanding and misuse of data. One approach to communicating about the level of processing applied to data products and their suitability for analysis is to describe data products using a Data Processing Level system. Data Processing Level systems are widely used in earth system sciences to characterize the extent of processing that has been applied to data products. For example, the NOAA National Centers for Environmental Information (NCEI) Satellite Program uses a Data Processing Level system to describe data on a scale of 0-4, where Level 0 is raw data and Level 4 is model output or results from analysis. Example of how NASA remote sensing data products are shared through a public data portal with levels of data processing and documentation.

For more information, see Sean Rohan's October 2022 SCRUGS presentation on the topic.

- **Level 0:** Raw and unprocessed data. Ex: Data on the G drive, some tables in RACE_DATA
- **Level 1:** Data products with QA/QC applied that may or may not be expanded to analysis units, but either not georeferenced or does not include full metadata. Ex: Some tables in RACE_DATA and RACEBASE
- **Level 2:** Analysis-ready data products that are derived for a standardized extent and account for zeros and missing/bad data. Ex: CPUE tables, some data products in public-facing archives and repositories

1. Survey Background

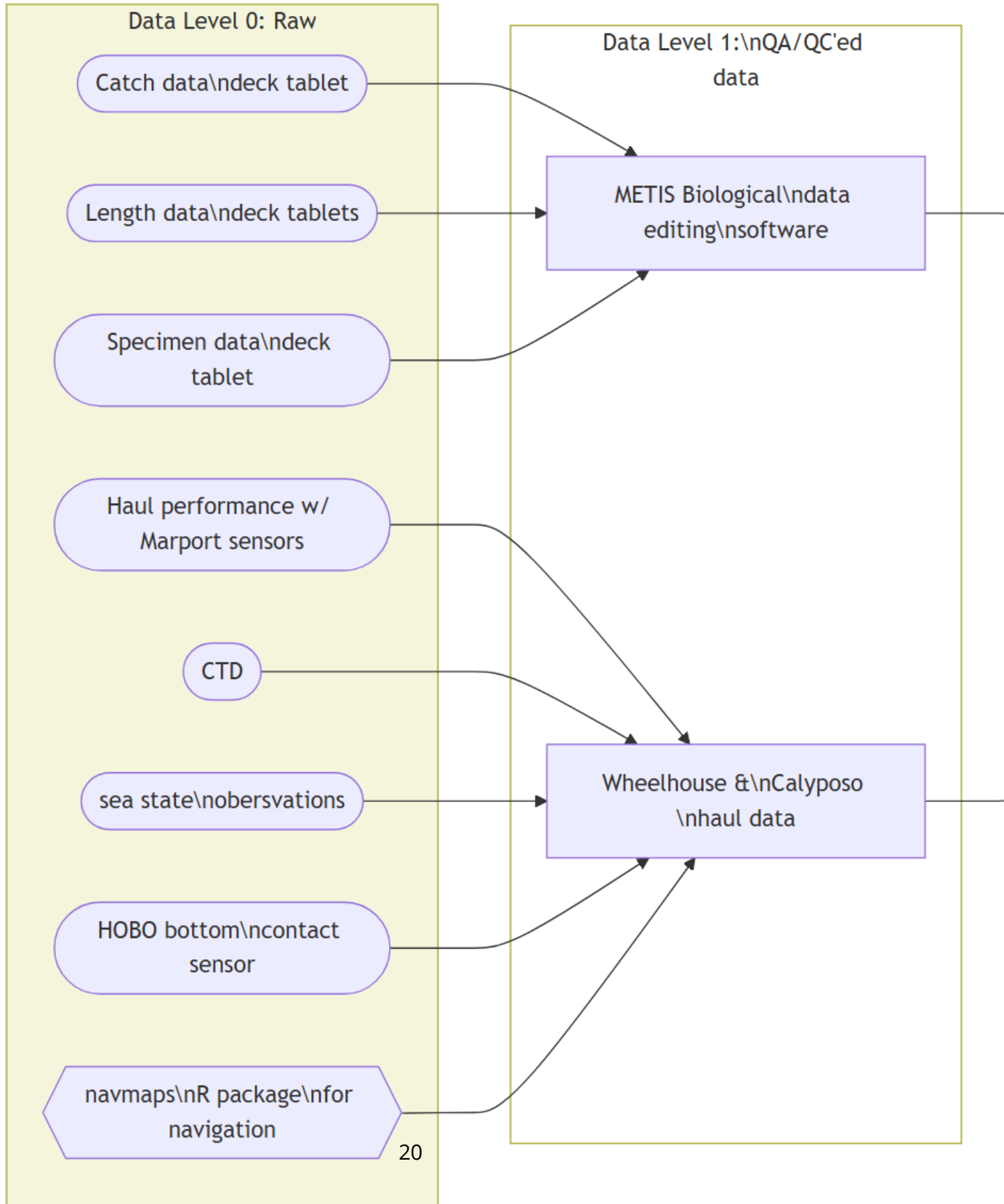
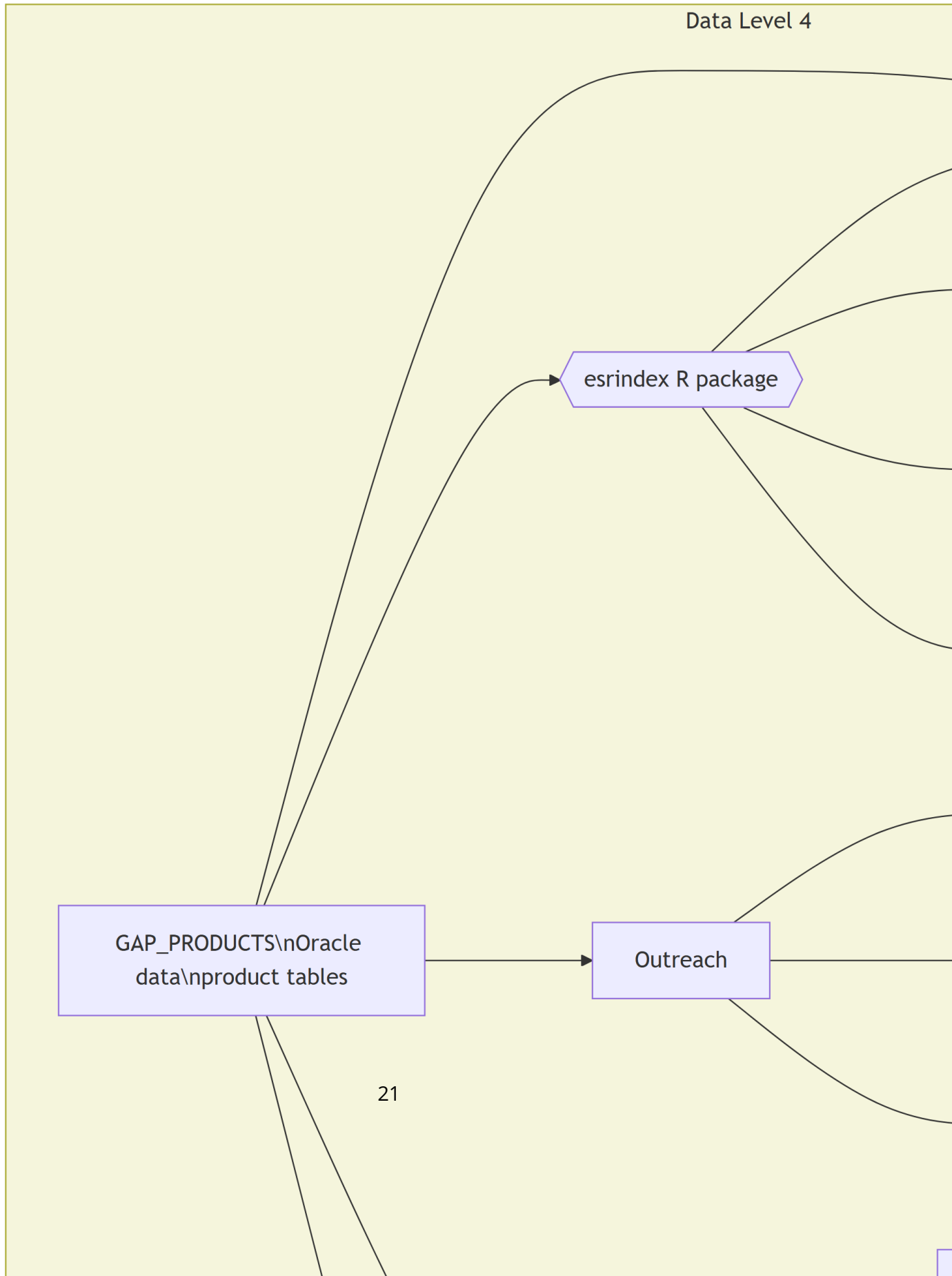


Figure 2.2.: Simplified data workflow from boat to production.

1. Survey Background



1. Survey Background

- **Level 3:** Data products that are synthesized across a standardized extent, often inputs in a higher-level analytical product. Ex: Abundance indices, some data products in public-facing archives and repositories
- **Level 4:** Analytically generated data products that are derived from lower-level data, often to inform management. Ex: Biological reference points from stock assessments, Essential Fish Habitat layers, indicators in Ecosystem Status Reports and Ecosystem and Socioeconomic Profiles

3. News

3.1. News/change logs

- GAP_PRODUCTS ChangeLog (last produced on 2026-06-26) using gapindex v3.1.0:
Run completed by: Zack Oyafuso
- GAP_PRODUCTS ChangeLog (last produced on 2026-06-25) using gapindex v3.1.0:
Run completed by: Zack Oyafuso
- GAP_PRODUCTS ChangeLog (last produced on 2026-04-24) using gapindex v3.1.0:
Run completed by: Zack Oyafuso
- GAP_PRODUCTS ChangeLog (last produced on 2026-03-13) using gapindex v3.1.0:
Run completed by: Zack Oyafuso
- GAP_PRODUCTS ChangeLog (last produced on 2025-09-29) using gapindex v3.0.3:
Run completed by: Duane Stevenson
- GAP_PRODUCTS ChangeLog (last produced on 2025-09-22) using gapindex v3.0.3:
Run completed by: Duane Stevenson
- GAP_PRODUCTS ChangeLog (last produced on 2025-08-25) using gapindex v3.0.2:
Run completed by: Zack Oyafuso
- GAP_PRODUCTS ChangeLog (last produced on 2025-07-15) using gapindex v2.2.0:
Run completed by: Zack Oyafuso
- GAP_PRODUCTS ChangeLog (last produced on 2025-06-05) using gapindex v3.0.2:
Run completed by: Zack Oyafuso
- GAP_PRODUCTS ChangeLog (last produced on 2025-05-20) using gapindex v3.0.2:
Run completed by: Zack Oyafuso
- GAP_PRODUCTS ChangeLog (last produced on 2025-04-30) using gapindex v3.0.2:
Run completed by: Zack Oyafuso
- GAP_PRODUCTS ChangeLog (last produced on 2025-03-20) using gapindex v3.0.2:
Run completed by: Zack Oyafuso

3. News

- GAP_PRODUCTS ChangeLog (last produced on 2024-12-10) using gapindex v3.0.2:
Run completed by: Sean Rooney
- GAP_PRODUCTS ChangeLog (last produced on 2024-10-22) using gapindex v2.2.0:
Run completed by: Zack Oyafuso
- GAP_PRODUCTS ChangeLog (last produced on 2024-10-21) using gapindex v2.2.0:
Run completed by: Duane Stevenson, Ned Laman, Zack Oyafuso
- GAP_PRODUCTS ChangeLog (last produced on 2024-09-05) using gapindex v2.2.0:
Run completed by: Ned Laman, Zack Oyafuso
- GAP_PRODUCTS ChangeLog (last produced on 2024-09-03) using gapindex v2.2.0:
Run completed by: Ned Laman, Zack Oyafuso
- GAP_PRODUCTS ChangeLog (last produced on 2024-08-29) using gapindex v2.2.0:
The additions of previous years' age data and 2024 EBS catch, effort, and size data
- GAP_PRODUCTS ChangeLog (last produced on 2024-08-20) using gapindex v2.2.0:
Initial 2024 post-survey run with new ages since last run and all of EBS Shelf 2024 survey data but none of AI 2024 survey data. While trying to update the records in the GAP_PRODUCTS table, the connection was terminated, partially uploading records in the agecomp tables and outputting NA to the N_HAUL and N_LENGTH fields in the biomass tables. At this point, the GAP_PRODUCTS tables are incomplete. The AKFIN and FOSS tables were NOT updated in this run.
- GAP_PRODUCTS ChangeLog (last produced on 2024-05-04) using gapindex v2.2.0: A development branch version of gapindex called using_datatable uses the data.table package for many dataframe manipulations, which greatly decreased the computation time of many of the functions. There were no major changes in the calculations in this version of the gapindex package and thus the major changes listed below are not related to the gapindex package. The only major change from this run was the addition of GOA 2023 Pacific Ocean perch read otolith data.
- GAP_PRODUCTS ChangeLog (last produced on 2024-04-09) using gapindex v2.2.0: A development branch version of gapindex called using_datatable uses the data.table package for many dataframe manipulations, which greatly decreased the computation time of many of the functions. There were no major changes in the calculations in this version of the gapindex package and thus the major changes listed below are not related to the gapindex package.
- GAP_PRODUCTS ChangeLog (last produced on 2024-02-29) using gapindex v2.2.0: A new version of gapindex 2.2.0 was used for this production run and now accesses taxonomic information from RACEBASE.SPECIES instead of GAP_PRODUCTS.TAXONOMIC_CLASSIFICATION. As a result, there will be some SPECIES_CODE values that are

3. News

supported due to slight differences between the two tables. Discussion in this github issue #54. As a result there are new cpue records for SPECIES_CODE values 22290 and 22292 and removed cpue records for SPECIES_CODE values 21345, 22200 and 69326.

- GAP_PRODUCTS ChangeLog (last produced on 2024-01-09) using gapindex v2.1.3: A new version of gapindex (v2.1.3) was used to produced these data. Data for SPECIES_CODE 68590 (Chionoecetes hybrids) are now removed, per this issue (https://github.com/afsc-gap-products/gap_products/issues/3). New read otolith data were incorporated into the age compositions. GOA depth subareas are now included in the size comps, and there were some modifications with EBS skate length data that are now incorporated into the length compositions.
- GAP_PRODUCTS ChangeLog (last produced on 2023-11-17) using gapindex v2.1.2: A new version of gapindex (v2.1.2) was used to produced these data. There was a slight change to how subarea biomass totals are calculated that was not fully addressed in v2.1.1. The modified biomass records reflect this change.
- GAP_PRODUCTS ChangeLog (last produced on 2023-11-14) using gapindex v2.1.1: A new version of gapindex (v2.1.1) was used to produced these data. There was a slight change to how subarea biomass totals are calculated. The modified biomass records reflect this change. New 2022 otolith data were available since the last iteration of the GAP_PRODUCTS for Aleutian Island Pacific ocean perch and northern rockfish and Eastern Bering Sea northern rock sole. Zero-filled CPUE records for four GOA species codes (SPECIES_CODE: 21210, 30010, 30360, 77102, 98101) were added due to how the 1990 data were integrated in the last production run of GAP_PRODUCTS. Two Arctic cod (SPECIES_CODE: 21725) and one plain sculpin (SPECIES_CODE: 21371) count records were modified in the NBS data, which changes the numerical CPUE estimates for those hauls which changes the estimated population abundance and size composition for those species.
- Groundfish Assessment Program Survey Data Serving and Data Improvements: Initial data changes brief distributed to SSMA and other partners by Ned Laman, Zack Oyafuso, and Emily Markowitz
- Run 2023-06-01 gapindex v2.1.0: Initial compiling and planning notes

4. Code of Conduct

4.1. What are Codes of Conduct?

Codes of Conduct are voluntary sets of rules that assist creators, developers, and users of code and data with data protection compliance and accountability in specific sectors or relating to particular processing operations.

Codes can help organizations to ensure all participants follow best practices and rules designed specifically for their sector or processing operations, thus enhancing compliance and collaboration. They are developed and managed by an association or other body (the 'Code Owner') which is representative of a sector (or category of data controllers or processors), with the expert and sectoral knowledge of how to enhance data protection in their area.

4.1.1. Code of Conduct from the nmfs-opensci GitHub.

5. NOAA Fisheries Open Science Code of Conduct

This code of conduct was developed and adapted from the Atom code of conduct in October 2021.

5.1. Our Pledge

In the interest of fostering an open and welcoming environment, we as contributors and maintainers pledge to making participation in our project and our community a harassment-free experience for everyone, regardless of age, body size, disability, ethnicity, gender identity and expression, level of experience, nationality, personal appearance, race, religion, or sexual identity and orientation.

5.2. Our Standards

Examples of behavior that contributes to creating a positive environment include:

- Using welcoming and inclusive language
- Being respectful of differing viewpoints and experiences
- Gracefully accepting constructive criticism
- Focusing on what is best for the community
- Showing empathy towards other community members

Examples of unacceptable behavior by participants include:

- The use of sexualized language or imagery and unwelcome sexual attention or advances
- Trolling, insulting/derogatory comments, and personal or political attacks
- Public or private harassment

5. NOAA Fisheries Open Science Code of Conduct

- Publishing others' private information, such as a physical or electronic address, without explicit permission
- Other conduct which could reasonably be considered inappropriate in a professional setting

5.3. Our Responsibilities

Project maintainers are responsible for clarifying the standards of acceptable behavior and are expected to take appropriate and fair corrective action in response to any instances of unacceptable behavior.

Project maintainers have the right and responsibility to remove, edit, or reject comments, commits, code, wiki edits, issues, and other contributions that are not aligned to this Code of Conduct, or to ban temporarily or permanently any contributor for other behaviors that they deem inappropriate, threatening, offensive, or harmful.

5.4. Scope

This Code of Conduct applies both within project spaces and in public spaces when an individual is representing the project or its community. Examples of representing a project or community include using an official project e-mail address, posting via an official social media account, or acting as an appointed representative at an online or offline event. Representation of a project may be further defined and clarified by project maintainers.

5.5. Enforcement

Instances of abusive, harassing, or otherwise unacceptable behavior may be reported by contacting the project team. All complaints will be reviewed and investigated and will result in a response that is deemed necessary and appropriate to the circumstances. Further details of specific enforcement policies may be posted separately.

5. NOAA Fisheries Open Science Code of Conduct

5.6. Attribution

This Code of Conduct is adapted from the Contributor Covenant, version 1.4, available at <https://contributor-covenant.org/version/1/4>

Part II.

GAP Production Data

Data Description

The Resource Assessment and Conservation Engineering Division (RACE) Groundfish Assessment Program (GAP) of the Alaska Fisheries Science Center (AFSC) conducts fisheries-independent bottom trawl surveys to monitor the condition of the demersal fish and crab stocks of Alaska. These data are developed to describe the temporal distribution and abundance of commercially and ecologically important groundfish species, examine the changes in the species composition of the fauna over time and space, and describe the physical environment of the groundfish habitat. These data are created using the gapindex R package v2.1.0.

Users must read and fully comprehend the metadata prior to use. Data should not be used beyond the limits of the source scale. Acknowledgement of NOAA, as the source from which these data were obtained, in any publications and/or other representations of these data, is suggested. These data are compiled and approved annually after each summer survey season. The data from previous years are unlikely to change substantially once published. Some survey data are excluded, such as non-standard stations, surveys completed in earlier years using different/non-standard gear, and special tows and non-standard data collections.

gapindex

Code to generate design-based catch-per-unit-effort (CPUE), indices of abundance, biomass, and size and age compositions from survey data is available from gapindex. See the gapindex documentation for more information. Make sure you have installed R packages devtools, RODBC, and getPass and are connected to the AFSC network or VPN while using this package.

Cite this data

Use the below bibtext citation, as cited in our group's citation repository for citing the data created and maintained in this repository. Add "note = {Accessed: mm/dd/yyyy}" to append the day this data was accessed.

```
[1] "@misc{GAPProducts,"  
[2] "  author = {{NOAA Fisheries Alaska Fisheries Science Center, Groundfish Assessment Program},  
[3] "  year = {2024}, "
```

The Alaska Fisheries Information Network

```
[4] " title = {AFSC Goundfish Assessment Program Design-Based Production Data},"  
[5] " howpublished = {https://www.fisheries.noaa.gov/alaska/science-data/groundfish-  
assessment-program-bottom-trawl-surveys},"  
[6] " publisher = {{U.S. Dep. Commer.}}, "  
[7] " copyright = {Public Domain} "  
[8] "]"
```

The Alaska Fisheries Information Network

These data are used directly by stock assessors and are provided to The Alaska Fisheries Information Network (AKFIN).

The Alaska Fisheries Information Network (AKFIN) is a regional program that consolidates and supports the processing, analysis, and reporting of fisheries data for Alaskan fisheries. AKFIN integrates this information into a single data management system using consistent methods and standardized formats. The resulting data enables fishery managers, scientists, and associated agencies to supervise fisheries resources more effectively and efficiently. The AKFIN database contains much of the data needed to complete stock assessments, including GAP trawl survey data.

Data Access Options

Direct database connection: If you are an AFSC employee you may access the AKFIN oracle database directly while on the NOAA network or VPN. Note that this is a separate database from the AFSC oracle database referenced above, and requires separate credentials. If you do not already have an AKFIN account you can request one here. NOAA IT will need to add AKFIN access to your tnsnames.ora file (they do this frequently). Once your connection is established data may be accessed through SQL queries using SQL developer, R, or python.

AKFIN Answers

(AKFIN Answers)[<https://akfin.psmfc.org/akfin-answers/>] is an Oracle BI tool used for distributing data to stock assessors and other users. Usernames and passwords are distinct from AKFIN direct database credentials. The distribution of GAP_PRODUCTS on AKFIN Answers is planned but not yet implemented. The RACE Survey tab on the stock assessment dashboard contains reports generated from now depreciated tables

The Alaska Fisheries Information Network

that predated the GAP_PRODUCTS tables. AKFIN will keep these reports for reference but they will not be updated 2024 onward.

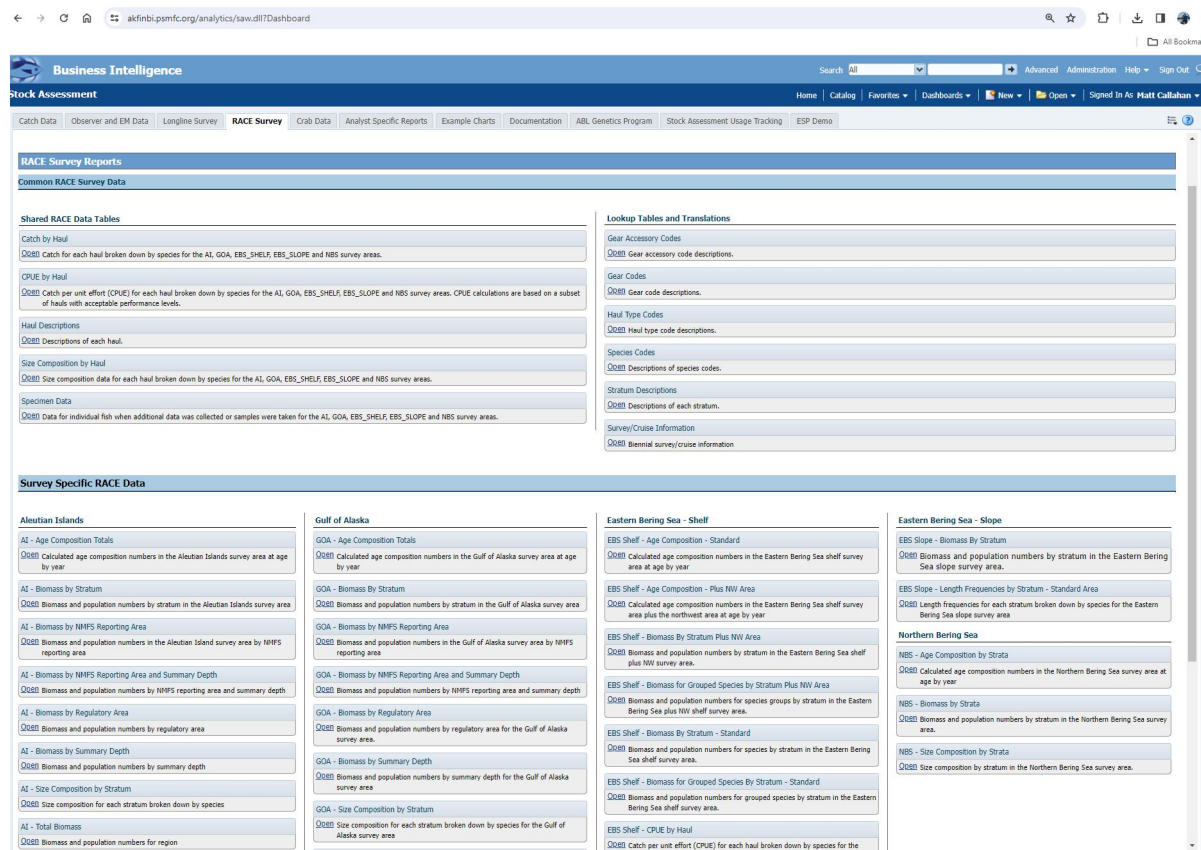


Figure 5.1.: AKFIN platform.

Web Service

AKFIN has developed web services (apis) to distribute GAP data. Like the GAP_PRODUCTS schema, these are under active development. These do not require VPN or an oracle connection but they are protected by Oracle authentication, please contact matt.callahan@noaa.gov for information on how to get an api token to use this option.

The url structure is "https://apex.psmfc.org/akfin/data_marts/gap_products/gap_[base table name]". For example "https://apex.psmfc.org/akfin/data_marts/gap_products/gap_biomass" is the base url to get data from the akfin_biomass table. Web services linked

Fisheries One Stop Shop data platform

to large tables have mandatory parameters to reduce data download size. For example to get agecomp data for Bering Sea pollock in area_id 10 in 2022 you would use "https://apex.psmfc.org/akfin/data_marts/gap_products/gap_biomass?survey_definition_id=98&area_id=10&species_code=21740&start_year=2022&end_year=2022".

If you're using R to pull data through web services you might find the `akfingapdata` (pronounced `akfin-gap-data` not `ak-eff-ing-app-data`) R package helpful.

Cite this data

Use the below bibtext citation, as cited in our group's citation repository for citing the data created and maintained in this repo (Alaska Fisheries Information Network (AKFIN), 2024). Add "note = {Accessed: mm/dd/yyyy}" to append the day this data was accessed.

```
[1] "@misc{GAPakfin,"
[2] "  author = {{Alaska Fisheries Information Network (AKFIN)}},"
[3] "  institution = {{NOAA Fisheries Alaska Fisheries Science Center, Groundfish Assessment Program Design-Based Production Data}}, "
[4] "  year = {2024}, "
[5] "  title = {AFSC Groundfish Assessment Program Design-Based Production Data},"
[6] "  howpublished = {https://akfinbi.psmfc.org/analytics/},"
[7] "  url = {https://www.psmfc.org/program/alaska-fisheries-information-network-akfin},"
[8] "  publisher = {{U.S. Dep. Commer.}},"
[9] "  copyright = {Public Domain} "
[10] "}"
```

Fisheries One Stop Shop data platform

The final, validated survey data are publicly accessible soon after surveys are completed on the Fisheries One Stop Shop (FOSS) platform. This data includes catch, haul, and environmental data collected at each station. On the FOSS data platform, users can interactively select, view, and download data. Descriptive documentation and user-examples are available on the metadata page.

This data contains all of the catch, environmental, and haul data from the fisheries-independent Groundfish and Shellfish Assessment Program surveys in the Bering Sea, Aleutian Islands, and Gulf of Alaska. This data is sought after by the general

Fisheries One Stop Shop data platform

public, private entities, and NOAA partners alike, including tribal organizations, K-12 classrooms, academic institutions, for-profit groups, and non-profit groups. This data is compiled and approved once a year after each summer survey season and is available for open access.

Collaborators and data users

Below are a few packages and products currently using this data. If you have developed a product, performed an analysis, or exhibited this data in any way, reach out so we can showcase your hard work.

- **NOAA Fisheries Distribution Mapping and Analysis Portal**; *NOAA Fisheries Office of Science and Technology*
- **Pull data with python and explore the in-browser visualization tool. Reference their example Python notebook**; *The Eric and Wendy Schmidt Center for Data Science and the Environment at UC Berkeley, including sam.pottinger@berkeley.edu, ccmartinez@berkeley.edu, gzarpellon@berkeley.edu, and kkoy@berkeley.edu.*

Access Constraints

User Constraints: Users must read and fully comprehend the metadata prior to use. Data should not be used beyond the limits of the source scale. Acknowledgment of AFSC Groundfish Assessment Program, as the source from which these data were obtained, in any publications and/or other representations of these data, is suggested.

General questions and more specific data requests can be sent to nmfs.afsc.gap.metadata@noaa.gov or submitted as an issue on our GitHub Organization. The version of this data used for stock assessments can be found through the Alaska Fisheries Information Network (AKFIN). For questions about the eastern Bering Sea surveys, contact Duane Stevenson (Duane.Stevenson@noaa.gov). For questions about the Gulf of Alaska or Aleutian Islands surveys, contact Ned Laman (Ned.Laman@noaa.gov). For questions specifically about crab data in any region, contact Mike Litzow (Mike.Litzow@noaa.gov), the Shellfish Assessment Program lead.

For questions, comments, and concerns specifically about the Fisheries One Stop Shop (FOSS) platform, please contact us using the Comments page on the FOSS webpage.

Cite this data

Use the below bibtext citation, as cited in our group's citation repository for citing the data created and maintained in this repository. Add "note = {Accessed: mm/dd/yyyy}" to append the day this data was accessed.

```
[1] "@misc{FOSSAFSCData,"
[2] "  author = {{NOAA Fisheries Alaska Fisheries Science Center}}, "
[3] "  year = {2024}, "
[4] "  title = {Fisheries One Stop Shop Public Data: RACE Division Bottom Trawl Survey Data
[5] "  howpublished = {https://www.fisheries.noaa.gov/foss},"
[6] "  publisher = {{U.S. Dep. Commer.}}, "
[7] "  copyright = {Public Domain} "
[8] "]"
```


6. Data Description

6.1. Data tables

6.1.1. AGECOMP

Stratum- and region-level age compositions by sex. This table was created by the Resource Assessment and Conservation Engineering Division (RACE) Groundfish Assessment Program (GAP) of the Alaska Fisheries Science Center (AFSC). The GitHub repository for the scripts that created this code can be found at (https://github.com/afsc-gap-products/gap_products). There are no legal restrictions on access to the data. Last updated on 14 March 2026.

Number of rows: 680,450

Number of columns: 10

Column name from data

Descriptive column Name

Units

Oracle data type

Column description

SURVEY_DEFINITION_ID

Survey ID

ID key code

NUMBER(38,0)

The survey definition ID key code is an integer that uniquely identifies a survey region/survey design. The column `survey_definition_id` is associated with the `srvy` and survey columns. Full list of survey definition IDs are in `RACE_DATA.SURVEY_DEFINITIONS` and in the code books.

6. Data Description

YEAR

Survey year

year

NUMBER(10,0)

Year the observation (survey) was collected.

AREA_ID

Area ID

ID key code

NUMBER(38,0)

An ID code representing a broad range of spatial areas from statistical sampling strata to management and regional areas composed of multiple strata.

SPECIES_CODE

Taxon code

ID key code

NUMBER(38,0)

The species code of the organism associated with the common_name and scientific_name columns. For a complete species list, review the code books.

SEX

Sex of a specimen

ID key code

NUMBER(38,0)

Sex of a specimen where "1" = "Male", "2" = "Female", "3" = Unsexed.

AGE

Taxon age bin (yrs)

integer

NUMBER(38,0)

Age bin of taxon. Age bin of a taxon in years estimated by the age comp estimate. Age -9 indicates unaged lengths for a particular sex because no otoliths were collected for

6. Data Description

that sex/length combination. Age -99 indicates a case where no lengths were collected within a stratum for a species/year even though catch numbers were recorded.

POPULATION_COUNT

Estimated population

numeric

NUMBER(38,0)

The estimated population caught in the survey for a species, group, or total for a given survey.

LENGTH_MM_MEAN

Mean length at age weighted by numbers at length

numeric

NUMBER(38,3)

Mean length (millimeters).

LENGTH_MM_SD

Standard deviation of length at age weighted by numbers at length

numeric

NUMBER(38,3)

Variance of mean length.

AREA_ID_FOOTPRINT

Survey Footprint

text

VARCHAR2(4000 BYTE)

Survey footprint, usually equivalent to the SURVEY_DEFINITION_ID with the exception of the Standard and Standard +NW survey footprints in the Eastern Bering Sea shelf bottom trawl survey.

6. Data Description

6.1.2. AREA

Information related to the various strata, subareas, INPFC and NMFS management areas, and regions for the Aleutian Islands, Gulf of Alaska, and Bering Sea shelf and slope bottom trawl surveys. This table was created by the Resource Assessment and Conservation Engineering Division (RACE) Groundfish Assessment Program (GAP) of the Alaska Fisheries Science Center (AFSC). The GitHub repository for the scripts that created this code can be found at (https://github.com/afsc-gap-products/gap_products). There are no legal restrictions on access to the data. Last updated on 14 March 2026.

Number of rows: 499

Number of columns: 9

Column name from data

Descriptive column Name

Units

Oracle data type

Column description

DESIGN_YEAR

Design year

year

NUMBER(10,0)

Year ID associated with a given value AREA_ID. This field describes the changes in the survey design over time.

SURVEY_DEFINITION_ID

Survey ID

ID key code

NUMBER(38,0)

The survey definition ID key code is an integer that uniquely identifies a survey region/survey design. The column survey_definition_id is associated with the srvy and survey columns. Full list of survey definition IDs are in RACE_DATA.SURVEY_DEFINITIONS and in the code books.

AREA_ID

6. Data Description

Area ID

ID key code

NUMBER(38,0)

An ID code representing a broad range of spatial areas from statistical sampling strata to management and regional areas composed of multiple strata.

AREA_TYPE

Area ID type description

category

VARCHAR2(255 BYTE)

The type of stratum that AREA_ID represents. Types include: STRATUM (the smallest building-block unit of area in these surveys), REGION, DEPTH, SUBAREA, INPFC BY DEPTH, INPFC, SUBAREA BY DEPTH, REGULATORY AREA, NMFS STATISTICAL AREA.

AREA_NAME

Area ID name

text

VARCHAR2(4000 BYTE)

Descriptive name of each AREA_ID. These names often identify the region, depth ranges, or other regional information for the area ID.

DESCRIPTION

Description

text

VARCHAR2(4000 BYTE)

Description of row observation.

AREA_KM2

Area (km2)

kilometers squared

NUMBER(38,3)

Area in square kilometers.

DEPTH_MIN_M

6. Data Description

Area ID minimum depth (m)

meters

NUMBER(38,3)

Minimum depth (meters).

DEPTH_MAX_M

Area ID maximum depth (m)

meters

NUMBER(38,3)

Maximum depth (meters).

6.1.3. BIOMASS

Stratum/subarea/region-level mean CPUE (weight and numbers), total biomass, and total abundance with associated variances. This table was created by the Resource Assessment and Conservation Engineering Division (RACE) Groundfish Assessment Program (GAP) of the Alaska Fisheries Science Center (AFSC). The GitHub repository for the scripts that created this code can be found at (https://github.com/afsc-gap-products/gap_products). There are no legal restrictions on access to the data. Last updated on 14 March 2026.

Number of rows: 2,655,773

Number of columns: 16

Column name from data

Descriptive column Name

Units

Oracle data type

Column description

SURVEY_DEFINITION_ID

Survey ID

ID key code

NUMBER(38,0)

6. Data Description

The survey definition ID key code is an integer that uniquely identifies a survey region/survey design. The column `survey_definition_id` is associated with the `srvy` and `survey` columns. Full list of survey definition IDs are in `RACE_DATA.SURVEY_DEFINITIONS` and in the code books.

`YEAR`

Survey year

year

`NUMBER(10,0)`

Year the observation (survey) was collected.

`SPECIES_CODE`

Taxon code

ID key code

`NUMBER(38,0)`

The species code of the organism associated with the `common_name` and `scientific_name` columns. For a complete species list, review the code books.

`AREA_ID`

Area ID

ID key code

`NUMBER(38,0)`

An ID code representing a broad range of spatial areas from statistical sampling strata to management and regional areas composed of multiple strata.

`CPUE_KGKM2_MEAN`

Mean weight CPUE

kilograms per kilometers squared

`NUMBER(38,6)`

The mean catch weight (kilograms) per unit effort (area swept by the net, units squared kilometers).

`CPUE_NOKM2_MEAN`

Mean numeric CPUE

6. Data Description

count per kilometers squared

NUMBER(38,6)

The mean of numerical catch per unit effort (area swept by the net, units square kilometers).

N_HAUL

Valid hauls

count

NUMBER(38,0)

Total number of hauls.

N_WEIGHT

Hauls with catch

count

NUMBER(38,0)

Total number of hauls with positive catch biomass.

N_COUNT

Hauls with taxon counts

numeric

NUMBER(38,0)

Total number of hauls with positive count data.

N_LENGTH

Hauls with taxon lengths

count

NUMBER(38,0)

Total number of hauls with length data.

BIOMASS_MT

Estimated biomass

numeric

NUMBER(38,6)

6. Data Description

The estimated total biomass.

BIOMASS_VAR

Estimated biomass variance

numeric

NUMBER(38,6)

The estimated variance associated with the total biomass.

POPULATION_COUNT

Estimated population

numeric

NUMBER(38,0)

The estimated population caught in the survey for a species, group, or total for a given survey.

POPULATION_VAR

Estimated population variance

numeric

NUMBER(38,6)

The estimated population variance caught in the survey for a species, group, or total for a given survey.

CPUE_KGKM2_VAR

Variance of the mean weight CPUE

kilograms per kilometers squared

NUMBER(38,6)

The variance of mean catch weight (kilograms) per unit effort (area swept by the net, units squared kilometers).

CPUE_NOKM2_VAR

Variance of the mean numeric CPUE

count per kilometers squared

NUMBER(38,6)

6. Data Description

The variance of mean numerical catch per unit effort (area swept by the net, units square kilometers).

6.1.4. CPUE

Haul-level zero-filled weight and numerical catch-per-unit-effort. This table was created by the Resource Assessment and Conservation Engineering Division (RACE) Groundfish Assessment Program (GAP) of the Alaska Fisheries Science Center (AFSC). The GitHub repository for the scripts that created this code can be found at (https://github.com/afsc-gap-products/gap_products). There are no legal restrictions on access to the data. Last updated on 14 March 2026.

Number of rows: 21,915,584

Number of columns: 7

Column name from data

Descriptive column Name

Units

Oracle data type

Column description

HAULJOIN

Haul ID

ID key code

NUMBER(38,0)

This is a unique numeric identifier assigned to each (vessel, cruise, and haul) combination.

SPECIES_CODE

Taxon code

ID key code

NUMBER(38,0)

The species code of the organism associated with the common_name and scientific_name columns. For a complete species list, review the code books.

WEIGHT_KG

6. Data Description

Sample or taxon weight (kg)

kilograms

NUMBER(38,3)

Total weight (kilograms).

COUNT

Taxon count

count, whole number resolution

NUMBER(38,0)

Total whole number of individuals caught in haul or samples collected.

AREA_SWEPT_KM2

Area swept (km)

kilometers

NUMBER(38,6)

The area the net covered while the net was fishing (kilometers squared), defined as the distance fished times the net width.

CPUE_KGKM2

Weight CPUE (kg/km²)

kilograms per kilometers squared

NUMBER(38,6)

Catch weight (kilograms) per unit effort (area swept by the net, units square kilometers).

CPUE_NOKM2

Number CPUE (no/km²)

count per kilometers squared

NUMBER(38,6)

Numerical catch per unit effort (area swept by the net, units square kilometers).

6. Data Description

6.1.5. METADATA_COLUMN

This table is used to string together the various field comments for the tables in GAP_PRODUCTS. This table was created by the Resource Assessment and Conservation Engineering Division (RACE) Groundfish Assessment Program (GAP) of the Alaska Fisheries Science Center (AFSC). The GitHub repository for the scripts that created this code can be found at (https://github.com/afsc-gap-products/gap_products). There are no legal restrictions on access to the data. Last updated on 14 March 2026.

Number of rows: 170

Number of columns: 5

Column name from data

Descriptive column Name

Units

Oracle data type

Column description

METADATA_COLNAME

Column name

text

VARCHAR2(4000 BYTE)

Name of the column in a table.

METADATA_COLNAME_LONG

Column name spelled out

text

VARCHAR2(4000 BYTE)

Long name for the column.

METADATA_UNITS

Units

category

VARCHAR2(4000 BYTE)

6. Data Description

Units of the column.

METADATA_DATATYPE

Oracle datatype code

text

VARCHAR2(4000 BYTE)

Oracle data type of data column.

METADATA_COLNAME_DESC

Column description

text

VARCHAR2(4000 BYTE)

Description of the column.

6.1.6. SIZECOMP

Stratum/subarea/region-level size compositions by sex. This table was created by the Resource Assessment and Conservation Engineering Division (RACE) Groundfish Assessment Program (GAP) of the Alaska Fisheries Science Center (AFSC). The GitHub repository for the scripts that created this code can be found at (https://github.com/afsc-gap-products/gap_products). There are no legal restrictions on access to the data. Last updated on 14 March 2026.

Number of rows: 3,234,183

Number of columns: 7

Column name from data

Descriptive column Name

Units

Oracle data type

Column description

SURVEY_DEFINITION_ID

Survey ID

ID key code

6. Data Description

NUMBER(38,0)

The survey definition ID key code is an integer that uniquely identifies a survey region/survey design. The column `survey_definition_id` is associated with the `srvy` and `survey` columns. Full list of survey definition IDs are in `RACE_DATA.SURVEY_DEFINITIONS` and in the code books.

YEAR

Survey year

year

NUMBER(10,0)

Year the observation (survey) was collected.

AREA_ID

Area ID

ID key code

NUMBER(38,0)

An ID code representing a broad range of spatial areas from statistical sampling strata to management and regional areas composed of multiple strata.

SPECIES_CODE

Taxon code

ID key code

NUMBER(38,0)

The species code of the organism associated with the `common_name` and `scientific_name` columns. For a complete species list, review the code books.

LENGTH_MM

Length of a specimen

millimeters

NUMBER(10,0)

Length bin in millimeters. A length of -9 indicates cases where no lengths were collected within a stratum for a species/year, even though catch numbers were recorded.

SEX

6. Data Description

Sex of a specimen

ID key code

NUMBER(38,0)

Sex of a specimen where "1" = "Male", "2" = "Female", "3" = Unsexed.

POPULATION_COUNT

Estimated population

numeric

NUMBER(38,0)

The estimated population caught in the survey for a species, group, or total for a given survey.

6.1.7. SPECIES_YEAR

This is a table

Number of rows: 18

Number of columns: 3

Column name from data

Descriptive column Name

Units

Oracle data type

Column description

SPECIES_CODE

Taxon code

ID key code

NUMBER(38,0)

The species code of the organism associated with the common_name and scientific_name columns. For a complete species list, review the code books.

6. Data Description

6.1.8. STRATUM_GROUPS

Lookup table for which strata are contained within a given subarea, INPFC or NMFS management area, or region for the Aleutian Islands, Gulf of Alaska, and Bering Sea shelf and slope bottom trawl surveys. This table was created by the Resource Assessment and Conservation Engineering Division (RACE) Groundfish Assessment Program (GAP) of the Alaska Fisheries Science Center (AFSC). The GitHub repository for the scripts that created this code can be found at (<https://github.com/afsc-gap-products/gap-products>). There are no legal restrictions on access to the data. Last updated on 14 March 2026.

Number of rows: 1,063

Number of columns: 4

Column name from data

Descriptive column Name

Units

Oracle data type

Column description

AREA_ID

Area ID

ID key code

NUMBER(38,0)

An ID code representing a broad range of spatial areas from statistical sampling strata to management and regional areas composed of multiple strata.

SURVEY_DEFINITION_ID

Survey ID

ID key code

NUMBER(38,0)

The survey definition ID key code is an integer that uniquely identifies a survey region/survey design. The column `survey_definition_id` is associated with the `srvy` and `survey` columns. Full list of survey definition IDs are in `RACE_DATA.SURVEY_DEFINITIONS` and in the code books.

6. Data Description

DESIGN_YEAR

Design year

year

NUMBER(10,0)

Year ID associated with a given value AREA_ID. This field describes the changes in the survey design over time.

STRATUM

Stratum ID

ID key code

NUMBER(10,0)

RACE database statistical area for analyzing data. Strata were designed using bathymetry and other geographic and habitat-related elements. The strata are unique to each survey region. Stratum of value 0 indicates experimental tows.

6.1.9. SURVEY_DESIGN

This table contains for a given survey (via SURVEY_DEFINITION_ID) and survey year (YEAR), which version (DESIGN_YEAR) of the AREA_IDs that were used to calculate the various standard data products. This table was created by the Resource Assessment and Conservation Engineering Division (RACE) Groundfish Assessment Program (GAP) of the Alaska Fisheries Science Center (AFSC). The GitHub repository for the scripts that created this code can be found at (<https://github.com/afsc-gap-products/gap-products>). There are no legal restrictions on access to the data. Last updated on 14 March 2026.

Number of rows: 87

Number of columns: 3

Column name from data

Descriptive column Name

Units

Oracle data type

Column description

6. Data Description

SURVEY_DEFINITION_ID

Survey ID

ID key code

NUMBER(38,0)

The survey definition ID key code is an integer that uniquely identifies a survey region/survey design. The column `survey_definition_id` is associated with the `srvy` and `survey` columns. Full list of survey definition IDs are in `RACE_DATA.SURVEY_DEFINITIONS` and in the code books.

YEAR

Survey year

year

NUMBER(10,0)

Year the observation (survey) was collected.

DESIGN_YEAR

Design year

year

NUMBER(10,0)

Year ID associated with a given value `AREA_ID`. This field describes the changes in the survey design over time.

6.1.10. TAXONOMIC_CLASSIFICATION

Taxonomic classification information for the current list of `SPECIES_CODE` values. This table was created by the Resource Assessment and Conservation Engineering Division (RACE) Groundfish Assessment Program (GAP) of the Alaska Fisheries Science Center (AFSC). The GitHub repository for the scripts that created this code can be found at (https://github.com/afsc-gap-products/gap_products). There are no legal restrictions on access to the data. Last updated on 14 March 2026.

Number of rows: 4,872

Number of columns: 22

Column name from data

6. Data Description

Descriptive column Name

Units

Oracle data type

Column description

SURVEY_SPECIES

Species used in survey

logical

BINARY_DOUBLE

Designates whether or not species name is accepted/actively used in the RACE surveys

SPECIES_NAME

Scientific name of species

text

VARCHAR2(255 BYTE)

Scientific name of species.

COMMON_NAME

Taxon common name

text

VARCHAR2(255 BYTE)

The common name of the marine organism associated with the scientific_name and species_code columns. For a complete species list, review the code books.

SPECIES_CODE

Taxon code

ID key code

NUMBER(38,0)

The species code of the organism associated with the common_name and scientific_name columns. For a complete species list, review the code books.

ID_RANK

6. Data Description

Lowest taxonomic rank

text

VARCHAR2(255 BYTE)

Lowest taxonomic rank of a given species entry.

DATABASE

Database source

category

VARCHAR2(255 BYTE)

Taxonomic database source, either ITIS or WoRMS.

DATABASE_ID

Species ID in database

ID key code

VARCHAR2(255 BYTE)

Species ID key code of a species in the taxonomic "DATABASE" source.

GENUS_TAXON

Genus phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of genus of a given species.

SUBFAMILY_TAXON

Subfamily phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of subfamily of a given species.

FAMILY_TAXON

Family phylogenetic rank

category

6. Data Description

VARCHAR2(255 BYTE)

Phylogenetic latin rank of family of a given species.

SUPERFAMILY_TAXON

Superfamily phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of superfamily of a given species.

INFRAORDER_TAXON

Infraorder phylogenetic rank

category

VARCHAR2(255 BYTE)

Infraorder phylogenetic rank. Phylogenetic latin rank of infraorder of a given speices.

SUBORDER_TAXON

Suborder phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of suborder of a given species.

ORDER_TAXON

Order phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of order of a given species.

SUPERORDER_TAXON

Superorder phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of superorder of a given species.

6. Data Description

INFRAClass_TAXON

Infraclass phylogenetic rank

category

VARCHAR2(255 BYTE)

Infraclass phylogenetic rank. Phylogenetic latin rank of infraclass of a given speices.

SUBCLASS_TAXON

Subclass phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of subclass of a given species.

CLASS_TAXON

Class phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of class of a given species.

SUPERCLASS_TAXON

Superclass phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of superclass of a given species.

SUBPHYLUM_TAXON

Subphylum phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of subphylum of a given species.

PHYLUM_TAXON

Phylum phylogenetic rank

6. Data Description

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of phylum of a given species.

KINGDOM_TAXON

Kingdom phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of kingdom of a given species.

6.1.11. TAXON_GROUPS

GAP_PRODUCTS.TAXONOMIC_CLASSIFICATION subsetting for taxonomic classifications accepted by the GAP bottom trawl survey and added GROUP_CODE to denote taxonomic aggregations. This table was created by the Resource Assessment and Conservation Engineering Division (RACE) Groundfish Assessment Program (GAP) of the Alaska Fisheries Science Center (AFSC). The GitHub repository for the scripts that created this code can be found at (https://github.com/afsc-gap-products/gap_products). There are no legal restrictions on access to the data. Last updated on 25 October 2024.

Number of rows: 2,777

Number of columns: 22

Column name from data

Descriptive column Name

Units

Oracle data type

Column description

SPECIES_CODE

Taxon code

ID key code

NUMBER(38,0)

6. Data Description

The species code of the organism associated with the common_name and scientific_name columns. For a complete species list, review the code books.

SPECIES_NAME

Scientific name of species

text

VARCHAR2(255 BYTE)

Scientific name of species.

COMMON_NAME

Taxon common name

text

VARCHAR2(255 BYTE)

The common name of the marine organism associated with the scientific_name and species_code columns. For a complete species list, review the code books.

ID_RANK

Lowest taxonomic rank

text

VARCHAR2(255 BYTE)

Lowest taxonomic rank of a given species entry.

DATABASE

Database source

category

VARCHAR2(255 BYTE)

Taxonomic database source, either ITIS or WoRMS.

DATABASE_ID

Species ID in database

ID key code

VARCHAR2(255 BYTE)

Species ID key code of a species in the taxonomic "DATABASE" source.

6. Data Description

GENUS_TAXON

Genus phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of genus of a given species.

SUBFAMILY_TAXON

Subfamily phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of subfamily of a given species.

FAMILY_TAXON

Family phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of family of a given species.

SUPERFAMILY_TAXON

Superfamily phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of superfamily of a given species.

INFRAORDER_TAXON

Infraorder phylogenetic rank

category

VARCHAR2(255 BYTE)

Infraorder phylogenetic rank. Phylogenetic latin rank of infraorder of a given speices.

SUBORDER_TAXON

Suborder phylogenetic rank

6. Data Description

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of suborder of a given species.

ORDER_TAXON

Order phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of order of a given species.

SUPERORDER_TAXON

Superorder phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of superorder of a given species.

INFRAClass_TAXON

Infraclass phylogenetic rank

category

VARCHAR2(255 BYTE)

Infraclass phylogenetic rank. Phylogenetic latin rank of infraclass of a given speices.

SUBCLASS_TAXON

Subclass phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of subclass of a given species.

CLASS_TAXON

Class phylogenetic rank

category

VARCHAR2(255 BYTE)

6. Data Description

Phylogenetic latin rank of class of a given species.

SUPERCLASS_TAXON

Superclass phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of superclass of a given species.

SUBPHYLUM_TAXON

Subphylum phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of subphylum of a given species.

PHYLUM_TAXON

Phylum phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of phylum of a given species.

KINGDOM_TAXON

Kingdom phylogenetic rank

category

VARCHAR2(255 BYTE)

Phylogenetic latin rank of kingdom of a given species.

GROUP_CODE

Species or Complex ID

ID key code

NUMBER(38,0)

Equivalent to the SPECIES_CODE if the taxon is reported as a single taxon in GAP_PRODUCTS, otherwise denotes a SPECIES_CODE of a higher taxonomic group to which the taxon is aggregated in the GAP_PRODUCTS CPUE and BIOMASS tables.

6. Data Description

6.1.12. CRUISE

snapshot table for snapshot GAP_PRODUCTS.CRUISE

Number of rows: 180

Number of columns: 10

Column name from data

Descriptive column Name

Units

Oracle data type

Column description

CRUISEJOIN

Cruise ID

ID key code

NUMBER(38,0)

Unique integer ID assigned to each survey, vessel, and year combination.

CRUISE

Cruise Name

ID key code

NUMBER(38,0)

This is a six-digit integer identifying the cruise number of the form: YYYY99 (where YYYY = year of the cruise; 99 = 2-digit number and is sequential; 01 denotes the first cruise that vessel made in this year, 02 is the second, etc.).

YEAR

Survey year

year

NUMBER(10,0)

Year the observation (survey) was collected.

SURVEY_DEFINITION_ID

6. Data Description

Survey ID

ID key code

NUMBER(38,0)

The survey definition ID key code is an integer that uniquely identifies a survey region/survey design. The column `survey_definition_id` is associated with the `srvy` and `survey` columns. Full list of survey definition IDs are in `RACE_DATA.SURVEY_DEFINITIONS` and in the code books.

SURVEY_NAME

Survey name official

text

VARCHAR2(255 BYTE)

Long name of the survey conducted

VESSEL_ID

Vessel ID

ID key code

NUMBER(38,0)

ID number of the vessel used to collect data for that haul. The column `vessel_id` is associated with the `vessel_name` column. Note that it is possible for a vessel to have a new name but the same vessel id number. For a complete list of vessel ID key codes, review the code books.

VESSEL_NAME

Vessel name

text

VARCHAR2(255 BYTE)

Name of the vessel used to collect data for that haul. The column `vessel_name` is associated with the `vessel_id` column. Note that it is possible for a vessel to have a new name but the same vessel id number. For a complete list of vessel ID key codes, review the code books.

DATE_START

Start date

6. Data Description

YYYY-MM-DD

DATE

The date (YYYY-MM-DD) of the beginning of the event (e.g., cruise).

DATE_END

End date

YYYY-MM-DD

DATE

The date (YYYY-MM-DD) of the end of the event (e.g., cruise).

6.1.13. HAUL

snapshot table for snapshot GAP_PRODUCTS.HAUL

Number of rows: 34,839

Number of columns: 25

Column name from data

Descriptive column Name

Units

Oracle data type

Column description

PERFORMANCE

Haul performance code

category

NUMBER(38,0)

This denotes what, if any, issues arose during the haul. For more information, review the code books.

DATE_TIME_START

Start date and time

MM/DD/YYYY HH::MM

6. Data Description

TIMESTAMP

The date (MM/DD/YYYY) and time (HH:MM) of the beginning of the haul. All dates and times are in Alaska time (AKDT) of Anchorage, AK, USA (UTC/GMT -8 hours).

DURATION_HR

Tow duration (decimal hr)

hours

NUMBER(38,1)

This is the elapsed time between start and end of a haul (decimal hours).

DISTANCE_FISHED_KM

Distance fished (km)

kilometers

NUMBER(38,3)

Distance the net fished (kilometers).

NET_WIDTH_M

Net width (m)

meters

NUMBER(38,1)

Measured or estimated distance (meters) between wingtips of the trawl.

NET_MEASURED

Net measured during haul

logical

BINARY_DOUBLE

Logical, describing if the net was measured (TRUE) or not (FALSE) by wheelhouse and marport programs during the haul.

NET_HEIGHT_M

Net height (m)

meters

NUMBER(38,1)

6. Data Description

Measured or estimated distance (meters) between footrope and headrope of the trawl.

STRATUM

Stratum ID

ID key code

NUMBER(10,0)

RACE database statistical area for analyzing data. Strata were designed using bathymetry and other geographic and habitat-related elements. The strata are unique to each survey region. Stratum of value 0 indicates experimental tows.

LATITUDE_DD_START

Start latitude (decimal degrees)

decimal degrees

NUMBER(38,6)

Latitude (one hundred thousandth of a decimal degree) of the start of the haul.

LATITUDE_DD_END

End latitude (decimal degrees)

decimal degrees

NUMBER(38,6)

Latitude (one hundred thousandth of a decimal degree) of the end of the haul.

LONGITUDE_DD_START

Start longitude (decimal degrees)

decimal degrees

NUMBER(38,6)

Longitude (one hundred thousandth of a decimal degree) of the start of the haul.

LONGITUDE_DD_END

End longitude (decimal degrees)

decimal degrees

NUMBER(38,6)

6. Data Description

Longitude (one hundred thousandth of a decimal degree) of the end of the haul.

STATION

Station ID

ID key code

VARCHAR2(255 BYTE)

Alpha-numeric designation for the station established in the design of a survey.

DEPTH_GEAR_M

Depth of gear (m)

meters

NUMBER(38,1)

Depth of gear (meters).

DEPTH_M

Depth (m)

meters

NUMBER(38,1)

Bottom depth (meters).

BOTTOM_TYPE

Seafloor bottom type code

ID key code

NUMBER(38,0)

Bottom type on sea floor at haul location. For a complete list of bottom type ID key codes, review the code books.

SURFACE_TEMPERATURE_C

Surface temperature (degrees Celsius)

degrees Celsius

NUMBER(38,1)

Surface temperature (tenths of a degree Celsius); NA indicates removed or missing values.

6. Data Description

GEAR_TEMPERATURE_C

Gear temperature (degrees Celsius)

degrees Celsius

NUMBER(38,1)

Temperature recorded by net gear (tenths of a degree Celsius); NA indicates removed or missing values.

WIRE_LENGTH_M

Trawl wire length

meters

NUMBER(38,0)

Length of wire deployed during a given haul in meters.

GEAR

Type of gear used on the net

ID key code

NUMBER(38,0)

Type of gear used on net. For a complete list of gear ID key codes, review the code books.

ACCESSORIES

Type of gear accessories used on the net

ID key code

NUMBER(38,0)

Type of accessories used on net. For a complete list of accessories ID key codes, review the code books.

CRUISEJOIN

Cruise ID

ID key code

NUMBER(38,0)

Unique integer ID assigned to each survey, vessel, and year combination.

6. Data Description

HAULJOIN

Haul ID

ID key code

NUMBER(38,0)

This is a unique numeric identifier assigned to each (vessel, cruise, and haul) combination.

HAUL

Haul number

ID key code

NUMBER(38,0)

This number uniquely identifies a sampling event (haul) within a cruise. It is a sequential number, in chronological order of occurrence.

HAUL_TYPE

Haul sampling type

ID key code

NUMBER(38,0)

Type of haul sampling method. For a complete list of haul type ID key codes, review the code books.

6.1.14. LENGTH

snapshot table for snapshot GAP_PRODUCTS.LENGTH

Number of rows: 4,573,329

Number of columns: 7

Column name from data

Descriptive column Name

Units

Oracle data type

Column description

6. Data Description

HAULJOIN

Haul ID

ID key code

NUMBER(38,0)

This is a unique numeric identifier assigned to each (vessel, cruise, and haul) combination.

SPECIES_CODE

Taxon code

ID key code

NUMBER(38,0)

The species code of the organism associated with the common_name and scientific_name columns. For a complete species list, review the code books.

SEX

Sex of a specimen

ID key code

NUMBER(38,0)

Sex of a specimen where "1" = "Male", "2" = "Female", "3" = Unsexed.

FREQUENCY

Count of observation

count

NUMBER(38,0)

Frequency, or count, of an observation.

LENGTH_MM

Length of a specimen

millimeters

NUMBER(10,0)

Length bin in millimeters. A length of -9 indicates cases where no lengths were collected within a stratum for a species/year, even though catch numbers were recorded.

6. Data Description

LENGTH_TYPE

Length type

ID key code

NUMBER(38,0)

How the taxon was measured (e.g., fork length, carapace width). For a complete list of length_type ID key codes, review the code books.

SAMPLE_TYPE

Sample type

ID key code

NUMBER(38,0)

Sampling information on how the taxon was sampled. For a complete list of length_type ID key codes, review the code books.

6.1.15. SPECIMEN

snapshot table for snapshot GAP_PRODUCTS.SPECIMEN

Number of rows: 604,551

Number of columns: 12

Column name from data

Descriptive column Name

Units

Oracle data type

Column description

HAULJOIN

Haul ID

ID key code

NUMBER(38,0)

This is a unique numeric identifier assigned to each (vessel, cruise, and haul) combination.

6. Data Description

SPECIMEN_ID

Specimen unique ID

ID key code

NUMBER(38,0)

Each individual examined must have a number assigned to it that is unique within each haul (0001 to 9999), though specimen numbers may be repeated between hauls

SPECIES_CODE

Taxon code

ID key code

NUMBER(38,0)

The species code of the organism associated with the common_name and scientific_name columns. For a complete species list, review the code books.

LENGTH_MM

Length of a specimen

millimeters

NUMBER(10,0)

Length bin in millimeters. A length of -9 indicates cases where no lengths were collected within a stratum for a species/year, even though catch numbers were recorded.

SEX

Sex of a specimen

ID key code

NUMBER(38,0)

Sex of a specimen where "1" = "Male", "2" = "Female", "3" = Unsexed.

WEIGHT_G

Specimen weight (g)

grams

NUMBER(38,1)

Weight of specimen (grams).

6. Data Description

AGE

Taxon age bin (yrs)

integer

NUMBER(38,0)

Age bin of taxon. Age bin of a taxon in years estimated by the age comp estimate. Age -9 indicates unaged lengths for a particular sex because no otoliths were collected for that sex/length combination. Age -99 indicates a case where no lengths were collected within a stratum for a species/year even though catch numbers were recorded.

MATURITY

Specimen maturity code

ID key code

NUMBER(38,0)

The maturity code or the condition identified by the maturity code.

GONAD_G

Weight of gonads (g)

grams

NUMBER(38,1)

Weight of specimen gonads (grams).

SPECIMEN_SUBSAMPLE_METHOD

Specimen subsample method

ID key code

NUMBER(38,0)

For a complete list of specimen subsample method ID key codes, review the code books.

SPECIMEN_SAMPLE_TYPE

Specimen sample type

ID key code

NUMBER(38,0)

6. Data Description

The specimen sample type ID key code as defined in the RACE_DATA.SPECIMEN_SAMPLE_TYPES table. For a complete list of Specimen sample type ID key codes, review the code books.

AGE_DETERMINATION_METHOD

Aging method

ID key code

NUMBER(10,0)

Numeric code corresponding to the method of age determination. For a complete list of age determination codes, review the code books.

6.1.16. NA

NA

Number of rows: NA

Number of columns: NA

Column name from data

Descriptive column Name

Units

Oracle data type

Column description

7. Universal Column Metadata

This table is used to string together the various field comments for the tables in GAP_PRODUCTS. This table was created by the Resource Assessment and Conservation Engineering Division (RACE) Groundfish Assessment Program (GAP) of the Alaska Fisheries Science Center (AFSC). The GitHub repository for the scripts that created this code can be found at (https://github.com/afsc-gap-products/gap_products). There are no legal restrictions on access to the data. Last updated on 14 March 2026.

7. Universal Column Metadata

Table 7.1.: Universal stock metadata that users can use to document their table columns.

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
ABUNDANCE_-HAUL	Design-based index approved haul	logical	VAR-CHAR2(255 BYTE)	Logical, describing if this haul was conducted in a standard manner and thus used for design-based index estimates (TRUE) or not (FALSE).

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
ACCES-SORIES	Type of gear acces-sories used on the net	ID key code	NUM-BER(38,0)	Type of acces-sories used on net. For a complete list of acces-sories ID key codes, review the [code books](https://www.fish-eries.noaa.gov/re-source/doc-u-ment/groundfish-survey-species-code-manual-and-data-codes-manual).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column description
ACTION	Database action	text	VAR-CHAR2(255 BYTE)	Standard action taken to alter current database record
ACTIVE	Vessel active/in-active	logical	VAR-CHAR2(255 BYTE)	Logical, describing if a vessel is active (TRUE) or not (FALSE).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
AGE	Taxon age bin (yrs)	integer	NUM- BER(38,0)	Age bin of taxon. Age bin of a taxon in years esti- mated by the age comp esti- mate. Age -9 indi- cates unaged lengths for a particu- lar sex because no otoliths were col- lected for that sex/length combi- nation. Age -99 indi- cates a case where no lengths were col- lected within a stratum for a species/year even

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column description
AGENCY_ ACRONYM	Acronym of listed Agency	text abbrevi- ated	VAR- CHAR2(255 BYTE)	Abbrevi- ated agen- cies that are affiliated with the Alaska bottom trawl survey. The column agency_ acronym is asso- ciated with the agency_ short and agency_ long columns.
AGENCY_ JOIN	Agency ID	ID key code	NUM- BER(38,0)	Affiliated agency ID key code.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
AGENCY_LONG	Official name of agency	text	VAR-CHAR2(255 BYTE)	Full official name of affiliated agencies to the Alaska bottom trawl survey. The column agency_long is associated with the agency_acronym and agency_short columns.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
AGENCY_ SHORT	Agency short- hand name	text	VAR- CHAR2(255 BYTE)	A sort version of the full official name of affiliated agen- cies to the Alaska bottom trawl survey. The column agency_ short is associ- ated with the agency_ acronym and agency_ long columns.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
AGE_- DETER- MINA- TION_- METHOD	Aging method	ID key code	NUM- BER(10,0)	Numeric code corresponding to the method of age determination. For a complete list of age determination codes, review the [code books](https://www.fisheries.noaa.gov/resource/document/groundfish-survey-species-code-manual-and-data-codes-manual).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column description
AGE_- DETER- MINA- TION_- METH- ODS	Age determi- nation method	ID key code	NUM- BER(38,0)	A unique ID used to identify this age determi- nation method.
AGE_- YEAR	Age bin of taxon	year	NUM- BER(38,0)	Age bin of a taxon in years esti- mated by the age comp esti- mate.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
AREA- JOIN	Area polygon ID	ID key code	NUM- BER(38,0)	A call sign is a desig- nated se- quence of letters and numbers that are as- signed when a vessel, whether it be a sailing yacht, motor yacht, rib or com- mercial vessel, receives its Ship Radio Licence. The vessel also receives its MMSI number, so that each vessel is uniquely identi- fied.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
AREA_ ID	Area ID	ID key code	NUM- BER(38,0)	An ID code representing a broad range of spatial areas from statistical sampling strata to management and regional areas composed of multiple strata.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
AREA_ ID_ FOOT- PRINT	Survey Foot- print	text	VAR- CHAR2(40 BYTE)	Survey footprint, usually equivalent to the SURVEY_DEFINITION_ID with the exception of the Standard and Standard +NW survey footprints in the Eastern Bering Sea shelf bottom trawl survey.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
AREA_ KM2	Area (km2)	kilome- ters squared	NUM- BER(38,3)	Area in square kilome- ters.
AREA_ NAME	Area ID name	text	VAR- CHAR2(40 BYTE)	Descrip- tive name of each AREA_ ID. These names often identify the region, depth ranges, or other regional informa- tion for the area ID.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
AREA_- SWEPT_- KM2	Area swept (km)	kilometers	NUMBER(38,6)	The area the net covered while the net was fishing (kilometers squared), defined as the distance fished times the net width.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
AREA_ TYPE	Area ID type de- scription	category	VAR- CHAR2(255 BYTE)	The type of stratum that AREA_ID represents. Types include: STRATUM (the smallest building-block unit of area in these surveys), REGION, DEPTH, SUB-AREA, INPFC BY DEPTH, INPFC, SUB-AREA BY DEPTH, REGULATORY AREA, NMFS STATISTICAL AREA.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
BIOMASS-MT	Estimated biomass	numeric	NUMBER(38,6)	The estimated total biomass.
BIOMASS-VAR	Estimated biomass variance	numeric	NUMBER(38,6)	The estimated variance associated with the total biomass.
BOTTOM-TEMPERATURE-C	Bottom temperature (degrees Celsius)	degrees Celsius	NUMBER(38,1)	Bottom temperature (tenths of a degree Celsius); NA indicates removed or missing values.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
BOT- TOM_ TYPE	Seafloor bottom type code	ID key code	NUM- BER(38,0)	Bottom type on sea floor at haul location. For a com- plete list of bottom type ID key codes, review the [code books](https://www.fisheries.noaa.gov/resource/document/groundfish-survey-species-code-manual-and-data-codes-manual).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
CATALOG_NUM	Catalog number	text	VAR-CHAR2(255 BYTE)	Museum catalog number associated with record
CATCHJC	Catch observation ID	ID key code	NUMBER(38,0)	Unique integer ID assigned to each survey, vessel, year, and catch observation combination.
CLASSIFICATION	Taxonomic classification rank group	category	VAR-CHAR2(255 BYTE)	Phylogenetic classification group rank for a given species.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column description
CLASS_ TAXON	Class phyloge- netic rank	category	VAR- CHAR2(255 BYTE)	Phylo- genetic latin rank of class of a given species.
COL- LECTED_ BY	Person who col- lected speci- men	text	VAR- CHAR2(255 BYTE)	Initials of person who col- lected speci- men in the field
COM- MENTS	Com- ments	text	VAR- CHAR2(4096 BYTE)	Com- ments regard- ing row observa- tion.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
COM-MON_-NAME	Taxon common name	text	VAR-CHAR2(255 BYTE)	The common name of the marine organism associated with the scientific_ - name and species_ - code columns. For a complete species list, review the [code books](https://www.fisheries.noaa.gov/re-source/doc-ument/groundfish-survey-species-code-manual-and-data-codes-manual).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
COUNT	Taxon count	count, whole number resolu- tion	NUM- BER(38,0)	Total whole number of indi- viduals caught in haul or samples col- lected.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
COUN- TRY_ID	Country code	ID key code	NUM- BER(38,0)	Country ID key code of where a vessel, for ex- ample, may be from. For a com- plete list of country ID key codes, review the [code books](https://www.fisheries.noaa.gov/resource/document/groundfish-survey-species-code-manual-and-data-codes-manual).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
CPUE_- KGHA	Weight CPUE (kg/ha)	kilo- grams per hectare	NUM- BER(38,6)	Catch weight (kilo- grams) per unit effort (area swept by the net, units hectares).
CPUE_- KGKM2	Weight CPUE (kg/km2)	kilo- grams per kilo- meters squared	NUM- BER(38,6)	Catch weight (kilo- grams) per unit effort (area swept by the net, units square kilome- ters).

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
CPUE_-KGKM2_-MEAN	Mean weight CPUE	kilo-grams per kilo-meters squared	NUM-BER(38,6)	The mean catch weight (kilo-grams) per unit effort (area swept by the net, units squared kilome-ters).
CPUE_-KGKM2_-VAR	Variance of the mean weight CPUE	kilo-grams per kilo-meters squared	NUM-BER(38,6)	The variance of mean catch weight (kilo-grams) per unit effort (area swept by the net, units squared kilome-ters).

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
CPUE_-NOHA	Number CPUE (no/ha)	count per hectare	NUMBER(38,6)	Numerical catch per unit effort (area swept by the net, units hectares).
CPUE_-NOKM2	Number CPUE (no/km2)	count per kilometers squared	NUMBER(38,6)	Numerical catch per unit effort (area swept by the net, units square kilometers).

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
CPUE_- NOKM2_- MEAN	Mean numeric CPUE	count per kilometers squared	NUMBER(38,6)	The mean of numerical catch per unit effort (area swept by the net, units square kilometers).
CPUE_- NOKM2_- VAR	Variance of the mean numeric CPUE	count per kilometers squared	NUMBER(38,6)	The variance of mean numerical catch per unit effort (area swept by the net, units square kilometers).

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
CRS	Coordinate reference system	ID key code	VAR-CHAR2(255 BYTE)	The coordinate reference system (CRS) that shape-files were created in or areas (like AREA_KM2) are calculated in, as defined by https://spatialreference.org/ (e.g., "+proj=longlat", "EPSG:3338").

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
CRUISE	Cruise Name	ID key code	NUM- BER(38,0)	This is a six-digit integer identifying the cruise number of the form: YYYY99 (where YYYY = year of the cruise; 99 = 2-digit number and is sequential; 01 denotes the first cruise that vessel made in this year, 02 is the second, etc.).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column description
CRUISE-JOIN	Cruise ID	ID key code	NUMBER(38,0)	Unique integer ID assigned to each survey, vessel, and year combination.
DATABASE	Database source	category	VARCHAR2(255 BYTE)	Taxonomic database source, either ITIS or WoRMS.
DATABAS ID	Species ID in database	ID key code	VARCHAR2(255 BYTE)	Species ID key code of a species in the taxonomic "DATABASE" source.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
DATE	Date	YYYY-MM-DD	DATE	The date (YYYY-MM-DD) of the event (e.g., cruise).
DATE_- END	End date	YYYY-MM-DD	DATE	The date (YYYY-MM-DD) of the end of the event (e.g., cruise).
DATE_- START	Start date	YYYY-MM-DD	DATE	The date (YYYY-MM-DD) of the beginning of the event (e.g., cruise).

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
DATE_ TIME	Date and time	MM/DD/Y' HH::MM	DATE	The date (MM/DD/YYYY) and time (HH:MM) of the haul. All dates and times are in Alaska time (AKDT) of Anchorage, AK, USA (UTC/GMT -8 hours).

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
DATE_- TIME_- END	End date and time	MM/DD/YYYY HH::MM	TIMES- TAMP	The date (MM/DD/YYYY) and time (HH:MM) of the end of the haul. All dates and times are in Alaska time (AKDT) of Anchorage, AK, USA (UTC/GMT -8 hours).

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
DATE_- TIME_- START	Start date and time	MM/DD/Y' HH::MM	TIMES-TAMP	The date (MM/DD/YYYY) and time (HH:MM) of the beginning of the haul. All dates and times are in Alaska time (AKDT) of Anchorage, AK, USA (UTC/GMT -8 hours).
DEPTH_- GEAR_- M	Depth of gear (m)	meters	NUM-BER(38,1)	Depth of gear (meters).
DEPTH_- M	Depth (m)	meters	NUM-BER(38,1)	Bottom depth (meters).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column description
DEPTH_ MAX_M	Area ID maxi- mum depth (m)	meters	NUM- BER(38,3)	Maxi- mum depth (me- ters).
DEPTH_ MIN_M	Area ID mini- mum depth (m)	meters	NUM- BER(38,3)	Mini- mum depth (me- ters).
DE- SCRIP- TION	Descrip- tion	text	VAR- CHAR2(4000 BYTE)	Descrip- tion of how observa- tion.
DE- SIGN_ YEAR	Design year	year	NUM- BER(10,0)	Year ID associ- ated with a given value AREA_ ID. This field de- scribes the changes in the survey design over time.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
DIS- TANCE_ FISHED_ KM	Distance fished (km)	kilome- ters	NUM- BER(38,3)	Distance the net fished (kilome- ters).
DUMMY	dummy	dummy	VAR- CHAR2(255 BYTE)	dummy.
DURA- TION_ HR	Tow duration (decimal hr)	hours	NUM- BER(38,1)	This is the elapsed time between start and end of a haul (decimal hours).
FAM- ILY_ TAXON	Family phyloge- netic rank	category	VAR- CHAR2(255 BYTE)	Phylo- genetic latin rank of family of a given species.
FIELD_ ID	Field speci- men identifi- cation	text	VAR- CHAR2(255 BYTE)	Field identifi- cation for the vouchered speci- men

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
FRE- QUENCY	Count of observation	count	NUM- BER(38,0)	Fre- quency, or count, of an observa- tion.
GEAR	Type of gear used on the net	ID key code	NUM- BER(38,0)	Type of gear used on net. For a complete list of gear ID key codes, review the [code books](https://www.fisheries.noaa.gov/resource/document/groundfish-survey-species-code-manual-and-data-codes-manual).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
GEAR_- DEPTH_- M	Gear depth	meters	NUM- BER(38,1)	Depth gear was de- ployed at (tenths of a meter). Gear depth plus net height equals bottom depth.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
GEAR_ ID	Gear ID	ID key code	NUM- BER(38,0)	Type of trawl or gear deployed. For a complete list of vessel gear type ID key codes, review the [code books](https://www.fisheries.noaa.gov/resource/document/groundfish-survey-species-code-manual-and-data-codes-manual).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
GEAR_- TEM- PERA- TURE_- C	Gear temper- ature (de- grees Celsius)	degrees Celsius	NUM- BER(38,1)	Temper- ature recorded by net gear (tenths of a degree Celsius); NA indi- cates removed or missing values.
GENUS_- TAXON	Genus phyloge- netic rank	category	VAR- CHAR2(255 BYTE)	Phylo- genetic latin rank of genus of a given species.
GEOM- ENTRY	Spatial geome- try	text	VAR- CHAR2(255 BYTE)	Spatial geome- try informa- tion (like points, lines, or poly- gons) a feature.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
GO- NAD_G	Weight of gonads (g)	grams	NUM- BER(38,1)	Weight of speci- men gonads (grams).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
GROUP_ CODE	Species or Com- plex ID	ID key code	NUM- BER(38,0)	Equiva- lent to the SPECIES_ CODE if the taxon is reported as a single taxon in GAP_ PROD- UCTS, other- wise denotes a SPECIES_ CODE of a higher taxo- nomic group to which the taxon is aggre- gated in the GAP_ PROD- UCTS CPUE and BIOMASS tables.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
HAUL	Haul number	ID key code	NUMBER(38,0)	This number uniquely identifies a sampling event (haul) within a cruise. It is a sequential number, in chronological order of occurrence.
HAULJOIN	Haul ID	ID key code	NUMBER(38,0)	This is a unique numeric identifier assigned to each (vessel, cruise, and haul) combination.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
HAUL_- TYPE	Haul sam- pling type	ID key code	NUM- BER(38,0)	Type of haul sam- pling method. For a com- plete list of haul type ID key codes, review the [code books](https://www.fisheries.noaa.gov/resource/document/groundfish-survey-species-code-manual-and-data-codes-manual).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
ID_- RANK	Lowest taxo- nomic rank	text	VAR- CHAR2(255 BYTE)	Lowest taxo- nomic rank of a given species entry.
INFRA- CLASS_- TAXON	Infra- class phyloge- netic rank	category	VAR- CHAR2(255 BYTE)	Infra- class phyloge- netic rank. Phylo- genetic latin rank of infra- class of a given speices.
IN- FRAORDE TAXON	In- fraorder phyloge- netic rank	category	VAR- CHAR2(255 BYTE)	In- fraorder phyloge- netic rank. Phylo- genetic latin rank of in- fraorder of a given speices.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
ITIS	Inte- grated taxo- nomic informa- tion system (ITIS) serial number	ID key code	NUM- BER(38,0)	Species code as identi- fied in the Inte- grated Taxo- nomic Informa- tion System (https://itis.gov/).
KING- DOM_ TAXON	King- dom phyloge- netic rank	category	VAR- CHAR2(255 BYTE)	Phylo- genetic latin rank of kingdom of a given species.
LATI- TUDE_ DD	Latitude (decimal de- grees)	decimal degrees	NUM- BER(38,6)	Latitude (one hundred thou- sandth of a decimal degree).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column description
LATI- TUDE_- DD_- END	End latitude (decimal de- grees)	decimal degrees	NUM- BER(38,6)	Latitude (one hundred thou- sandth of a decimal degree) of the end of the haul.
LATI- TUDE_- DD_- START	Start latitude (decimal de- grees)	decimal degrees	NUM- BER(38,6)	Latitude (one hundred thou- sandth of a decimal degree) of the start of the haul.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
LENGTH_MM	Length of a specimen	millime- ters	NUM- BER(10,0)	Length bin in millime- ters. A length of -9 indi- cates cases where no lengths were col- lected within a stratum for a species/year, even though catch numbers were recorded.
LENGTH_MM_- MEAN	Mean length at age weighted by numbers at length	numeric	NUM- BER(38,3)	Mean length (millime- ters).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column description
LENGTH_MM_SD	Standard deviation of length at age weighted by numbers at length	numeric	NUMBER(38,3)	Variance of mean length.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column description
LENGTH_ TYPE	Length type	ID key code	NUM- BER(38,0)	How the taxon was mea- sured (e.g., fork length, cara- pace width). For a com- plete list of length_ - type ID key codes, review the [code books](https://www.fisheries.noaa.gov/resource/docu-ment/groundfish-survey-species-code-manual-and-data-codes-manual).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
LONGI- TUDE_ DD	Longi- tude (decimal de- grees)	decimal degrees	NUM- BER(38,6)	Longi- tude (one hundred thou- sandth of a decimal degree).
LONGI- TUDE_ DD_ END	End lon- gitude (decimal de- grees)	decimal degrees	NUM- BER(38,6)	Longi- tude (one hundred thou- sandth of a decimal degree) of the end of the haul.
LONGI- TUDE_ DD_ START	Start longi- tude (decimal de- grees)	decimal degrees	NUM- BER(38,6)	Longi- tude (one hundred thou- sandth of a decimal degree) of the start of the haul.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
MATURITY	Specimen maturity code	ID key code	NUMBER(38,0)	The maturity code or the condition identified by the maturity code.
META-DATA_-COL-NAME	Column name	text	VARCHAR2(4000 BYTE)	Name of the column in a table.
META-DATA_-COL-NAME_-DESC	Column description	text	VARCHAR2(4000 BYTE)	Description of the column.
META-DATA_-COL-NAME_-LONG	Column name spelled out	text	VARCHAR2(4000 BYTE)	Long name for the column.
META-DATA_-DATATYPE	Oracle datatype code	text	VARCHAR2(4000 BYTE)	Oracle data type of data column.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
META-DATA_-SEN-TENCE	Sentence	text	VAR-CHAR2(4000 BYTE)	Table meta-data sentence.
META-DATA_-SEN-TENCE_-NAME	Meta-data sentence name	text	VAR-CHAR2(4000 BYTE)	Name of table meta-data sentence.
META-DATA_-SEN-TENCE_-TYPE	Sentence type	text	VAR-CHAR2(4000 BYTE)	Type of sentence to have in table meta-data.
META-DATA_-UNITS	Units	category	VAR-CHAR2(4000 BYTE)	Units of the column.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
NET_- HEIGHT_- M	Net height (m)	meters	NUM- BER(38,1)	Mea- sured or esti- mated distance (meters) between footrope and head- rope of the trawl.
NET_- MEA- SURED	Net mea- sured during haul	logical	BI- NARY_- DOU- BLE	Logical, describ- ing if the net was mea- sured (TRUE) or not (FALSE) by wheel- house and marport pro- grams during the haul.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
NET_ WIDTH_ M	Net width (m)	meters	NUM- BER(38,1)	Mea- sured or esti- mated distance (meters) between wingtips of the trawl.
NEW_ID	New speci- men identifi- cation	text	VAR- CHAR2(255 BYTE)	Con- firmed tax- onomist identifi- cation of the vouchered speci- men
NEW_ SPECIES_ CODE	New species code	ID key code	NUM- BER(10,0)	Species code as- sociated with new species name
NEW_ SPECIES_ NAME	New species name	text	VAR- CHAR2(255 BYTE)	Updated taxo- nomic name

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
N_-COUNT	Hauls with taxon counts	numeric	NUMBER(38,0)	Total number of hauls with positive count data.
N_-HAUL	Valid hauls	count	NUMBER(38,0)	Total number of hauls.
N_-LENGTH	Hauls with taxon lengths	count	NUMBER(38,0)	Total number of hauls with length data.
N_-SAMPLE	Hauls with sample	count	NUMBER(38,0)	Total number of hauls with positive sample collection.
N_-SPECIMENS	Number of specimens in the lot	count	NUMBER(38,0)	Number of specimens in the voucher lot

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column description
N_- WEIGHT	Hauls with catch	count	NUM- BER(38,0)	Total number of hauls with positive catch biomass.
OLD_- SPECIES_ CODE	Old species code	ID key code	NUM- BER(10,0)	Species code associated with old species name
OLD_- SPECIES_ NAME	Old species name	text	VAR- CHAR2(255 BYTE)	Taxonomic name previously used in the database
OR- DER_- TAXON	Order phylogenetic rank	category	VAR- CHAR2(255 BYTE)	Phylogenetic latin rank of order of a given species.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
PER-FOR-MANCE	Haul performance code	category	NUMBER(38,0)	This denotes what, if any, issues arose during the haul. For more information, review the [code books](https://www.fisheries.noaa.gov/resource/document/groundfish-survey-species-code-manual-and-data-codes-manual).

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
PHY-LUM_TAXON	Phylum phylogenetic rank	category	VAR-CHAR2(255 BYTE)	Phylogenetic latin rank of phylum of a given species.

7. Universal Column Metadata

Column name from data	De-scrip-tive column Name	Units	Oracle data type	Column description
POLYGON_WKB	Polygon binary string	code string	VAR-CHAR2(255 BYTE)	Well-known binary (WKB) representation of geometry for a AREA_JOIN polygon. WKB is used to transfer and store the same information in a more compact form convenient for computer processing but that is not human-readable.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
POLY- GON_ WKT	Polygon well known text	code string	VAR- CHAR2(255 BYTE)	Well- known text (WKT) repre- sentation of geome- try for a AREA_ JOIN polygon. WKT is a text markup lan- guage for repre- senting vector geome- try objects.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
POPULATION_COUNT	Estimated population	numeric	NUMBER(38,0)	The estimated population caught in the survey for a species, group, or total for a given survey.
POPULATION_VAR	Estimated population variance	numeric	NUMBER(38,6)	The estimated population variance caught in the survey for a species, group, or total for a given survey.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
PRESERVATIVE	Chemical specimen stored in	text	VAR-CHAR2(255 BYTE)	Chemical specimen currently stored in
PRINCIPAL_INVESTIGATOR	Principle investigator	text	VAR-CHAR2(255 BYTE)	First and last name of principal investigator for a project.
PROJECT_TITLE	Title of special project	text	VAR-CHAR2(255 BYTE)	Special project title.
PROJECT_TITLE_SHORT	Short title of special project	text	VAR-CHAR2(255 BYTE)	Special project short title (short version of PROJECT_TITLE).
RANK_ID	Taxonomic rank	category	VAR-CHAR2(255 BYTE)	The taxonomic rank of a taxon identification.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
REA- SON	Reason for taxo- nomic change	text	VAR- CHAR2(255 BYTE)	Reason for taxo- nomic change; pulled directly from online database (i.e. WoRMS or ITIS)

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
SAM- PLE_- TYPE	Sample type	ID key code	NUM- BER(38,0)	Sam- pling informa- tion on how the taxon was sam- pled. For a com- plete list of length_- type ID key codes, review the [code books](https://www.fish- eries.noaa.gov/re- source/doc- u- ment/groundfish- survey- species- code- manual- and- data- codes- manual).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
SCIEN- TIFIC_ NAME	Taxon scientific name	text	VAR- CHAR2(255 BYTE)	The scientific name of the or- ganism associ- ated with the com- mon_ name and species_ code columns. For a com- plete taxon list, review the [code books](https://www.fisheries.noaa.gov/resource/document/groundfish-survey-species-code-manual-and-data-codes-manual).

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
SEX	Sex of a specimen	ID key code	NUMBER(38,0)	Sex of a specimen where "1" = "Male", "2" = "Female", "3" = Un-sexed.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
SPECIES_	Taxon	ID key	NUM-	The
CODE	code	code	BER(38,0)	species code of the or- ganism associ- ated with the com- mon_- name and sci- entific_- name columns. For a com- plete species list, review the [code books](https://www.fish- eries.noaa.gov/re- source/doc- u- ment/groundfish- survey- species- code- manual- and- data- codes- manual).

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
SPECIES_NAME	Scientific name of species	text	VAR-CHAR2(255 BYTE)	Scientific name of species.
SPECIES_NAME_ACCEPTED	Scientific name used in taxonomic database	text	VAR-CHAR2(255 BYTE)	Scientific name of species used in taxonomic "DATABASE" column.
SPECIES_NAME_SURVEY	Scientific name used in survey data	text	VAR-CHAR2(255 BYTE)	Scientific name of species historically or currently used in the survey.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
SPECIMEN_ID	Specimen unique ID	ID key code	NUMBER(38,0)	Each individual examined must have a number assigned to it that is unique within each haul (0001 to 9999), though specimen numbers may be repeated between hauls

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
SPECIMEN_-SAMPLE_-TYPE	Specimen sample type	ID key code	NUMBER(38,0)	The specimen sample type ID key code as defined in the RACE_-DATA.SPECIMEN_-SAMPLE_-TYPES table. For a complete list of Specimen sample type ID key codes, review the [code books](https://www.fisheries.noaa.gov/resource/document/groundfish-survey-species-code-manual-and-data-codes-manual).

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
SPECI-MEN_-SUB-SAMPLE_-METHOD	Speci-men subsam-ple method	ID key code	NUM-BER(38,0)	For a complete list of speci-men subsam-ple method ID key codes, review the [code books](https://www.fisheries.noaa.gov/resource/docu-ment/groundfish-survey-species-code-manual-and-data-codes-manual).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
SRVY	Survey abbrevi- ation	text abbrevi- ated	VAR- CHAR2(255 BYTE)	Abbrevi- ated survey names. The column srvy is associ- ated with the survey and survey_ defini- tion_id columns. Northern Bering Sea (NBS), South- eastern Bering Sea (EBS), Bering Sea Slope (BSS), Gulf of Alaska (GOA), Aleutian Islands (AI).

7. Universal Column Metadata

Column name from data	De-scrip-tive column Name	Units	Oracle data type	Column description
STANDARD_LENGTH_MM	Standard length of specimens (mm)	numeric	VAR-CHAR2(255 BYTE)	Standard length of specimen or range of lengths if multiple specimens in lot; measured by taxonomists in lab
STATION	Station ID	ID key code	VAR-CHAR2(255 BYTE)	Alpha-numeric designation for the station established in the design of a survey.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
STRATUM	Stratum ID	ID key code	NUMBER(10,0)	RACE database statistical area for analyzing data. Strata were designed using bathymetry and other geographic and habitat-related elements. The strata are unique to each survey region. Stratum of value 0 indicates experimental tows.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
SUB-CLASS_-TAXON	Sub-class phylogenetic rank	category	VAR-CHAR2(255 BYTE)	Phylogenetic latin rank of subclass of a given species.
SUB-FAMILY_-TAXON	Subfamily phylogenetic rank	category	VAR-CHAR2(255 BYTE)	Phylogenetic latin rank of subfamily of a given species.
SUB-MISSION_-DATE	Date	YYYY-MM-DD	DATE	Date special projects were due to be submitted for the upcoming survey season.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
SUBORDER_TAXON	Suborder phylogenetic rank	category	VAR-CHAR2(255 BYTE)	Phylogenetic latin rank of suborder of a given species.
SUBPHY-LUM_TAXON	Subphylum phylogenetic rank	category	VAR-CHAR2(255 BYTE)	Phylogenetic latin rank of subphylum of a given species.
SUPER-CLASS_TAXON	Superclass phylogenetic rank	category	VAR-CHAR2(255 BYTE)	Phylogenetic latin rank of super-class of a given species.
SUPER-FAMILY_TAXON	Superfamily phylogenetic rank	category	VAR-CHAR2(255 BYTE)	Phylogenetic latin rank of super-family of a given species.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
SUPER-ORDER-TAXON	Super-order phylogenetic rank	category	VAR-CHAR2(255 BYTE)	Phylogenetic latin rank of super-order of a given species.
SURFACE-TEMPERATURE-C	Surface temperature (degrees Celsius)	degrees Celsius	NUMBER(38,1)	Surface temperature (tenths of a degree Celsius); NA indicates removed or missing values.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
SUR- VEY	Survey name	text	VAR- CHAR2(255 BYTE)	Name and de- scription of survey. The column survey is asso- ciated with the srvy and survey_ defini- tion_id columns.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
-----------------------	---	-------	------------------------	----------------------------

SUR-
VEY_-
DEFINI-
TION_-
ID

Survey
ID

ID key
code

NUM-
BER(38,0)

The survey definition ID key code is an integer that uniquely identifies a survey region/survey design. The column survey_definition_id is associated with the srvy and survey columns. Full list of survey definition IDs are in RACE_DATA.SURVEY_DEFINITIONS and in the [code books](<https://www.fisheries.noaa.gov/resource/doc->
U-

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
SUR-VEY_ID	Survey ID raw	ID key code	NUMBER(38,0)	The survey ID uniquely identifies a survey instance.
SUR-VEY_NAME	Survey name official	text	VAR-CHAR2(255 BYTE)	Long name of the survey conducted
SUR-VEY_SPECIES	Species used in survey	logical	BI-NARY-DOUBLE	Designates whether or not species name is accepted/actively used in the RACE surveys

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
TAX- ONOMIST	Tax- onomist	text	VAR- CHAR2(255 BYTE)	Tax- onomist(s) who re- identified speci- men(s)

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
				Confidence in the ability of the survey team to correctly identify the taxon to the specified level, based solely on identification skill (e.g., not likelihood of a taxon being caught at that station on a location-by-location basis). Quality codes follow: **High**: High confidence and consistency. Taxonomy is stable and

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
				Confidence in the ability of the survey team to correctly identify the taxon to the specified level, based solely on identification skill (e.g., not likelihood of a taxon being caught at that station on a location-by-location basis). Quality codes follow: **High**: High confidence and consistency. Taxonomy is stable and

7. Universal Column Metadata

Column name from data	De-scrip-tive column Name	Units	Oracle data type	Column description
TRAWLAE	Trawlable stations	logical	BI-NARY-DOUBLE	Logical, describing if stations are trawlable (TRUE) or not (FALSE).

7. Universal Column Metadata

Column name from data	De-scrip-tive column Name	Units	Oracle data type	Column description
VESSEL_CALLSIGN	Vessel call sign	ID key code	NUMBER(38,0)	A call sign is a designated sequence of letters and numbers that are assigned when a vessel, whether it be a sailing yacht, motor yacht, rib or commercial vessel, receives its Ship Radio Licence. The vessel also receives its MMSI number, so that each vessel is uniquely identified.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
VESSEL_COAST_GUARD_NUMBER	Vessel coast guard number	ID key code	NUMBER(38,0)	Official Identification number as defined by www.dco.uscg.mil . The Official Number (O/N) is the 6 or 7 digit number awarded to the vessel at the time it is first documented with the US Coast Guard. This number remains with the vessel indefinitely and should be marked in accordance with 46 CFR 67.121.

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
-----------------------	---	-------	------------------------	----------------------------

VES-
SEL_ID

Vessel
ID

ID key
code

NUM-
BER(38,0)

ID number of the vessel used to collect data for that haul. The column vessel_id is associated with the vessel_name column. Note that it is possible for a vessel to have a new name but the same vessel id number. For a complete list of vessel ID key codes, review the [code books](<https://www.fisheries.noaa.gov/resource/docu-ment/groundfish->

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
VES-SEL_-IMO	Vessel international maritime organization number	ID key code	NUMBER(38,0)	The International Maritime Organization (IMO) number consists of the letters "IMO" followed by a unique, seven-digit number: the pattern is "NNNNNNN", where N is a single-digit number, e.g., "1234567"
VES-SEL_-LENGTH_-M	Vessel length (m)	meters	NUMBER(38,0)	The length of vessel in meters.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
VES-SEL_-MMSI	Vessel maritime mobile service identities	ID key code	NUMBER(38,0)	Maritime Mobile Service Identities (MMSIs) are nine-digit numbers used by maritime digital selective calling (DSC), automatic identification systems (AIS) and certain other equipment to uniquely identify a ship or a coast radio station.

7. Universal Column Metadata

Column name from data	De-scriptive column Name	Units	Oracle data type	Column description
-----------------------	--------------------------	-------	------------------	--------------------

VES-
SEL_-
NAME

Vessel
name

text

VAR-
CHAR2(255
BYTE)

Name of the vessel used to collect data for that haul. The column vessel_ - name is associated with the vessel_ - id column. Note that it is possible for a vessel to have a new name but the same vessel id number. For a complete list of vessel ID key codes, review the [code books](<https://www.fisheries.noaa.gov/resource/document/groundfish-survey->

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column descrip- tion
VES- SEL_- OWNER	Vessel owner	text	VAR- CHAR2(255 BYTE)	Name of vessel owner or com- pany.
VES- SEL_- TON- NAGE	Vessel tonnage	metric tons	NUM- BER(38,0)	The tonnage of vessel in metric tons.
VOUCHER	Voucher number	numeric	NUM- BER(38,0)	The voucher number of the speci- men within a single haul
WEIGHT_ G	Speci- men weight (g)	grams	NUM- BER(38,1)	Weight of speci- men (grams).
WEIGHT_ KG	Sample or taxon weight (kg)	kilo- grams	NUM- BER(38,3)	Total weight (kilo- grams).

7. Universal Column Metadata

Column name from data	De- scrip- tive column Name	Units	Oracle data type	Column description
WIRE_- LENGTH_ M	Trawl wire length	meters	NUM- BER(38,0)	Length of wire deployed during a given haul in meters.
WORMS	World register of marine species (WoRMS) taxonomic serial number	ID key code	NUM- BER(38,0)	Species code as identified in the World Register of Marine Species (WoRMS) (https://www.marine-species.org/).
YEAR	Survey year	year	NUM- BER(10,0)	Year the observation (survey) was collected.
YEAR_- CHANGEI	Year changed	numeric	DATE	Year change implemented in database

8. Access data via Oracle and R

Access data via Oracle (AFSC only)

AFSC Oracle users can access the database via SQL developer to view and pull the production data directly from the GAP_PRODUCTS Oracle schema. The user can also use SQL developer to view and pull the GAP Products data directly from the GAP_PRODUCTS Oracle schema.

8.0.1. Connect to Oracle from R

Many users will want to access the data from Oracle using R. The user will need to install the RORDBC R package and ask OFIS (IT) connect R to Oracle. Then, use the following code in R to establish a connection from R to Oracle:

Here, the user can establish the oracle connection by entering their username and password in the `channel <- gapindex::oracle_connect()` function. Never save usernames or passwords in scripts that may be intentionally or unintentionally shared with others. If no username and password is entered in the function, pop-ups will appear on the screen asking for the username and password.

After you connect to VPN, you'll be able to log into Oracle.

```
library(RORDBC)
channel <- gapindex::get_connected()
```

Data SQL Query Examples:

Data SQL Query Examples:

```
library(gapindex)
library(RODBC)
library(flextable)
library(ggplot2)
library(magrittr)
library(dplyr)
```

8.0.1. Ex. Select all data from tables

You can download all of the tables locally using a variation of the code below. Once connected, pull and save the tables of interest into the R environment.

```
locations <- c(
  "GAP_PRODUCTS.AKFIN_AGEComp",
  "GAP_PRODUCTS.AKFIN_AREA",
  "GAP_PRODUCTS.AKFIN_BIOMASS",
  "GAP_PRODUCTS.AKFIN_CATCH",
  "GAP_PRODUCTS.AKFIN_CPUE",
  "GAP_PRODUCTS.AKFIN_CRUISE",
  "GAP_PRODUCTS.AKFIN_HAUL",
  "GAP_PRODUCTS.AKFIN_LENGTH",
  "GAP_PRODUCTS.AKFIN_METADATA_COLUMN",
  "GAP_PRODUCTS.AKFIN_SIZEComp",
  "GAP_PRODUCTS.AKFIN_SPECIMEN",
  "GAP_PRODUCTS.AKFIN_STRATUM_GROUPS",
  "GAP_PRODUCTS.AKFIN_SURVEY_DESIGN",
  "GAP_PRODUCTS.AKFIN_TAXONOMIC_CLASSIFICATION"
)

for (i in 1:length(locations)) {
  print(locations[i])
  a <- RODBC::sqlQuery(channel, paste0("SELECT * FROM ", locations[i]))
  write.csv(x = a, file = here::here("data", paste0(locations[i], ".csv")))
}
```

```
library(odbc)
library(RODBC)
library(dbplyr)
```

Data SQL Query Examples:

```
my_spp_codes <- c(
  30010, # Sebastolobus sp.      thornyhead unid.
  30020, # Sebastolobus alascanus shortspine thornyhead
  30025, # Sebastolobus macrochir broadfin thornyhead
  30330, # Sebastes melanops    black rockfish
  30430, # Sebastes proriger    redstripe rockfish
  30470, # Sebastes ruberrimus  yelloweye rockfish
  30475, # Sebastes babcocki    redbanded rockfish
  30535, # Sebastes variegatus  harlequin rockfish
  30560, # Sebastes zacentrus   sharpchin rockfish
  30600, # Sebastes reedi       yellowmouth rockfish
  30030, # Sebastolobus altivelis longspine thornyhead
  30040, # Sebastes sp.         rockfish unid.
  30100, # Sebastes brevispinis silvergray rockfish
  30150, # NA                   dusky and dark rockfishes unid.
  30152, # Sebastes variabilis  dusky rockfish
  30170, # Sebastes crameri     darkblotched rockfish
  30270) # Sebastes helvomaculatus rosethorn rockfish

a <- dplyr::tbl(channel, dplyr::sql('gap_products.akfin_biomass')) |>
  dplyr::rename_all(tolower) |>
  dplyr::select(survey_definition_id, area_id, species_code, year, biomass_mt, biomass_var)
  dplyr::filter(species_code %in% my_spp_codes &
    area_id %in% 99904 &
    year >= 1991) |>
  dplyr::collect()

flextable::flextable(head(a)) |>
  flextable::fit_to_width(max_width = 6) |>
  flextable::theme_zebra()
```

8.0.2. Ex. CPUE for all EBS and NBS stations with associated haul, cruise, and species information.

```
a <- RODB::sqlQuery(channel = channel, # NOT RACEBASE.HAUL
  query = paste0(
    "
-- Select columns for output data
```


Data SQL Query Examples:

```
SELECT
cr.CRUISEJOIN,
cr.CRUISE,
cr.YEAR,
cr.SURVEY_DEFINITION_ID,
cr.SURVEY_NAME,
cr.VESSEL_ID,
cr.VESSEL_NAME,
cp.HAULJOIN,
cp.SPECIES_CODE,
tt.SPECIES_NAME,
tt.COMMON_NAME,
cp.WEIGHT_KG,
cp.COUNT,
cp.AREA_SWEPT_KM2,
cp.CPUE_KGKM2,
cp.CPUE_NOKM2,
hh.HAUL,
hh.STATION

-- Identify what tables to pull data from
FROM GAP_PRODUCTS.AKFIN_HAUL hh
LEFT JOIN GAP_PRODUCTS.AKFIN_CRUISE cr
ON hh.CRUISEJOIN = cr.CRUISEJOIN
LEFT JOIN GAP_PRODUCTS.AKFIN_CPUE cp
ON hh.HAULJOIN = cp.HAULJOIN
LEFT JOIN GAP_PRODUCTS.TAXONOMIC_CLASSIFICATION tt
ON cp.SPECIES_CODE = tt.SPECIES_CODE

-- Filter for EBS and NBS observations
WHERE SURVEY_DEFINITION_ID IN (143, 98) -- 143 NBS, 98 EBS
AND tt.SURVEY_SPECIES = 1

-- Only return the first 3 rows because otherwise this would be a huge table!
FETCH FIRST 3 ROWS ONLY;"))

flextable::flextable(head(a[,2:8])) |>
  flextable::fit_to_width(max_width = 6) |>
  flextable::theme_zebra()
```

Data SQL Query Examples:

Table 8.1.: CPUE for all EBS and NBS stations with associated haul, cruise, and species information.

CRUISE	YEAR	SUR- VEY_- DEFINI- TION_- ID	SUR- VEY_- NAME	VES- SEL_ID	VES- SEL_- NAME	HAULJOIN
199,801	1,998	98	Eastern Bering Sea Crab/Grou fish Bottom Trawl Survey	88	ARC- TURUS	937,305
199,801	1,998	98	Eastern Bering Sea Crab/Ground- fish Bottom Trawl Survey	88	ARC- TURUS	937,305
199,801	1,998	98	Eastern Bering Sea Crab/Grou fish Bottom Trawl Survey	88	ARC- TURUS	937,305

8.0.3. Ex. CPUE for all stations contained in the INPFC Shumagin region (AREA_ID = 919) for Pacific cod.

```
dat <- RODB::sqlQuery(channel = channel,
                      query =
                        "
-- Select columns for output data
SELECT
HAULJOIN,
SPECIES_CODE,
STRATUM,
LATITUDE_DD_START,
LONGITUDE_DD_START,
CPUE_KGKM2,
GEAR_TEMPERATURE_C

-- Identify what tables to pull data from
FROM GAP_PRODUCTS.AKFIN_CPUE cpue
LEFT JOIN GAP_PRODUCTS.AKFIN_HAUL haul
USING (HAULJOIN)

-- Filter for P. Cod observations
WHERE SPECIES_CODE IN (21720)

-- Select all stratum within the area_id 919 (INPFC Shumagin region)
AND haul.STRATUM IN
(
SELECT
STRATUM
FROM GAP_PRODUCTS.AKFIN_STRATUM_GROUPS
WHERE AREA_ID = 919
);")

dat <- dat |>
  dplyr::select(HAULJOIN, STRATUM, SPECIES_CODE, LATITUDE_DD_START, LONGITUDE_DD_START, CPU
  dplyr::mutate(SPECIES_CODE = as.character(SPECIES_CODE),
                STRATUM = as.character(STRATUM)) |>
  dplyr::arrange(SPECIES_CODE)
```

Data SQL Query Examples:

```
flextable::flextable(head(dat)) |>
  flextable::fit_to_width(max_width = 6) |>
  flextable::theme_zebra()
```

Table 8.2.: CPUE for all stations contained in the Shumagin region (AREA_ID = 919).

HAULJOI	STRA-TUM	SPECIES CODE	LATI-TUDE_-DD_-START	LONGI-TUDE_-DD_-START	CPUE_-KGKM2	GEAR_-TEM-PERA-TURE_-C
-12,880	210	21720	52.55793	169.7829	6,863.367	
-12,881	10	21720	52.63840	169.7815	1,536.8594	4.9
-12,882	111	21720	52.67131	169.4279	10,044.84	4.7
-12,883	10	21720	53.24099	168.0725	1,937.7294	5.2
-12,884	10	21720	53.16771	167.9810	830.2039	5.1
-12,885	111	21720	53.06838	167.6713	2,891.8092	4.9

8.0.4. Ex. EBS Pacific Ocean perch CPUE and akgfmaps map

Pacific Ocean perch catch-per-unit-effort estimates for EBS in 2021 from GAP_PRODUCTS.AKFIN_-CPUE and map constructed using akgfmaps. Here, we'll use AKFIN HAUL and CRUISES data also included in this repo, for convenience, though they are very similar to their RACEBASE analogs.

```
dat <- RODB::sqlQuery(channel = channel,
  query =
    "
-- Select columns for output data
```

Data SQL Query Examples:

```
SELECT
(cp.CPUE_KGKM2/100) CPUE_KGHA, -- akghmaps is expecting hectares, but can take any units
hh.LATITUDE_DD_START LATITUDE,
hh.LONGITUDE_DD_START LONGITUDE

-- Use HAUL data to obtain LATITUDE & LONGITUDE and connect to cruisejoin
FROM GAP_PRODUCTS.AKFIN_CPUE cp
LEFT JOIN GAP_PRODUCTS.AKFIN_HAUL hh
ON cp.HAULJOIN = hh.HAULJOIN

-- Use CRUISES data to obtain YEAR and SURVEY_DEFINITION_ID
LEFT JOIN GAP_PRODUCTS.AKFIN_CRUISE cc
ON hh.CRUISEJOIN = cc.CRUISEJOIN

-- Filter data
WHERE cp.SPECIES_CODE = 30060
AND cc.SURVEY_DEFINITION_ID = 98
AND cc.YEAR = 2021;")
```

```
dat |>
  dplyr::arrange(desc(CPUE_KGHA)) |>
  head() |>
  flextable::flextable() |>
  flextable::fit_to_width(max_width = 6) |>
  flextable::theme_zebra()
```

Table 8.3.: EBS Pacific Ocean perch CPUE and akghmaps map.

CPUE_ KGHA	LATI- TUDE	LONGI- TUDE
10.176896	57.64871	- 173.3735
6.2734470	56.36952	- 169.4604
3.0252034	56.66253	- 171.9549

Data SQL Query Examples:

CPUE_ KGHA	LATI- TUDE	LONGI- TUDE
1.8214628	57.98912	173.4816
0.5535672	55.65865	168.1804
0.2813533	57.32545	173.3217

```
# pak::pak("afsc-gap-products/akgfmmaps")
library(akgfmmaps)

figure <- akgfmmaps::make_idw_map(
  x = dat, # Pass data as a data frame
  region = "bs.south", # Predefined EBS area
  set.breaks = "jenks", # Gets Jenks breaks from classint::classIntervals()
  in.crs = "+proj=longlat", # Set input coordinate reference system
  out.crs = "EPSG:3338", # Set output coordinate reference system
  grid.cell = c(20000, 20000), # 20x20km grid
  key.title = "Pacific Ocean perch") # Include in the legend title
```

```
[inverse distance weighted interpolation]
[inverse distance weighted interpolation]
```

```
figure$plot
```

Data SQL Query Examples:

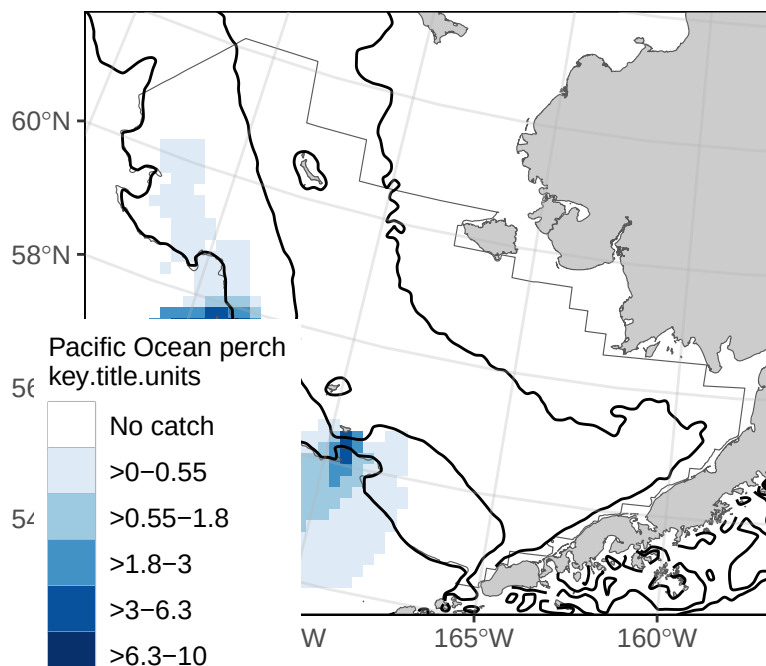


Figure 8.1.: EBS Pacific Ocean perch CPUE and akgfmaps map.

8.0.5. Ex. GOA Pacific Ocean perch biomass and abundance

Biomass and abundance for Pacific Ocean perch from 1990 – 2023 for the western/central/eastern GOA management areas as well as for the entire region.

```
dat <- RODB::sqlQuery(channel = channel,
                      query =
                        "
-- Manipulate data to join to
WITH FILTERED_STRATA AS (
SELECT AREA_ID, DESCRIPTION FROM GAP_PRODUCTS.AKFIN_AREA
WHERE AREA_TYPE in ('REGULATORY AREA', 'REGION')
AND SURVEY_DEFINITION_ID = 47
-- Use the AREA records associated with the GOA stratification prior to 2025
AND DESIGN_YEAR = 1984)

-- Select columns for output data
SELECT
```

Data SQL Query Examples:

```
BIOMASS_MT,  
POPULATION_COUNT,  
YEAR,  
DESCRIPTION
```

```
-- Identify what tables to pull data from  
FROM GAP_PRODUCTS.AKFIN_BIOMASS BIOMASS  
JOIN FILTERED_STRATA STRATA  
ON STRATA.AREA_ID = BIOMASS.AREA_ID
```

```
-- Filter data results  
WHERE BIOMASS.SPECIES_CODE = 30060  
AND BIOMASS.YEAR BETWEEN 1990 AND 2023")
```

```
dat0 <- dat |>  
  janitor::clean_names() |>  
  dplyr::select(biomass_mt, population_count, year, area = description) |>  
  pivot_longer(cols = c("biomass_mt", "population_count"),  
               names_to = "var",  
               values_to = "val") |>  
  dplyr::mutate(  
    val = ifelse(var == "biomass_mt", val/1e6, val/1e9),  
    var = ifelse(var == "biomass_mt", "Biomass (Mmt)", "Population (B)"),  
    area = gsub(x = area, pattern = " - ", replacement = "\n"),  
    area = gsub(x = area, pattern = ": ", replacement = "\n"),  
    type = sapply(X = strsplit(x = area, split = "\n", fixed = TRUE), `[[`, 2)) |>  
  dplyr::arrange(type) |>  
  dplyr::mutate(  
    area = factor(area, levels = unique(area), labels = unique(area), ordered = TRUE))  
  
flextable::flextable(head(dat)) |>  
  flextable::fit_to_width(max_width = 6) |>  
  flextable::theme_zebra() |>  
  flextable::colformat_num(j = "YEAR", big.mark = "")
```


Data SQL Query Examples:

Table 8.4.: GOA Pacific Ocean perch biomass and abundance.

BIOMASS MT	POPULATION_ COUNT	YEAR	DE- SCRIPTION
31,073.15	60,087,71	1990	CEN- TRAL GOA - INPFC
100,321.48	174,708,361	1990	EAST- ERN GOA - INPFC
24,435.56	79,343,91	1990	WEST- ERN GOA - INPFC
155,830.19	314,139,991	1990	GOA Region: All Strata
256,345.0	454,133,0	1993	CEN- TRAL GOA - INPFC
147,912.16	230,314,654	1993	EAST- ERN GOA - INPFC

```
# install.packages("scales")
library(scales)
figure <- ggplot2::ggplot(
  dat = dat0,
  mapping = aes(x = year, y = val, color = type)) +
  ggplot2::geom_point(size = 3) +
```

Data SQL Query Examples:

```
ggplot2::facet_grid(cols = vars(area), rows = vars(var), scales = "free_y") +
ggplot2::scale_x_continuous(name = "Year", n.breaks = 3) +
ggplot2::scale_y_continuous(name = "Estimate", labels = comma) +
ggplot2::labs(title = 'GOA Pacific Ocean perch biomass and abundance 1990 - 2023') +
ggplot2::guides(color=guide_legend(title = "Region Type"))+
ggplot2::scale_color_grey() +
ggplot2::theme_bw() +
ggplot2::theme(legend.direction = "horizontal",
               legend.position = "bottom")
```

figure

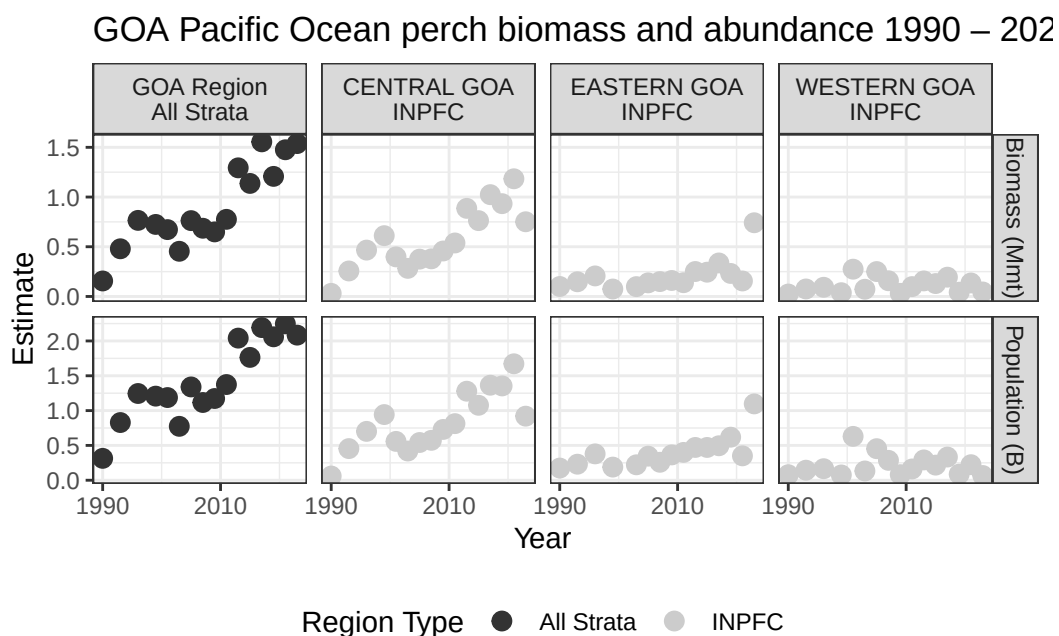


Figure 8.2.: GOA Pacific Ocean perch biomass and abundance.

8.0.6. Ex. AI rock sole size compositions and ridge plot

Northern and Southern rock sole size composition data from 1991 – 2022 for the Aleutian Islands, with Ridge plot from ggrridges.

Data SQL Query Examples:

```
dat <- RODB::sqlQuery(channel = channel,
                      query = "
SELECT
YEAR,
LENGTH_MM / 10 AS LENGTH_CM,
SUM(POPULATION_COUNT) AS POPULATION_COUNT

-- Identify what tables to pull data from
FROM GAP_PRODUCTS.AKFIN_SIZECOMP

-- 99904 is the AREA_ID that codes for the whole AI survey region
WHERE AREA_ID = 99904
-- including northern rock sole, southern rock sole, and rock sole unid.
AND SPECIES_CODE IN (10260, 10261, 10262)
-- remove the -9 LENGTH_MM code
AND LENGTH_MM > 0
-- sum over species_codes and sexes
GROUP BY (YEAR, LENGTH_MM)")
```

```
dat0 <- dat |>
  janitor::clean_names() |>
  head() |>
  flextable::flextable() |>
  flextable::fit_to_width(max_width = 6) |>
  flextable::theme_zebra() |>
  flextable::colformat_num(j = "year", big.mark = "")
dat0
```

Table 8.5.: AI Rock sole size compositions and ridge plot.

year	length_ cm	popula- tion_ count
1991	23	4,625,236
1991	38	2,254,964
1991	42	820,614
1991	52	11,225

Data SQL Query Examples:

year	length_ cm	popula- tion_ count
1994	16	741,246
1994	26	9,762,322

```
# install.packages("ggribes")
library(ggribes)
figure <- ggplot(dat,
  mapping = aes(x = LENGTH_CM,
    y = YEAR,
    height = POPULATION_COUNT,
    group = YEAR)) +
  ggribes::geom_density_ridges(stat = "identity", scale = 1) +
  ggplot2::ylab(label = "Year") +
  ggplot2::scale_x_continuous(name = "Length (cm)") +
  ggplot2::labs(title = paste0('Aleutian Islands Rock sole Size Compositions'),
    subtitle = paste0(min(dat$YEAR), ' - ', max(dat$YEAR))) +
  ggplot2::theme_bw()

figure
```

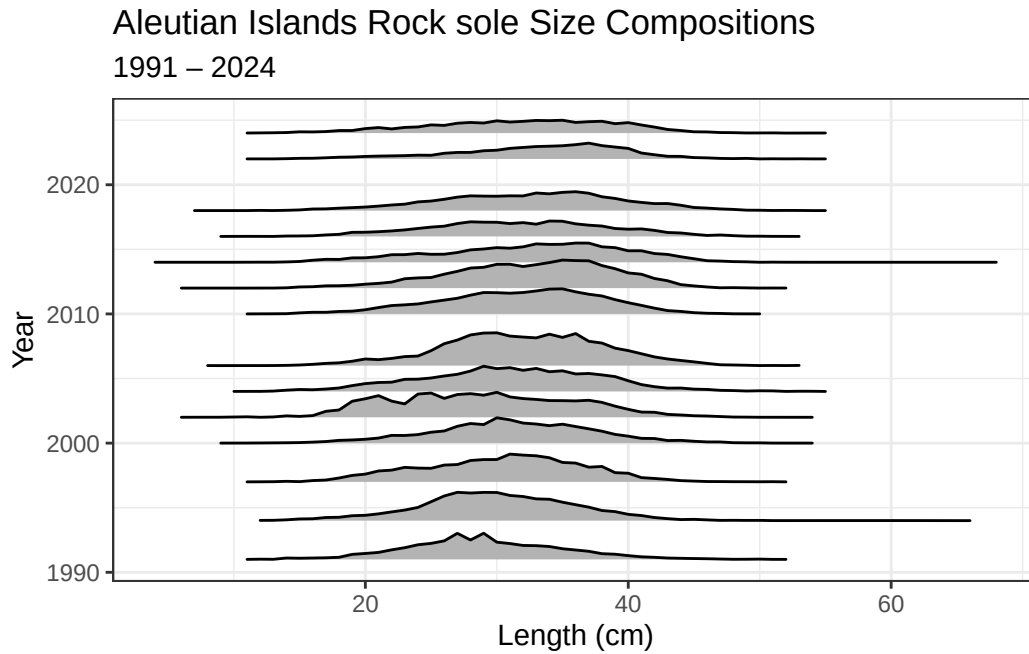


Figure 8.3.: AI Rock sole size compositions and ridge plot.

8.0.7. Ex. 2023 EBS Walleye Pollock Age Compositions and Age Pyramid

Walleye pollock age composition for the EBS standard + NW Area from 2023, with age pyramid plot.

```
dat <- RODB::sqlQuery(channel = channel,  
                      query = "  
-- Manipulate data to join to  
WITH FILTERED_STRATA AS (  
SELECT  
AREA_ID,  
DESCRIPTION  
FROM GAP_PRODUCTS.AKFIN_AREA  
-- Filter for EBS Standard + NW Area  
WHERE AREA_ID = 99900)  
  
-- Select columns for output data  
SELECT
```

Data SQL Query Examples:

```
AGECOMP.AGE,  
AGECOMP.POPULATION_COUNT,  
AGECOMP.SEX  
  
-- Identify what tables to pull data from  
FROM GAP_PRODUCTS.AKFIN_AGECOMP AGECOMP  
JOIN FILTERED_STRATA STRATA  
ON STRATA.AREA_ID = AGECOMP.AREA_ID  
  
-- Filter data results  
WHERE SPECIES_CODE = 21740  
AND YEAR = 2023  
AND AGE >= 0")
```

```
dat0 <- dat |>  
  janitor::clean_names() |>  
  dplyr::filter(sex %in% c(1,2)) |>  
  dplyr::mutate(  
    sex = ifelse(sex == 1, "M", "F"),  
    population_count = # change male population to negative  
      ifelse(sex=="M", population_count*(-1), population_count*1)/1e9)  
  
flextable::flextable(head(dat)) |>  
  flextable::fit_to_width(max_width = 6) |>  
  flextable::theme_zebra()
```

Table 8.6.: EBS Walleye Pollock Age Compositions and Age Pyramid.

AGE	POPULATION_COUNT	SEX
1	22,060,17	1
2	123,165,369	1
3	136,542,6	1
4	252,538,747	1

Data SQL Query Examples:

AGE	POPULATION_COUNT	SEX
5	964,790,9	1
6	242,135,720	1

```
figure <- ggplot2::ggplot(  
  data = dat0,  
  mapping =  
    aes(x = age,  
        y = population_count,  
        fill = sex)) +  
  ggplot2::scale_fill_grey() +  
  ggplot2::geom_bar(stat = "identity") +  
  ggplot2::coord_flip() +  
  ggplot2::scale_x_continuous(name = "Age") +  
  ggplot2::scale_y_continuous(name = "Population (billions)", labels = abs) +  
  ggplot2::ggtitle(label = "2023 EBS (Standard Area + NW) walleye pollock Age Composition")  
  ggplot2::guides(fill = guide_legend(title = "Sex"))+  
  ggplot2::theme_bw()
```

figure

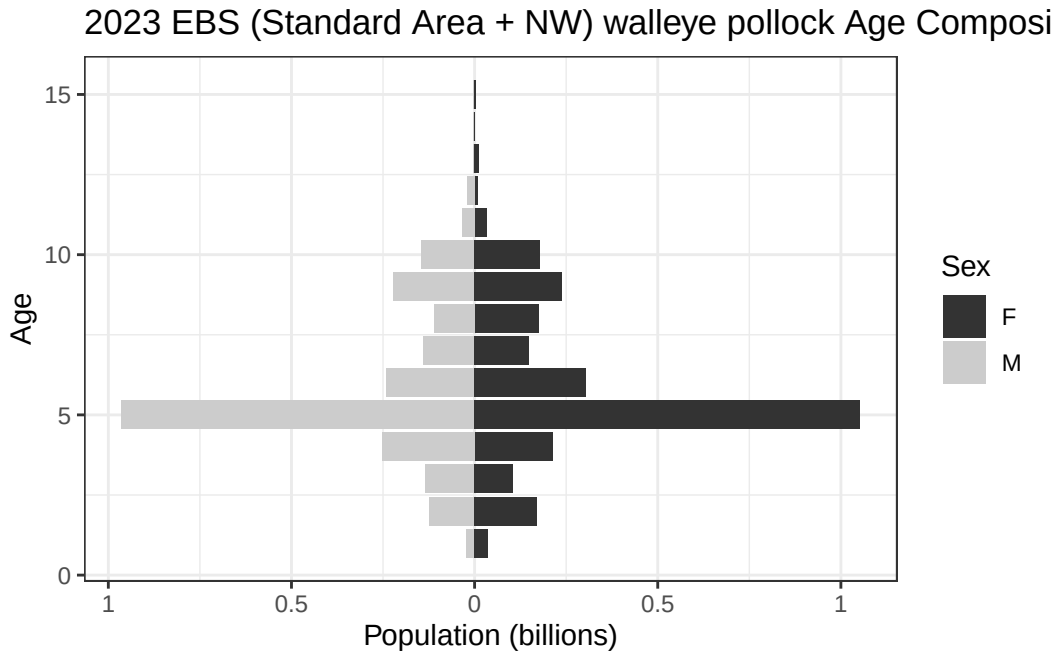


Figure 8.4.: 2023 EBS Walleye Pollock Age Compositions and Age Pyramid.

8.0.8. Ex. NBS Pacific cod biomass and abundance

Pacific cod biomass and abundance data for the NBS by stratum.

```
dat <- RODB::sqlQuery(channel = channel,
                      query =
                        "
SELECT YEAR, AREA_ID AS STRATUM, AREA_NAME, BIOMASS_MT, POPULATION_COUNT
FROM GAP_PRODUCTS.AKFIN_BIOMASS

JOIN ( -- join with area table
SELECT AREA_ID, AREA_NAME
FROM GAP_PRODUCTS.AKFIN_AREA
WHERE AREA_TYPE = 'STRATUM'
AND SURVEY_DEFINITION_ID = 143
AND DESIGN_YEAR = 2022)
USING (AREA_ID)
```


Data SQL Query Examples:

```
-- Filter data results to NBS Pacific cod
WHERE SURVEY_DEFINITION_ID IN 143
AND SPECIES_CODE = 21720
ORDER BY YEAR, STRATUM")
```

```
dat0 <- dat |>
  janitor::clean_names() |>
  dplyr::select(year, area_name, biomass_mt, population_count) |>
  pivot_longer(cols = c("biomass_mt", "population_count"),
               names_to = "var",
               values_to = "val") |>
  dplyr::mutate(
    val = ifelse(var == "biomass_mt", val/1e6, val/1e9),
    var = ifelse(var == "biomass_mt", "Biomass (Mmt)", "Population (B)"),
    area = factor(area_name, levels = unique(area_name), labels = unique(area_name), ordered = TRUE)
  )
flextable::flextable(dat) |>
  flextable::fit_to_width(max_width = 6) |>
  flextable::theme_zebra() |>
  flextable::colformat_num(j = "YEAR", big.mark = "")
```

Table 8.7.: NBS Pacific cod biomass and abundance.

YEAR	STRATUM	AREA_NAME	BIOMASS MT	POPULATION_COUNT
2010	70	Inner Domain	7,462.558	4,724,153
2010	71	Inner Domain	20,983.375	3,928,600
2010	81	Middle Domain	680.4357	250,837
2017	70	Inner Domain	132,490.15	65,187,245
2017	71	Inner Domain	147,971.4	65,078,489

Data SQL Query Examples:

YEAR	STRATUM	AREA_NAME	BIOMASS MT	POPULATION_COUNT
2017	81	Middle Domain	7,089.8740	4,191,118
2019	70	Inner Domain	107,096.7	102,734,142
2019	71	Inner Domain	194,846.723	30,495,085
2019	81	Middle Domain	63,061.27	25,926,805
2021	70	Inner Domain	95,849.983	58,767,498
2021	71	Inner Domain	53,814.63	17,941,471
2021	81	Middle Domain	77,917.108	42,991,939
2022	70	Inner Domain	96,500.69	60,433,135
2022	71	Inner Domain	26,747.074	10,447,602
2022	81	Middle Domain	30,487.27	15,157,597
2023	70	Inner Domain	76,708.432	79,605,860
2023	71	Inner Domain	19,130.00	8,459,469
2023	81	Middle Domain	12,507.856	4,128,368
2025	70	Inner Domain	48,587.67	31,324,540

Data SQL Query Examples:

YEAR	STRA-TUM	AREA_-NAME	BIOMASS MT	POPULATION-COUNT
2025	71	Inner Domain	11,129.2287	4,456,827
2025	81	Middle Domain	9,665.662	7,262,957

```
figure <- ggplot2::ggplot(
  dat = dat0,
  mapping = aes(y = val, x = year, fill = area)) +
  ggplot2::geom_bar(position="stack", stat="identity") +
  ggplot2::facet_grid(rows = vars(var), scales = "free_y") +
  ggplot2::scale_y_continuous(name = "Estimate", labels = comma) +
  ggplot2::scale_x_continuous(name = "Year", breaks = unique(dat0$year)) +
  ggplot2::labs(title = 'NBS Pacific cod biomass and abundance by stratum') +
  ggplot2::guides(fill=guide_legend(title = "Domain Type "))+
  ggplot2::scale_fill_grey() +
  ggplot2::theme_bw() +
  ggplot2::theme(legend.direction = "horizontal",
    legend.position = "bottom")

figure
```

Data SQL Query Examples:

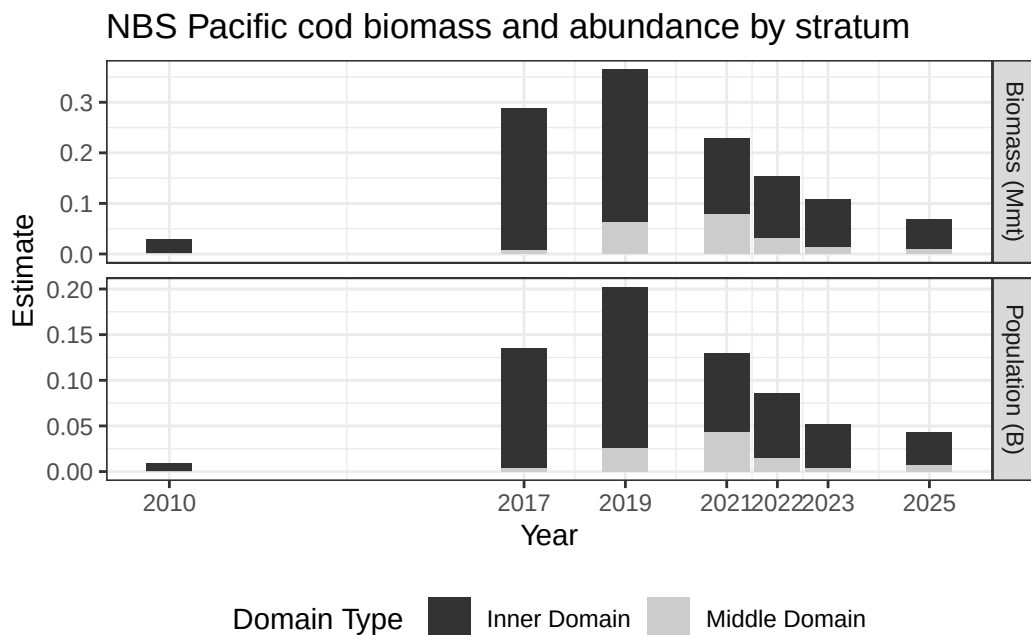


Figure 8.5.: NBS Pacific cod biomass and abundance.

8.0.9. Ex. GOA Pacific Ocean perch biomass and line plot

Pacific Ocean perch biomass totals for GOA between 1984-2021 from `GAP_PRODUCTS.AKFIN_BIOMASS`

```
dat <- RODB::sqlQuery(channel = channel,
                      query = "
-- Select columns for output data
SELECT
SURVEY_DEFINITION_ID,
BIOMASS_MT / 1000000 AS BIOMASS_MMT,
(BIOMASS_MT - 2 * SQRT(BIOMASS_VAR)) / 1000000 AS BIOMASS_CI_DW,
(BIOMASS_MT + 2 * SQRT(BIOMASS_VAR)) / 1000000 AS BIOMASS_CI_UP,
YEAR

-- Identify what tables to pull data from
FROM GAP_PRODUCTS.AKFIN_BIOMASS
```

Data SQL Query Examples:

```
-- Filter data results
WHERE SPECIES_CODE = 30060
AND SURVEY_DEFINITION_ID = 47
AND AREA_ID = 99903
AND YEAR BETWEEN 1990 AND 2023" ) |>
  janitor::clean_names()
```

```
flextable::flextable(head(dat)) |>
  flextable::fit_to_width(max_width = 6) |>
  flextable::theme_zebra() |>
  flextable::colformat_num(j = "year", big.mark = "")
```

Table 8.8.: GOA Pacific Ocean perch biomass and line plot.

sur- vey_ defini- tion_id	biomass_ mmt	biomass_ ci_dw	biomass_ ci_up	year
47	0.1558302	0.0618137	0.2498467	1990
47	0.4796687	0.2659632	0.6933741	1993
47	0.7651705	0.3604455	1.1698950	1996
47	0.7243655	0.0523802	1.5011113	1999
47	0.6723673	0.2290337	1.1157008	2001
47	0.4543899	0.3107735	0.5980063	2003

```
a_mean <- dat |>
  dplyr::group_by(survey_definition_id) |>
  dplyr::summarise(biomass_mmt = mean(biomass_mmt, na.rm = TRUE),
                   minyr = min(year, na.rm = TRUE),
                   maxyr = max(year, na.rm = TRUE))

figure <-
  ggplot(data = dat,
         mapping = aes(x = year,
                       y = biomass_mmt)) +
```

Data SQL Query Examples:

```
ggplot2::geom_point(size = 2.5, color = "grey40") +
ggplot2::scale_x_continuous(
  name = "Year",
  labels = scales::label_number(
    accuracy = 1,
    big.mark = "")) +
ggplot2::scale_y_continuous(
  name = "Biomass (Mmt)",
  labels = comma) +
ggplot2::geom_segment(
  data = a_mean,
  mapping = aes(x = minyr,
                xend = maxyr,
                y = biomass_mmt,
                yend = biomass_mmt),
  linetype = "dashed",
  linewidth = 2) +
ggplot2::geom_errorbar(
  mapping = aes(ymin = biomass_ci_dw, ymax = biomass_ci_up),
  position = position_dodge(.9),
  alpha = 0.5, width=.2) +
ggplot2::ggtitle(
  label = "GOA Pacific Ocean Perch Biomass 1984-2021",
  subtitle = paste0("Mean = ",
                    formatC(x = a_mean$biomass_mmt,
                              digits = 2,
                              big.mark = ",",
                              format = "f"),
                    " Mmt")) +
ggplot2::theme_bw()
```

figure

Data SQL Query Examples:

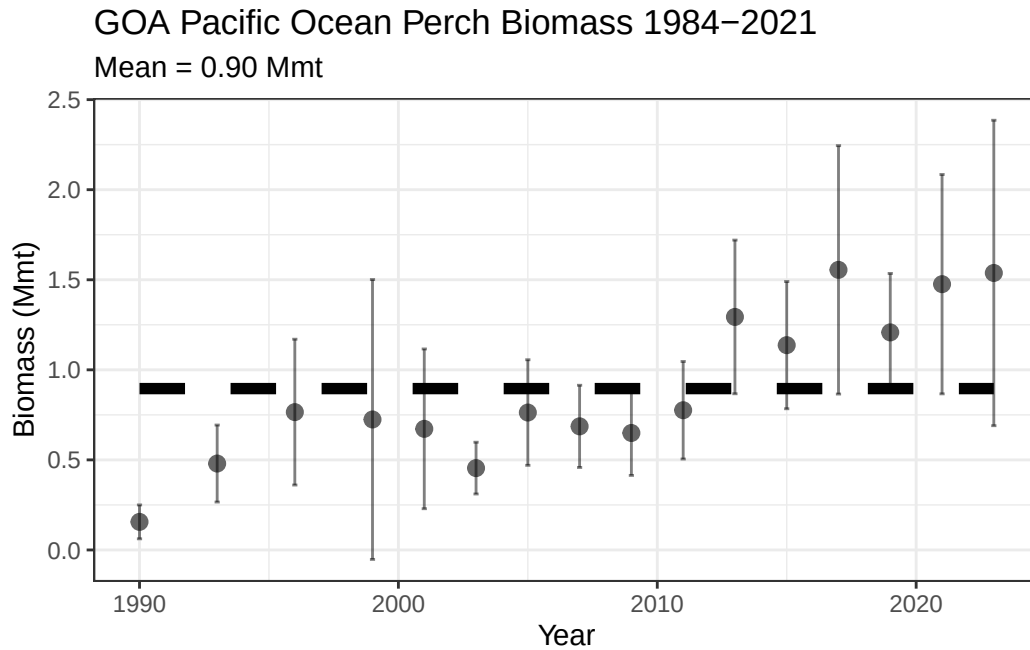


Figure 8.6.: GOA Pacific Ocean perch biomass and line plot.

8.0.10. Ex. 2022 AI Atka mackerel age specimen summary

8.0.10.1. All ages determined:

```
dat <- RODB::sqlQuery(channel = channel,  
                      query = "  
-- Select columns for output data  
SELECT SURVEY_DEFINITION_ID, YEAR, SPECIES_CODE, AGE  
  
-- Identify what tables to pull data from  
FROM GAP_PRODUCTS.AKFIN_SPECIMEN  
JOIN (SELECT HAULJOIN, CRUISEJOIN FROM GAP_PRODUCTS.AKFIN_HAUL)  
USING (HAULJOIN)  
JOIN (SELECT CRUISEJOIN, YEAR, SURVEY_DEFINITION_ID FROM GAP_PRODUCTS.AKFIN_CRUISE)  
USING (CRUISEJOIN)  
  
-- Filter data results
```

Data SQL Query Examples:

```
WHERE GAP_PRODUCTS.AKFIN_SPECIMEN.SPECIMEN_SAMPLE_TYPE = 1
AND SPECIES_CODE = 21921
AND YEAR = 2022
AND SURVEY_DEFINITION_ID = 52") |>
janitor::clean_names()
```

```
flextable::flextable(head(dat) |>
  dplyr::arrange(age)) |>
flextable::fit_to_width(max_width = 6) |>
flextable::theme_zebra() |>
flextable::colformat_num(j = c("year", "species_code"), big.mark = "")
```

Table 8.9.: 2022 AI Atka mackerel age specimen summary: all ages determined.

sur- vey_ defini- tion_id	year	species_ code	age
52	2022	21921	3
52	2022	21921	3
52	2022	21921	4
52	2022	21921	4
52	2022	21921	4
52	2022	21921	7

8.0.10.2. How many of each age was found:

```
dat <- RODB::sqlQuery(channel = channel,
  query = "
-- Select columns for output data
SELECT SURVEY_DEFINITION_ID, YEAR, SPECIES_CODE, AGE,
COUNT(AGE) AS COUNTAGE

-- Identify what tables to pull data from
```


Data SQL Query Examples:

```
FROM GAP_PRODUCTS.AKFIN_SPECIMEN
JOIN (SELECT HAULJOIN, CRUISEJOIN FROM GAP_PRODUCTS.AKFIN_HAUL)
USING (HAULJOIN)
JOIN (SELECT CRUISEJOIN, YEAR, SURVEY_DEFINITION_ID FROM GAP_PRODUCTS.AKFIN_CRUISE)
USING (CRUISEJOIN)

-- Filter data results
WHERE AGE >= 0
AND SPECIES_CODE = 21921
AND YEAR = 2022
AND SURVEY_DEFINITION_ID = 52
GROUP BY (YEAR, SURVEY_DEFINITION_ID, SPECIES_CODE, AGE)

ORDER BY AGE") |>
  janitor::clean_names()
```

```
flextable::flextable(dat) |>
  flextable::fit_to_width(max_width = 6) |>
  flextable::theme_zebra() |>
  flextable::colformat_num(j = c("year", "species_code"), big.mark = "")
```

Table 8.10.: Ex.: 2022 AI Atka mackerel age specimen summary: how many of each age were determined.

sur- vey_- defini- tion_id	year	species_- code	age	coun- tage
52	2022	21921	1	1
52	2022	21921	2	40
52	2022	21921	3	295
52	2022	21921	4	119
52	2022	21921	5	130
52	2022	21921	6	116
52	2022	21921	7	108

Data SQL Query Examples:

survey_definition_id	year	species_code	age	countage
52	2022	21921	8	61
52	2022	21921	9	88
52	2022	21921	10	73
52	2022	21921	11	20
52	2022	21921	12	9
52	2022	21921	13	1

8.0.10.3. How many otoliths were aged:

Using SQL

```
dat <- RODB::sqlQuery(channel = channel,
                      query = "
-- Select columns for output data
SELECT SURVEY_DEFINITION_ID, YEAR, SPECIES_CODE,
COUNT(AGE) AS COUNTAGE

-- Identify what tables to pull data from
FROM GAP_PRODUCTS.AKFIN_SPECIMEN
JOIN (SELECT HAULJOIN, CRUISEJOIN FROM GAP_PRODUCTS.AKFIN_HAUL)
USING (HAULJOIN)
JOIN (SELECT CRUISEJOIN, YEAR, SURVEY_DEFINITION_ID FROM GAP_PRODUCTS.AKFIN_CRUISE)
USING (CRUISEJOIN)

-- Filter data results
WHERE GAP_PRODUCTS.AKFIN_SPECIMEN.SPECIMEN_SAMPLE_TYPE = 1
AND SPECIES_CODE = 21921
AND YEAR = 2022
AND SURVEY_DEFINITION_ID = 52
GROUP BY (YEAR, SURVEY_DEFINITION_ID, SPECIES_CODE)") |>
  janitor::clean_names()
```

Data SQL Query Examples:

Using dbplyr:

```
library(odbc)
library(keyring)
library(dplyr)
library(dbplyr)

channel <- DBI::dbConnect(odbc::odbc(), "akfin", uid = keyring::key_list("akfin")$username,
                          pwd = keyring::key_get("akfin", keyring::key_list("akfin")$username))

dat <- dplyr::tbl(src = channel, dplyr::sql('gap_products.akfin_specimen')) |>
  dplyr::rename_all(tolower) |>
  dplyr::select(hauljoin, specimen = specimen_id, species_code, length = length_mm,
               weight = weight_g, age, sex, age_method = age_determination_method) |>
  dplyr::left_join(dplyr::tbl(akfin, dplyr::sql('gap_products.akfin_haul')) |>
    dplyr::rename_all(tolower) |>
    dplyr::select(cruisejoin, hauljoin, haul, date_collected = date_time_collected,
                  latitude = latitude_dd_start, longitude = longitude_dd_start) |>
    by = join_by(hauljoin)) |>
  dplyr::left_join(dplyr::tbl(akfin, dplyr::sql('gap_products.akfin_cruise')) |>
    dplyr::rename_all(tolower) |>
    dplyr::select(cruisejoin, year, vessel = vessel_id, survey_definition_id = survey_definition_id) |>
    by = join_by(cruisejoin)) |>
  dplyr::filter(year == YEAR &
                survey_definition_id == 52 &
                species_code %in% spp_codes &
                !is.na(age)) |>
  dplyr::collect()
```

Both scripts will produce this table:

```
flextable::flextable(head(dat)) |>
  flextable::fit_to_width(max_width = 6) |>
  flextable::theme_zebra() |>
  flextable::colformat_num(j = c("year", "species_code"), big.mark = "")
```

Data SQL Query Examples:

Table 8.11.: 2022 AI Atka mackerel age specimen summary: how many otoliths were aged. This query was created using SQL.

sur- vey_- defini- tion_id	year	species_- code	coun- tage
52	2022	21921	1,061

9. Access API data via R

AKFIN has developed web services (apis) to distribute GAP data. Like the GAP_PRODUCTS schema, these are under active development. These do not require VPN or an oracle connection but they are protected by Oracle authentication, please contact matt.callahan@noaa.gov for information on how to get an api token to use this option.

The url structure is "https://apex.psmfc.org/akfin/data_marts/gap_products/gap_[base table name]". For example "https://apex.psmfc.org/akfin/data_marts/gap_products/gap_biomass" is the base url to get data from the akfin_biomass table. Web services linked to large tables have mandatory parameters to reduce data download size. For example to get agecomp data for Bering Sea pollock in area_id 10 in 2022 you would use "https://apex.psmfc.org/akfin/data_marts/gap_products/gap_biomass?survey_definition_id=98&area_id=10&species_code=21740&start_year=2022&end_year=2022".

If you're using R to pull data through web services you might find the `akfingapdata` (pronounced **akfin-gap-data** not **ak-eff-ing-app-data**) R package helpful.

9.0.1. Load packages and helpful functions

9.1. Ex. Direct database query in R using the `akfingapdata` R package README:

Sign into akfin with token (need to request token from AKFIN)

```
akfingapdata::get_gap_catch()[,1:6] |>
  head() |>
  flextable::flextable() |>
  flextable::theme_zebra()
```

10. Using the FOSS platform

10.1. Select and filter

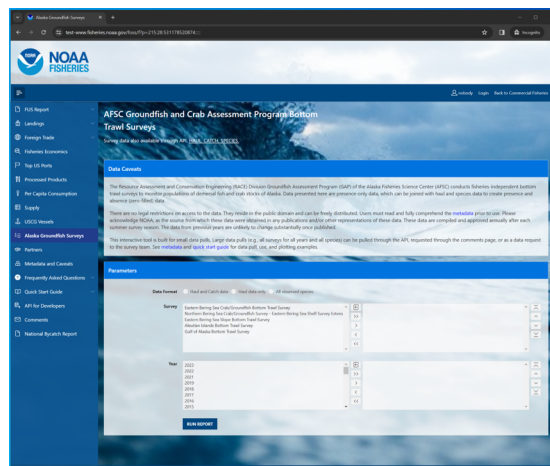


Figure 10.1.: AFSC Groundfish and Crab Assessment Program Bottom Trawl Survey data interface on the Fisheries One Stop Shop platform.

Select, filter, and download this and other NOAA Fisheries data from the Fisheries One Stop Shop (FOSS) platform. A user guide for the FOSS platform can be found [here](#). To begin a report, select the kind of data you need: Haul and catch data, Haul data only, All observed species.

In this example, we'll select for 2023 eastern Bering Sea Arctic cod data. Here, we used the Search Species box to search for species with the term "cod" in their common names and selected "Pacific cod" from that shortened list.

10. Using the FOSS platform

10.1.1. Catch and haul

Survey year	Survey ID	Survey name official	Survey name	Survey abbreviation	Cruise ID	Cruise code	Stratum ID	Station ID	Haul number	Vessel ID	Vessel name	Date and time	Start latitude (decimal degrees)	Start longitude (decimal degrees)	End latitude (decimal degrees)	End longitude (decimal degrees)	Depth
2023	98	Eastern Bering Sea Crab/Groundfish Bottom Trawl Survey	eastern Bering Sea Crab/Groundfish Bottom Trawl Survey	EBBS	202301	-759	42	A2019	195	162	ALASKA KNIGHT	28-JUN-2023 02:00PM	57.8451	-158.3702	57.82165	-160.36325	
2023	98	Eastern Bering Sea Crab/Groundfish Bottom Trawl Survey	eastern Bering Sea Crab/Groundfish Bottom Trawl Survey	EBBS	202301	-760	42	A2019	196	162	ALASKA KNIGHT	28-JUN-2023 02:00PM	57.8451	-158.3702	57.82165	-160.36325	

Figure 10.2.: Catch data on the AFSC Groundfish and Crab Assessment Program Bottom Trawl Survey data interface on the Fisheries One Stop Shop platform.

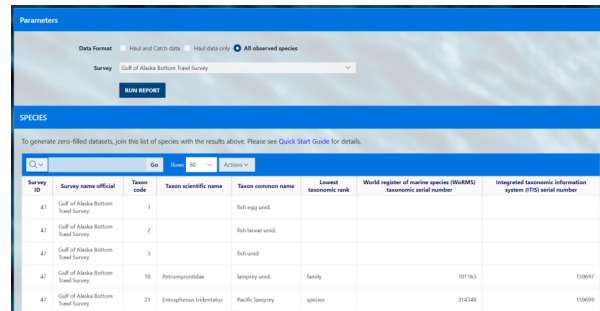
10.1.2. Haul

Survey year	Survey ID	Survey name official	Survey name	Survey abbreviation	Cruise ID	Cruise code	Stratum ID	Station ID	Haul number	Vessel ID	Vessel name	Date and time	Start latitude (decimal degrees)	Start longitude (decimal degrees)	End latitude (decimal degrees)	End longitude (decimal degrees)	Depth
2023	98	Eastern Bering Sea Crab/Groundfish Bottom Trawl Survey	eastern Bering Sea Crab/Groundfish Bottom Trawl Survey	EBBS	-759	202301	42	F-21	147	162	ALASKA KNIGHT	07-JUL-2023 12:48PM	56.67915	-170.14347	56.65574	-170.12189	
2023	98	Eastern Bering Sea Crab/Groundfish Bottom Trawl Survey	eastern Bering Sea Crab/Groundfish Bottom Trawl Survey	EBBS	-760	202301	42	J-19	106	134	NORTHWEST EXPLORER	25-JUN-2023 04:00PM	57.98219	-169.07093	58.00846	-169.07991	

Figure 10.3.: Haul data on the AFSC Groundfish and Crab Assessment Program Bottom Trawl Survey data interface on the Fisheries One Stop Shop platform.

10. Using the FOSS platform

10.1.3. Species



The screenshot shows the 'Parameters' section of the Fisheries One Stop Shop platform. The 'Data Format' is set to 'All observed species'. Below this, there is a 'Survey' dropdown menu set to 'Gulf of Alaska Bottom Trawl Survey' and a 'RUN REPORT' button. The 'SPECIES' section displays a table of species observed by survey.

Survey ID	Survey name official	Taxon code	Taxon scientific name	Taxon common name	Lowest taxonomic rank	World register of marine species (WoRMS) taxonomic serial number	Integrated taxonomic information system (ITIS) serial number
47	Gulf of Alaska Bottom Trawl Survey	1		fish-egg unit			
47	Gulf of Alaska Bottom Trawl Survey	2		fish-seaweed unit			
47	Gulf of Alaska Bottom Trawl Survey	3		fish-unit			
47	Gulf of Alaska Bottom Trawl Survey	10	Petromyzontidae	lamprey unit	family	101163	159607
47	Gulf of Alaska Bottom Trawl Survey	21	Esopterygionidae	Pacific lamprey	species	214340	159609

Figure 10.4.: All species observed by survey on the AFSC Groundfish and Crab Assessment Program Bottom Trawl Survey data interface on the Fisheries One Stop Shop platform.

10.2. Search options

The user must select a option in each of the three option boxes as they appear for catch, haul, and species:

- **Survey:** Each survey has different in design, time series, and history. More information on each survey and their designs can be found in our annual data reports.
- **Year:** Surveys are not conducted in all years, so only data from the years for which the survey was conducted will be returned.
- **Species:** Common name of all species ever encountered in the survey. Find more information about these species in our survey code books.

For a given box, select one or a few options from the options box (list on the left) to query. To select multiple options, hold down the CTRL key while clicking on the options of interest, or click and drag down the list. Once the options you wish to be included in your query are highlighted, click the right-pointing arrow (>) to move them into the "selection box" (list on the right). This can also be achieved by double clicking the option item of interest. If you accidentally select an option that you do not want to query, simply select the unwanted option from the selection box and click the left-pointing arrow (<).

If you wish to select all options from the options box and send them to the selection box, simply click the double right-pointing arrow (>>). If you want to unselect all options from the selection box, use the double left-pointing arrow (<<) or the reset icon.

10. Using the FOSS platform

To find a specific species or group more quickly you can use the **Search Species** option to quickly narrow the options. Search for parts of species common names in the **Search Species** box by entering a term and clicking the **search** button. The platform will return a shorter list in the **Species** options box of only species that contain a match to that search term.

Use the **Reset All Parameters** button to reset all parameters for entire form.

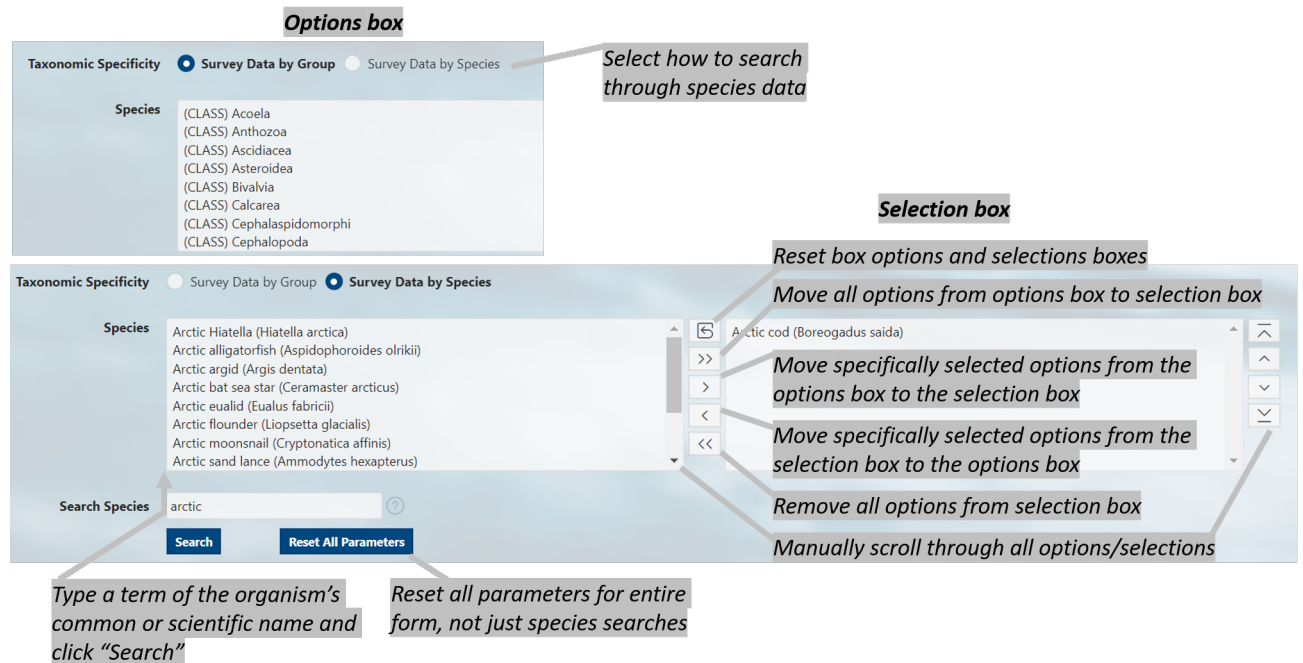


Figure 10.5.: Diagram of selection and search tools available on the FOSS platform.

10.3. Run report

Click the **RUN REPORT** button. Below the select and filter area, the results of your query will appear below the page in the format you selected. To change the format, make a different selection and run the report again. Further modifications to your results can be made by clicking on the **Actions** button above your data. Here you can download your data, select columns included in your results, and apply a variety of filters and mathematical tools.

10. Using the FOSS platform

The screenshot displays the FOSS platform interface with several annotations and overlays. At the top, a window titled "Select columns to display in your report" is open, showing a list of columns to be displayed in the report. Below this, a "Filter" window is visible, allowing users to filter data by column, operator, and expression. The main data table, titled "CATCH AND HAUL DATA", is shown with columns for Survey year, Survey ID, Survey name official, Survey name, Survey abbreviation, Cruise ID, Cruise code, Stratum ID, Station ID, and Haul number. A search bar is present above the table, and a "Rows Per Page" dropdown is set to 25. A blue sidebar on the right contains various action buttons: Sort, Control Break, Highlight, Compute, Aggregate, Chart, Group By, Pivot, Format, Flashback, and Reset. Annotations with arrows point to specific features: "Filter data displayed in report" points to the Filter window; "Search for elements of the data" points to the search bar; "Determine how many rows per page to display" points to the Rows Per Page dropdown; and "Apply mathematical aggregations to data" points to the Pivot button in the sidebar.

Filter data displayed in report

Select columns to display in your report

Search for elements of the data

Determine how many rows per page to display

Apply mathematical aggregations to data

Figure 10.6.: Example data returned from running the report.

10.4. API

APIs, or Application Programming Interfaces, allows users to pull data through a IDE, or integrated development environment, like RStudio or VS Code. Explore the API pages for each of the data pages (Haul and catch data, Haul data only, All observed species).

11. Use data

Learn how to pull and use this data through the

- API and R programming language
- API and python programming language using the `afscgap` python package
- Oracle and R programming language (AFSC scientists only)

12. Access via API and R

An application programming interface (API) is a way for two or more computer programs to communicate with each other. More information about how to amend API links can be found [here](#). Useful introductions to using APIs in R can be found [here](#).

There are three tables the user can pull from the API. Learn more about them on the FOSS data description page. Here, you can see them in their raw JSON format:

- haul: https://apps-st.fisheries.noaa.gov/ods/foss/afsc_groundfish_survey_haul/
- catch: https://apps-st.fisheries.noaa.gov/ods/foss/afsc_groundfish_survey_catch/
- species: https://apps-st.fisheries.noaa.gov/ods/foss/afsc_groundfish_survey_species/

Here are some examples of how to use the data with R:

12.1. Ex. Load all rows of the catch, haul, and species data tables

Note that without specifying, a basic query to the API will only return 25 entries.

12.1.1. Load haul data

```
# link to the API
api_link_haul <- 'https://apps-st.fisheries.noaa.gov/ods/foss/afsc_groundfish_survey_haul/'
```

12.1.1.1. Load first 25 rows of data

12. Access via API and R

```
res <- httr::GET(url = api_link_haul)
# res ## Test connection

## convert from JSON format
dat <- jsonlite::fromJSON(base::rawToChar(res$content))$items

# Find how many rows and columns are in the data pull
print(paste0("rows: ", nrow(dat), "; cols: ", ncol(dat)))
```

12.1.1.2. Load all data:

Since the maxim number of rows a user can pull is 10,000 rows in a query, the user needs to cycle through by offsetting to the next 10,000 rows (as is shown here).

```
dat <- data.frame()
for (i in seq(0, 500000, 10000)){
  ## find how many iterations it takes to cycle through the data
  print(i)
  ## query the API link
  res <- httr::GET(url = paste0(api_link_haul, "?offset=",i,"&limit=10000"))
  ## convert from JSON format
  data <- jsonlite::fromJSON(base::rawToChar(res$content))

  ## if there are no data, stop the loop
  if (is.null(nrow(data$items))) {
    break
  }

  ## bind sub-pull to dat data.frame
  dat <- dplyr::bind_rows(dat,
                          data$items |>
                          dplyr::select(-links)) # necessary for API accounting, but not
}
}
```

Explore the data contents:

```
# Find how many rows and columns are in the data pull
print(paste0("rows: ", nrow(dat), "; cols: ", ncol(dat)))
```

12. Access via API and R

```
# learn about the structure of the data
summary(dat)

# Print the first few lines of the data
dat |>
  head(3) |>
  flextable::flextable() |>
  flextable::colformat_num(
    j = c("year", "cruise", "cruisejoin"),
    big.mark = "" ) |>
  flextable::theme_zebra()

# save outputs for later comparison
dat_haul_api <- dat
```

12.1.2. Load catch data

```
# link to the API
api_link_catch <- 'https://apps-st.fisheries.noaa.gov/ods/foss/afsc_groundfish_survey_catch'
```

12.1.2.1. Load first 25 rows of data

```
res <- httr::GET(url = api_link_catch)
# res ## Test connection

## convert from JSON format
dat <- jsonlite::fromJSON(base::rawToChar(res$content))$items

# Find how many rows and columns are in the data pull
print(paste0("rows: ", nrow(dat), "; cols: ", ncol(dat)))
```

12.1.2.2. Load all data

Since the maxim number of rows a user can pull is 10,000 rows in a query, the user needs to cycle through by offsetting to the next 10,000 rows (as is shown here).

12. Access via API and R

```
dat <- data.frame()
# for (i in seq(0, 100000, 10000)){
for (i in seq(0, 1000000, 10000)){
  ## find how many iterations it takes to cycle through the data
  # print(i)
  ## query the API link
  res <- httr::GET(url = paste0(api_link_catch, "?offset=", i, "&limit=10000"))
  ## convert from JSON format
  data <- jsonlite::fromJSON(base::rawToChar(res$content))

  ## if there are no data, stop the loop
  if (is.null(nrow(data$items))) {
    break
  }

  ## bind sub-pull to dat data.frame
  dat <- dplyr::bind_rows(dat,
                           data$items |>
                           dplyr::select(-links)) # necessary for API accounting, but not
}
```

Explore the data contents:

```
# Find how many rows and columns are in the data pull
print(paste0("rows: ", nrow(dat), "; cols: ", ncol(dat)))

# learn about the structure of the data
summary(dat)

# Print the first few lines of the data
dat |>
  head(3) |>
  flextable::flextable() |>
  flextable::colformat_num(
    j = c("species_code"),
    big.mark = "" ) |>
  flextable::theme_zebra()

# save outputs for later comparison
dat_catch_api <- dat
```

12. Access via API and R

12.1.3. Load species data

Since there are less than 10,000 rows of species data (and the maximum number of rows a user can pull from this API is 10,000 rows in a query), we can simply call `?offset=0&limit=10000` in our query call.

```
# link to the API
api_link_species <- 'https://apps-st.fisheries.noaa.gov/ods/foss/afsc_groundfish_survey_spe

res <- httr::GET(url = paste0(api_link_species, "?offset=0&limit=10000"))

## convert from JSON format
data <- jsonlite::fromJSON(base::rawToChar(res$content))
dat <- data$items |>
  dplyr::select(-links) # necessary for API accounting, but not part of the dataset
```

Explore the data contents:

```
# Find how many rows and columns are in the data pull
print(paste0("rows: ", nrow(dat), "; cols: ", ncol(dat)))

# learn about the structure of the data
summary(dat)

# Print the first few lines of the data
dat |>
  head(3) |>
  flextable::flextable() |>
  flextable::colformat_num(
    j = c("species_code", "worms", "itis"), #
    big.mark = "" |>
  flextable::theme_zebra()

# save outputs for later comparison
dat_species_api <- dat
```


12.2. Ex. Create zero-filled data using data loaded in last example

It is important to create and have access to zero-fill (presence and absence) so you can do simple analyses and plot data.

First prepare a table with all combinations of what species should be listed for what hauls/surveys. For zero-filled data, all species caught in a survey need to have zero or non-zero row entries for a haul

```
comb <- dplyr::full_join(
  # find all species that have been caught, by survey
  x = dplyr::left_join(dat_catch_api, dat_haul_api, by = "hauljoin") |>
    dplyr::select(survey_definition_id, species_code) |>
    dplyr::distinct(),
  # find all haul events (hauljoins), by survey
  y = dat_haul_api |>
    dplyr::select(survey_definition_id, hauljoin) |>
    dplyr::distinct(),
  relationship = "many-to-many",
  by = "survey_definition_id"
) |>
  dplyr::select(-survey_definition_id) # now, redundant
```

Explore the data contents:

```
print(paste0("rows: ", nrow(comb), "; cols: ", ncol(comb)))

comb |> head(3) |>
  flextable::flextable() |>
  flextable::colformat_num(
    j = c("species_code", "hauljoin"),
    big.mark = "") |>
  flextable::theme_zebra()
```

Now, using that table of combinations (here, called `comb`), join data to make a full zero-filled CPUE dataset. When all of the data have been full joined together, there should be the maximum number of rows in `comb`.

12. Access via API and R

```
dat <- comb |>
  # add species data
  dplyr::left_join(dat_species_api) |> # , "species_code"
  # add haul data
  dplyr::left_join(dat_haul_api) |> # , c("hauljoin")
  # add catch data
  dplyr::left_join(dat_catch_api) |> # , c("species_code", "hauljoin")
  # modify/clean up zero-filled rows
  dplyr::mutate(
    cpue_kgkm2 = ifelse(is.na(cpue_kgkm2), 0, cpue_kgkm2),
    cpue_nokm2 = ifelse(is.na(cpue_nokm2), 0, cpue_nokm2),
    count = ifelse(is.na(count), 0, count),
    weight_kg = ifelse(is.na(weight_kg), 0, weight_kg))
```

Explore the data contents:

```
# Find how many rows and columns are in the data pull
print(paste0("rows: ", nrow(dat), "; cols: ", ncol(dat)))

# learn about the structure of the data
summary(dat)

# Print the first few lines of the data
dat |>
  head(3) |>
  flextable::flextable() |>
  flextable::colformat_num(
    j = c("species_code", "hauljoin", "year", "cruise", "cruisejoin", "worms", "itis"), #
    big.mark = "" ) |>
  flextable::theme_zebra()

# save outputs for later comparison
dat_zerofill_api <- dat
```

12.3. Ex. Visualize zero-filled data for 2023 eastern Bering Sea walleye pollock in CPUE data in distribution map

Using the zero-filled data from the previous example, we can make a few plots!

12. Access via API and R

Here is some example data of 2023 through 2019 (year %in% 2019:2023) eastern and northern Bering Sea (srvy %in% c("EBS", "NBS)) walleye pollock (species_code == 21740).

```
dat <- dat_zerofill_api |>
  dplyr::filter(year %in% 2019:2023 &
                srvy %in% c("EBS", "NBS") &
                species_code == 21740) |>
  dplyr::select(year, common_name, longitude_dd_start, latitude_dd_start, cpue_kgkm2)

# Find how many rows and columns are in the data pull
print(paste0("rows: ", nrow(dat), "; cols: ", ncol(dat)))

# # learn about the structure of the data
# summary(dat)

# Print the first few lines of the data
dat |>
  head(3) |>
  flextable::flextable() |>
  flextable::colformat_num(
    j = c("year"),
    big.mark = "" ) |>
  flextable::theme_zebra()
```

12.3.1. Plot locations on map

```
library(ggplot2)

ggplot2::ggplot(data = dat |> dplyr::filter(cpue_kgkm2 != 0),
  mapping = aes(x = longitude_dd_start,
                y = latitude_dd_start,
                size = cpue_kgkm2)) +
  ggplot2::geom_point(alpha = .75) +
  ggplot2::geom_point(data = dat |> dplyr::filter(cpue_kgkm2 == 0),
    color = "red",
    shape = 17,
    alpha = .75,
```

12. Access via API and R

```
      size = 3) +
ggplot2::xlab("Longitude *W") +
ggplot2::ylab("Latitude *N") +
ggplot2::ggtitle(label = "CPUE (kg/km2) of walleye pollock (Weight CPUE; kg/km2)",
                 subtitle = "Eastern Bering Sea bottom trawl survey") +
ggplot2::scale_size_continuous(name = "Weight (kg)") +
ggplot2::facet_wrap(facets = vars(year)) +
ggplot2::theme_bw()
```

12.3.2. Plot inverse-distance weighted plot of CPUE

This map is constructed using `akgfmmaps`. To make IDW plots, you must have data from all stations surveyed, even if no fish of interest were found there.

These plots are similar to those published in the annual Bering Sea data reports.

```
# pak::pak("afsc-gap-products/akgfmmaps")
library(akgfmmaps)
idw <- akgfmmaps::make_idw_stack(
  x = dat |>
    dplyr::select(COMMON_NAME = common_name,
                  CPUE_KGHA = cpue_kgkm2,
                  LATITUDE = latitude_dd_start,
                  LONGITUDE = longitude_dd_start,
                  year),
  grouping.vars = "year",
  region = "bs.all", # Predefined EBS area
  set.breaks = "jenks", # Gets Jenks breaks from classint::classIntervals()
  in.crs = "+proj=longlat", # Set input coordinate reference system
  out.crs = "EPSG:3338", # Set output coordinate reference system
  extrapolation.grid.type = "sf")

shps <- akgfmmaps::get_base_layers(
  select.region = "bs.all",
  # include.corners = TRUE,
  set.crs = "EPSG:3338")

shps$survey.area$SRVY <- c("EBS", "NBS")
shps$survey.area$SURVEY <- c("EBS", "NBS")
```

12. Access via API and R

```
# set.breaks <- akgfmmaps::eval_plot_breaks(CPUE = dat$cpue_kgkm2, n.breaks = 5)
# set.breaks <- as.vector(unlist(set.breaks[set.breaks$style == "pretty", -1]))
set.breaks <- c(0, 50000, 100000, 150000, 200000, 250000)

figure_print <- ggplot() +
  # add map of alaska
  ggplot2::geom_sf(data = shps$akland,
                   color = NA,
                   fill = "grey50") +
  # add IDW plots
  geom_sf(data = idw$extrapolation.stack,
          mapping = aes(fill = var1.pred),
          na.rm = FALSE,
          show.legend = TRUE,
          color = NA) +
  ggplot2::scale_fill_manual(
    name = "walleye pollock\nCPUE (kg/km2)",
    values = c("gray90",
               viridis::viridis(
                 option = "mako",
                 direction = -1,
                 n = length(set.breaks)-1,
                 begin = 0.20,
                 end = 0.80)),
    na.translate = FALSE, # Don't use NA
    drop = FALSE) +
  # separate plots by year
  ggplot2::facet_wrap(facets = vars(year), nrow = 2) +
  # add survey area
  ggplot2::geom_sf(
    data = shps$survey.area,
    mapping = aes(color = SURVEY,
                  geometry = geometry),
    fill = "transparent",
    linewidth = 1,
    show.legend = FALSE) +
  ggplot2::scale_color_manual(
    name = " ",
    values = c("grey30", "grey50"),
    breaks = shps$survey.area$SURVEY,
```

12. Access via API and R

```
labels = shps$survey.area$SRVY) +
# lat/lon axis and map bounds
ggplot2::scale_x_continuous(name = "Longitude °W",
                             breaks = seq(-180, -150, 5)) +
ggplot2::scale_y_continuous(name = "Latitude °N",
                             breaks = seq(50, 65, 5)) + # seq(52, 62, 2)
ggplot2::coord_sf(xlim = sf::st_bbox(shps$survey.area)[c(1,3)],
                  ylim = sf::st_bbox(shps$survey.area)[c(2,4)]) +
# add theme aesthetics
ggplot2::guides(
  fill = guide_legend(
    order = 1,
    title.position = "top",
    label.position = "bottom",
    title.hjust = 0.5,
    override.aes = list(color = NA),
    nrow = 1),
  color = "none") +
ggplot2::theme(
  panel.background = element_rect(fill = "white", colour = NA),
  panel.border = element_rect(fill = NA, colour = "grey20"),
  strip.background = element_blank(),
  strip.text = element_text(size = 10, face = "bold"),
  legend.text = element_text(size = 9),
  legend.background = element_rect(colour = "transparent",
                                    fill = "transparent"),
  legend.key = element_rect(colour = "transparent",
                             fill = "transparent"),
  legend.position = "bottom",
  legend.box = "horizontal",
  legend.box.spacing = unit(0, "pt"), # reduce space between legend & plot
  legend.margin=margin(0, 0, 0, 0) )

figure_print
```

12.4. Ex. Show catch data for 2023 eastern Bering Sea Walleye Pollock (one species in one survey region in one year)

Data downloads and joins for just one species, survey, and year are much faster and easier to do.

First, because `year` is identified in the haul table, we need to identify all of the hauls (or more specifically, `hauljoin` codes) that were completed in the eastern Bering Sea (`"srvy": "EBS"`) in 2023 (`"year": 2023`).

Note: Check how many rows and columns are in the data pull. The eastern Bering Sea survey (before 2024) has 376 stations in it, and pollock are often found in throughout the region so this should have a similar number of rows.

```
## query the API link
res <- httr::GET(url = paste0(api_link_haul, '?limit=10000&q={"year":2023,"srvy":"EBS"}'))

## convert from JSON format
data <- jsonlite::fromJSON(base::rawToChar(res$content))
dat <- data$items |>
  dplyr::select(-links) # necessary for API accounting, but not part of the dataset

## show summary of data to make sure it is subset correctly
summary(dat |> dplyr::mutate(srvy = as.factor(srvy)))

## Find how many rows and columns are in the data pull.
print(paste0("rows: ", nrow(dat), "; cols: ", ncol(dat)))

# save outputs for later comparison
dat_haul_ex <- dat

# Print the first few lines of the data
dat_haul_ex |>
  head(3) |>
  flextable::flextable() |>
  flextable::colformat_num(
    j = c("year", "hauljoin", "cruise"),
    big.mark = "" ) |>
  flextable::theme_zebra()
```

12.4.1. Identify species_code for walleye pollock

In the catch data, we itemize species catches by species_code. To find out which species_code to use, you can check variations on the following code. Note that here the word pollock is case sensitive. All species common_name entries are lower case except for proper nouns (e.g., “Pacific”). The notation for finding a string is to use % around the phrase. Since % is a reserved character in a URL, you have to replace % with %25. Similarly, %20 needs to be used in place of a space (e.g., between “walleye” and “pollock”: “walleye%20pollock”}’).

```
## query the API link. Use:
res <- httr::GET(url = paste0(api_link_species, '?q={%22common_name%22:%22walleye%20pollock%22}')
# OR
res <- httr::GET(url = paste0(api_link_species, '?q={"common_name":{"$like":"%25pollock%25'})
# OR
res <- httr::GET(url = paste0(api_link_species, '?q={"common_name":"walleye%20pollock"}'))

## convert from JSON format
data <- jsonlite::fromJSON(base::rawToChar(res$content))

# save outputs for later comparison
dat_species_ex <- data$items |> dplyr::select(-links) # necessary for API accounting, but n

# Print the first few lines of the data
dat_species_ex |>
  head(3) |>
  flextable::flextable() |>
  flextable::colformat_num(
    j = c("species_code"),
    big.mark = "" ) |>
  flextable::theme_zebra()
```

12.4.2. Then, apply the hauljoins and species_code to catch query

We’ll use the data from the haul and species table we collected before to select 2023 eastern Bering Sea walleye pollock catch data.

12. Access via API and R

```
## query the API link
# data for all walleye pollock caught in all 2023 eastern Bering Sea survey hauls
dat <- data.frame()
# there must be a better way to select multiple values for one parameter,
# but saving that, we will loop through each hauljoin and collect the data of interest
for (i in 1:nrow(dat_haul_ex)) {
  res <- httr::GET(url = paste0(
    api_link_catch,
    '?q={"species_code":21740,"hauljoin":', dat_haul_ex$hauljoin[i],'}'))
  ## convert from JSON format
  data <- jsonlite::fromJSON(base::rawToChar(res$content))
  if (length(data$items) != 0) {
    dat <- dplyr::bind_rows(
      dat,
      data$items |>
        dplyr::select(-links)) # necessary for API accounting, but not part of the dataset
  }
}
```

Explore data:

```
# Find how many rows and columns are in the data pull
print(paste0("rows: ", nrow(dat), "; cols: ", ncol(dat)))

# learn about the structure of the data
summary(dat)

# Print the first few lines of the data
dat |>
  head(3) |>
  flextable::flextable() |>
  flextable::colformat_num(
    j = c("hauljoin", "species_code"),
    big.mark = "") |>
  flextable::theme_zebra()

# save outputs for later comparison
dat_catch_ex <- dat
```

For reference and to help break down the above query, see these other query examples:

12. Access via API and R

```
# data for haul -22775 (i.e., one specific haul)?
res <- httr::GET(url = paste0(api_link_catch,
                              '?offset=', i, '&limit=10000&q={"hauljoin":-22775}'))

# data for all walleye pollock (i.e., one species) caught in all years and surveys
res <- httr::GET(url = paste0(api_link_catch,
                              '?offset=', i, '&limit=10000&q={"species_code":21740}'))
```

12.4.3. Create zero-filled data for 2023 eastern Bering Sea walleye pollock and plot

It is important to create and have access to zero-fill (presence and absence) so you can do simple analyses and plot data.

```
dat <- dplyr::full_join(
  dat_haul_ex,
  dat_catch_ex) |>
dplyr::full_join(
  dat_species_ex) |>
# modify zero-filled rows
dplyr::mutate(
  cpue_kgkm2 = ifelse(is.na(cpue_kgkm2), 0, cpue_kgkm2),
  cpue_nokm2 = ifelse(is.na(cpue_nokm2), 0, cpue_nokm2),
  count = ifelse(is.na(count), 0, count),
  weight_kg = ifelse(is.na(weight_kg), 0, weight_kg))
```

Explore data

```
# Find how many rows and columns are in the data pull
print(paste0("rows: ", nrow(dat), "; cols: ", ncol(dat)))

# learn about the structure of the data
summary(dat)

# Print the first few lines of the data
dat |>
  head(3) |>
  flextable::flextable() |>
```

```
flextable::colformat_num(
  j = c("year", "cruise", "cruisejoin", "species_code"),
  big.mark = "") |>
flextable::theme_zebra()
```

12.4.4. Visualize CPUE data in distribution map

Using the zero-filled data from the previous example, we can make a few plots!

12.5. Plot locations

```
library(ggplot2)

ggplot2::ggplot(data = dat |> dplyr::filter(cpue_kgkm2 != 0),
  mapping = aes(x = longitude_dd_start,
    y = latitude_dd_start,
    size = cpue_kgkm2)) +
  ggplot2::geom_point(alpha = .75) +
  ggplot2::geom_point(data = dat |> dplyr::filter(cpue_kgkm2 == 0),
    color = "red",
    shape = 17,
    alpha = .75,
    size = 3) +
  ggplot2::xlab("Longitude *W") +
  ggplot2::ylab("Latitude *N") +
  ggplot2::ggtitle(label = "Catches of walleye pollock (Weight CPUE; kg/km2)",
    subtitle = "2023 eastern Bering Sea bottom trawl survey") +
  ggplot2::scale_size_continuous(name = "Weight (kg)") +
  ggplot2::theme_bw()
```

12.5.1. Plot inverse-distance weighted modeled product of locations

This map is constructed using `akgfmmaps`

12. Access via API and R

```
# pak::pak("afsc-gap-products/akgfmmaps")
library(akgfmmaps)

figure0 <- akgfmmaps::make_idw_map(
  CPUE_KGHA = dat$cpue_kgkm2, # calculates the same, regardless of units.
  LATITUDE = dat$latitude_dd_start,
  LONGITUDE = dat$longitude_dd_start,
  region = "bs.south", # Predefined EBS area
  set.breaks = "jenks", # Gets Jenks breaks from classint::classIntervals()
  in.crs = "+proj=longlat", # Set input coordinate reference system
  out.crs = "EPSG:3338", # Set output coordinate reference system
  extrapolation.grid.type = "sf")

figure0$plot + # 20x20km grid
  ggplot2::guides(fill=guide_legend(title = "walleye pollock\nCPUE (kg/km2)"))
```

13. Access via API and Python

13.0.1. {afscgap} Library Installation

author: Sam Pottinger (sam.pottinger@berkeley.edu; GitHub::sampottinger)
date: May 13, 2023

The third-party `afscgap` Python package interfaces with FOSS to access AFSC GAP data. It can be installed via pip:

```
#The reticulate package provides a comprehensive set of tools for interoperability between  
library(reticulate)
```

```
pip install afscgap  
pip install git+https://github.com/SchmidtDSE/afscgap.git@main
```

For more information on installation and deployment, see the library documentation.

13.0.2. Basic query

This first example queries for Pacific glass shrimp (*Pasiphaea pacifica*) in the Gulf of Alaska in 2021. The library will automatically generate HTTP queries, converting from Python types to ORDS query syntax.

```
import afscgap  
  
query = afscgap.Query()  
query.filter_year(eq=2021)  
query.filter_srvy(eq='GOA')  
query.filter_scientific_name(eq='Pasiphaea pacifica')  
  
results = query.execute()
```

The `results` variable in this example is an iterator that will automatically perform pagination behind the scenes.

13.0.3. Iterating with a for loop

The easiest way to interact with results is a simple for loop. This next example determines the frequency of different catch per unit effort where Pacific glass shrimp were reported:

```
import afscgap

# Mapping from CPUE to count
count_by_cpue = {}

# Build query
query = afscgap.Query()
query.filter_year(eq=2021)
query.filter_srvy(eq='GOA')
query.filter_scientific_name(eq='Pasiphaea pacifica')
results = query.execute()

# Iterate through results and count
for record in results:
    cpue = record.get_cpue_weight(units='kg/ha')
    cpue_rounded = round(cpue)
    count = count_by_cpue.get(cpue_rounded, 0) + 1
    count_by_cpue[cpue_rounded] = count

# Print the result
print(count_by_cpue)
```

Note that, in this example, only records with Pacific glass shrimp are included (“presence-only” data). See zero catch inference below. In other words, it reports on CPUE only for hauls in which Pacific glass shrimp were recorded, excluding some hauls like those in which Pacific glass shrimp were not found at all.

13.0.4. Iterating with functional programming

A for loop is not the only option for iterating through results. List comprehensions and other functional programming methods can be used as well.

13. Access via API and Python

```
import statistics

import afscgap

# Build query
query = afscgap.Query()
query.filter_year(eq=2021)
query.filter_srvy(eq='GOA')
query.filter_scientific_name(eq='Pasiphaea pacifica')
results = query.execute()

# Get temperatures in Celsius
temperatures = [record.get_bottom_temperature(units='c') for record in results]

# Take the median
print(statistics.median(temperatures))
```

This example reports the median temperature in Celcius for when Pacific glass shrimp was reported.

13.0.5. Load into Pandas

The results from the `afscgap` package are serializable and can be loaded into other tools like Pandas. This example loads Pacific glass shrimp from 2021 Gulf of Alaska into a data frame.

```
import pandas

import afscgap

query = afscgap.Query()
query.filter_year(eq=2021)
query.filter_srvy(eq='GOA')
query.filter_scientific_name(eq='Pasiphaea pacifica')
results = query.execute()

pandas.DataFrame(results.to_dicts())
```

13. Access via API and Python

Specifically, `to_dicts` provides an iterator over a dictionary form of the data that can be read into tools like Pandas.

13.0.6. Advanced filtering

Queries so far have focused on filters requiring equality but range queries can be built as well.

```
import afscgap

# Build query
query = afscgap.Query()
query.filter_year(min_val=2015, max_val=2019)  # Note min/max_val
query.filter_srvy(eq='GOA')
query.filter_scientific_name(eq='Pasiphaea pacifica')
results = query.execute()

# Sum weight
weights = map(lambda x: x.get_weight(units='kg'), results)
total_weight = sum(weights)
print(total_weight)
```

This example queries for Pacific glass shrimp data between 2015 and 2019, summing the total weight caught. Note that most users will likely take advantage of built-in Python to ORDS query generation which dictates how the library communicates with the API service. However, users can provide raw ORDS queries as well using manual filtering.

13.0.7. Zero-catch inference

Until this point, these examples use presence-only data. However, the `afscgap` package can infer negative or “zero catch” records as well.

```
import afscgap

# Mapping from CPUE to count
count_by_cpue = {}
```


13. Access via API and Python

```
# Build query
query = afscgap.Query()
query.filter_year(eq=2021)
query.filter_srvy(eq='GOA')
query.filter_scientific_name(eq='Pasiphaea pacifica')
query.set_presence_only(False) # Added to earlier example
results = query.execute()

# Iterate through results and count
for record in results:
    cpue = record.get_cpue_weight(units='kg/ha')
    cpue_rounded = round(cpue)
    count = count_by_cpue.get(cpue_rounded, 0) + 1
    count_by_cpue[cpue_rounded] = count

# Print the result
print(count_by_cpue)
```

This example revisits the earlier snippet for CPUE counts but `set_presence_only(False)` directs the library to look at additional data on hauls, determining which hauls did not have Pacific glass shrimp. This lets the library return records for hauls in which Pacific glass shrimp were not found. This can be seen in differences in counts reported:

Rounded CPUE	Count with set_presence_only(True)	Count with set_presence_only(False)
0 kg/ha	44	521
1 kg/ha	7	7
2 kg/ha	1	1

Put simply, while the earlier example showed CPUE counts for hauls in which Pacific glass shrimp were seen, this revised example reports for all hauls in the Gulf of Alaska in 2021.

13.0.8. More information

Please see the API documentation for the Python library for additional details.

Part III.

Data Products & Tools

To accompany these data, we also produce data products to make using our data more accessible and straightforward.

Table 13.2.: Survey of products developed by GAP

Product	Point of Contact AI	Point of Contact GOA	Point of Contact BS	Descr
<i>Data</i>				
Finalized bottom trawl data	Susanne McDermott	Ned Laman	Duane Stevenson	NOAA trawl o post-s
Data requests	Alexandra Dowlin		Chris Anderson	To rec NOAA trawl r
Species codebook	Chris Anderson			List of inverte NOAA Division
Survey protocols	Em?			Docur NOAA ground
<i>Analysis</i>				
Design-based indices for target species	Susanne McDermott	Ned Laman	Duane Stevenson	Stand biom NOAA trawl s
Design-based age or length composition	Susanne McDermott	Ned Laman	Duane Stevenson	Stand and a NOAA trawl s
Model-based indices, age comps (stock assessment), area occupied, and COG (ESP)	Lewis Barnett			Spatio indice comp NOAA trawl s

Product	Point of Contact AI	Point of Contact GOA	Point of Contact BS	Descr
Annual bottom and surface temperature summary (ESR, stock assessment)	Rebecca Howard		Sean Rohan & Lewis Barnett	Summ and s histori
Bering Sea cold pool index and temperature data products (ESR, ESP, stock assessment)	-		Sean Rohan & Lewis Barnett	Create the EE index and p
Annual fish condition (ESR)	Rebecca Howard, Sean Rohan, & Bianca Prohaska	Rebecca Howard & Bianca Prohaska	Bianca Prohaska & Sean Rohan	Groun fish in and G
Rockfish indices vs environmental gradients (ESR)	Alexandra Dowlin & Christina Conrath		-	GOA/ abund enviro Pacifi
Structure-Forming Invertebrates-Habitat Areas of Particular Concern (SFI-HAPC) (ESR)	Christina Conrath		Thaddeus Buser	Relati hydro anemo and A
Forage fishes (ESR)	-	Megsie Siple	-	Relati eulack pricke
Miscellaneous species (ESR)	Sarah Friedman		Thaddeus Buser	Relati poach and A
Jellies (ESR)	Alexandra Dowlin		Thaddeus Buser	Relati and A
Essential fish habitat	Megsie Siple		Sean Rohan	Habita based Updat
<i>Visualization Tools</i>				

Product	Point of Contact AI	Point of Contact GOA	Point of Contact BS	Descr
Alaska groundfish maps (CPUE, etc.)	Megsie Siple		Sean Rohan	
<i>Communication</i>				
Annual survey data report	Megsie Siple, Bethany Riggle, Alex Dowlin		Emily Markowitz, Sophia Wassermann, Nicole Charriere, Chris Anderson	Alaska Techn survey availa public below (https
ADF&G report of research activities	Alexandra Dowlin		Nicole Charriere & Rebecca Haehn	Report fishing Alaska
IPHC report of research activities	Ned Laman		Rebecca Haehn	
Plan team survey results presentation	Megsie Siple, Susanne McDermott	Megsie Siple, Ned Laman	Duane Stevenson	NOAA their fi Group record here: couno groun
Community highlights report	Susanne McDermott		Emily Markowitz	Comp NOAA finding
Bottom Trawl Survey Temperature and Progress Maps	Ned Laman		Emily Markowitz	Near ocean Aleutia Bering

14. Open source code

14.1. R Packages

14.1.1. akgfmaps R package

Bottom trawl survey maps layers and plotting examples. **POC:** Sean Rohan

14.1.2. coldpool R package

Cold pool area and temperature data products for the Bering Sea. **POC:** Sean Rohan

14.1.3. akfishcondition R package

Groundfish morphometric condition indicators for fish in the Bering Sea, Aleutian Islands, and Gulf of Alaska. **POC:** Sean Rohan

14.1.4. gapindex R package

Calculation of Design-Based Indices of Abundance and Composition for AFSC GAP Bottom Trawl Surveys. **POC:** Zack Oyafuso and Margaret Siple

Part IV.

Contact us

This code is primarily maintained by:

Thank you for using our data guide!

This code is always in development. Find code used for various reports in the code releases.

This code is primarily maintained by:

Emily Markowitz (Emily.Markowitz AT noaa.gov; @EmilyMarkowitz-NOAA)

Zack Oyafuso (Zack.Oyafuso AT noaa.gov; @zoyafuso-NOAA)

Sarah Friedman (Sarah.Friedman AT noaa.gov; @SarahFriedman-NOAA)

Alaska Fisheries Science Center,

National Marine Fisheries Service,

National Oceanic and Atmospheric Administration,

Seattle, WA 98195

General questions and more specific data requests can be sent to nmfs.afsc.gap.metadata@noaa.gov or submitted as an issue on our GitHub Organization. The version of this data used for stock assessments can be found through the Alaska Fisheries Information Network (AKFIN). For questions about the eastern Bering Sea surveys, contact Duane Stevenson (Duane.Stevenson@noaa.gov). For questions about the Gulf of Alaska or Aleutian Islands surveys, contact Ned Laman (Ned.Laman@noaa.gov). For questions specifically about crab data in any region, contact Mike Litzow (Mike.Litzow@noaa.gov), the Shellfish Assessment Program lead.

For questions, comments, and concerns specifically about the Fisheries One Stop Shop (FOSS) platform, please contact us using the Comments page on the FOSS webpage.

15. Production run notes

Report run date: Tuesday, July 07, 2026

16. R Version Metadata

```
R version 4.5.3 (2026-03-11 ucrt)
Platform: x86_64-w64-mingw32/x64
Running under: Windows 11 x64 (build 22631)
```

```
Matrix products: default
  LAPACK version 3.12.1
```

```
locale:
[1] LC_COLLATE=English_United States.utf8
[2] LC_CTYPE=English_United States.utf8
[3] LC_MONETARY=English_United States.utf8
[4] LC_NUMERIC=C
[5] LC_TIME=English_United States.utf8
```

```
time zone: America/Los_Angeles
tzcode source: internal
```

```
attached base packages:
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

```
loaded via a namespace (and not attached):
 [1] compiler_4.5.3    fastmap_1.2.0     cli_3.6.6         tools_4.5.3
 [5] htmltools_0.5.9   otel_0.2.0        rstudioapi_0.19.0 yaml_2.3.12
 [9] tinytex_0.60      rmarkdown_2.31    knitr_1.51        jsonlite_2.0.0
[13] xfun_0.57         digest_0.6.39     rlang_1.2.0       evaluate_1.0.5
```

16.0.1. NOAA README

This repository is a scientific product and is not official communication of the National Oceanic and Atmospheric Administration, or the United States Department of Commerce. All NOAA GitHub project code is provided on an ‘as is’ basis and the user assumes responsibility for its use. Any claims against the Department of Commerce or

16. R Version Metadata

Department of Commerce bureaus stemming from the use of this GitHub project will be governed by all applicable Federal law. Any reference to specific commercial products, processes, or services by service mark, trademark, manufacturer, or otherwise, does not constitute or imply their endorsement, recommendation or favoring by the Department of Commerce. The Department of Commerce seal and logo, or the seal and logo of a DOC bureau, shall not be used in any manner to imply endorsement of any commercial product or activity by DOC or the United States Government.

16.0.2. NOAA License

Software code created by U.S. Government employees is not subject to copyright in the United States (17 U.S.C. §105). The United States/Department of Commerce reserve all rights to seek and obtain copyright protection in countries other than the United States for Software authored in its entirety by the Department of Commerce. To this end, the Department of Commerce hereby grants to Recipient a royalty-free, nonexclusive license to use, copy, and create derivative works of the Software outside of the United States.

17. Acknowledgments

18. Community Acknowledgments

We would like to thank the many communities of Alaska and their members who have helped contribute to this body of work. The knowledge, experiences, and insights have been instrumental in expanding the scope of our science and knowledge to encompass the many issues that face this important ecosystem. We appreciate feedback from those residing in the region that are willing to share their insights and participation in an open dialog about how we can improve our collective knowledge of the ecosystem and the region.

19. Land Acknowledgements

We would like to thank the many communities of the Bering Strait region and their members who have helped contribute to this document. The knowledge, experiences, and insights of the people of the Bering Strait region have been instrumental in expanding the scope of our science and knowledge to encompass the many issues that face this important ecosystem. We appreciate feedback from those residing in the region that are willing to share their insights, including the local names used for the species covered by this document, identifying species of interest or concern that should be included in this document, and participation in an open dialog about how we can improve our collective knowledge of the ecosystem and the region.

NOAA Fisheries Alaska Fisheries Science Center's work is conducted in the waters and along the coastlines of Alaska, which include the traditional home lands and waters of the Inupiat, Yupiit, Siberian Yupiit, Unangax, Alutiiq/Sugpiaq, Eyak, Dena'ina Athabascan, Tlingit, Haida, and Tsimshian who have stewarded their lands and waters since time immemorial. We are indebted to these peoples for their wisdom and knowledge of their lands and waters.

This document was prepared in the greater Seattle area, which are the traditional lands of the Coast Salish people, including the Duwamish people, past and present. We are grateful for their continued sharing of vision, wisdom, values, and leadership.

20. Technical Acknowledgments

This quarto book is based off the NOAA-quarto-book GitHub repo designed by Eli Holmes.

This repo and GitHub Action was based on the tutorial by Openscapes quarto-website-tutorial by Julia Lowndes and Stefanie Butland.

20.1. Partners

Scientists from the Alaska Fisheries Science Center conduct these bottom trawl surveys with participation from the Alaska Department of Fish & Game (ADF&G), the International Pacific Halibut Commission (IPHC), and universities. This research is conducted on chartered fishing vessels.

20.2. Collaborators

Our data are used in many annual publications, including but not limited to the list below:

- Alaska Stock Assessments
- North Pacific Groundfish Stock Assessment and Fishery Evaluation Reports
- Groundfish Economic Status Reports for the Gulf of Alaska and Bering Sea and Aleutian Islands
- Alaska Marine Ecosystem Status Report Database
- Southeast Alaska Coastal Monitoring Survey Reports
- Alaska Fisheries Life History Database
- Essential Fish Habitat Research Plan in Alaska

21. Citations and References

22. Access Constraints

There are no legal restrictions on access to the data. They reside in public domain and can be freely distributed.

User Constraints: Users must read and fully comprehend the metadata prior to use. Data should not be used beyond the limits of the source scale. Acknowledgement of AFSC Groundfish Assessment Program, as the source from which these data were obtained, in any publications and/or other representations of these data, is suggested.

23. References

- Alaska Fisheries Information Network (AKFIN). (2024). *AFSC groundfish assessment program design-based production data*. NOAA Fisheries Alaska Fisheries Science Center, Groundfish Assessment Program; <https://akfinbi.psmfc.org/analytics/>; U.S. Dep. Commer. <https://www.psmfc.org/program/alaska-fisheries-information-network-akfin>
- Hoff, G. R. (2016). *Results of the 2016 eastern Bering Sea upper continental slope survey of groundfishes and invertebrate resources* (NOAA Tech. Memo. NOAA-AFSC-339). U.S. Dep. Commer. <https://doi.org/10.7289/V5/TM-AFSC-339>
- Markowitz, E. H., Dawson, E. J., Wassermann, S., Anderson, C. B., Rohan, S. K., Charriere, B. K., and Stevenson, D. E. (2024). *Results of the 2023 eastern and northern Bering Sea continental shelf bottom trawl survey of groundfish and invertebrate fauna* (NOAA Tech. Memo. NMFS-AFSC-487; p. 242). U.S. Dep. Commer. <https://doi.org/10.25923/2mry-yx09>
- Markowitz, E. H., Wassermann, S., Rohan, S. K., Charriere, B. K., Anderson, C. B., and Stevenson, D. E. (2025). *Results of the 2024 eastern and northern Bering Sea continental shelf bottom trawl survey of groundfish and invertebrate fauna* (NOAA Tech. Memo. NMFS-AFSC-499; p. 203). U.S. Dep. Commer. <https://doi.org/10.25923/8qa3-x785>
- NOAA Fisheries Alaska Fisheries Science Center. (2024). *Fisheries one stop shop public data: RACE division bottom trawl survey data query*. <https://www.fisheries.noaa.gov/foss>; U.S. Dep. Commer.
- NOAA Fisheries Alaska Fisheries Science Center, Groundfish Assessment Program. (2024). *AFSC groundfish assessment program design-based production data*. <https://www.fisheries.noaa.gov/alaska/science-data/groundfish-assessment-program-bottom-trawl-surveys>; U.S. Dep. Commer.
- Siple, M. C., Szalay, P. G. von, Raring, N. W., Dowlin, A. N., and Riggle, B. C. (2024). *Data report: 2023 gulf of alaska bottom trawl survey* (NOAA Tech. Memo. AFSC processed report; 2024-09). U.S. Dep. Commer. <https://doi.org/10.25923/gbb1-x748>
- Von Szalay, P. G., Raring, N. W., Siple, M. C., Dowlin, A. N., Riggle, B. C., and Laman, E. A. and. (2023). *Data report: 2022 Aleutian Islands bottom trawl survey* (AFSC Processed Rep. 2023-07; p. 230). U.S. Dep. Commer. <https://doi.org/10.25923/85cy-g225>